NEURAL NETWORK Part II

Neural Network Architectures CNN, RNN, Attention

Recap from last time

- DNN
- Training & Optimizing NN

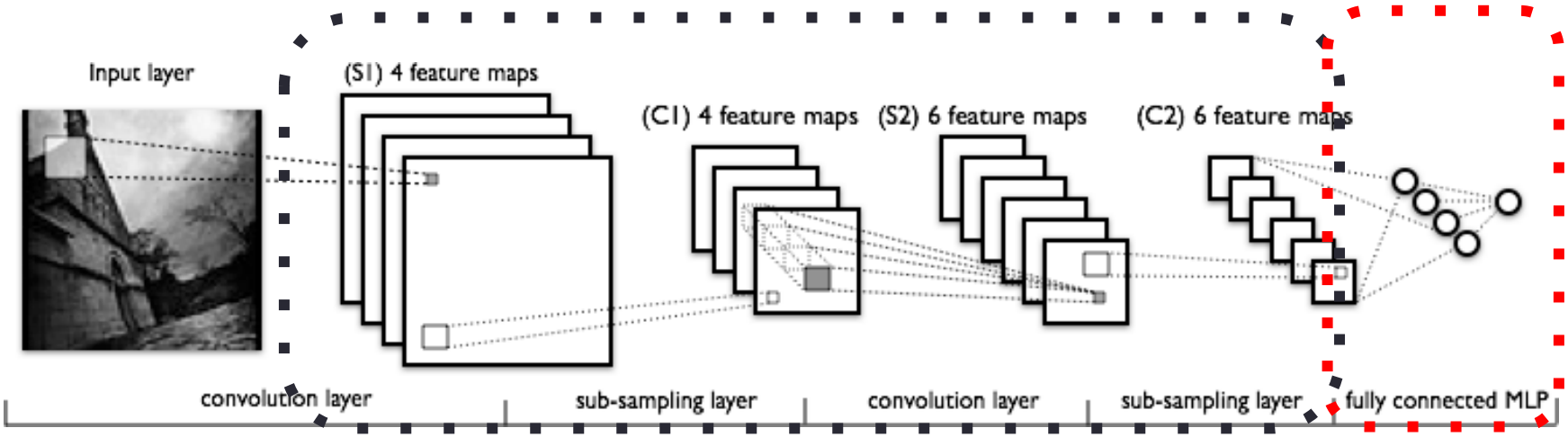
Tips to tune

- Feeling and experience ¯\(°_o)/¯
- Take numbers from papers
- Grid search
  - Heuristic search
  - Random search
  - Genetic Algorithm
  - Bayesian optimization (optuna)
- Picking the right type of model is more important than picking the right number of neurons
  - Inductive bias
- Tips for debugging (will make more sense next class)
  - http://karpathy.github.io/2019/04/25/recipe/

Neural networks

- Layers
  - CNN
  - RNN, LSTM, GRU
  - Attention
- Architectures

Convolution Neural Network (CNN)

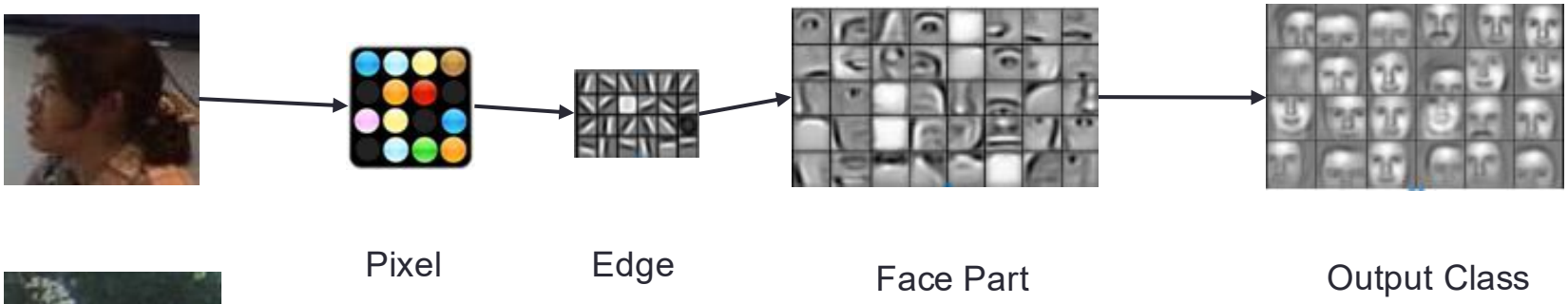


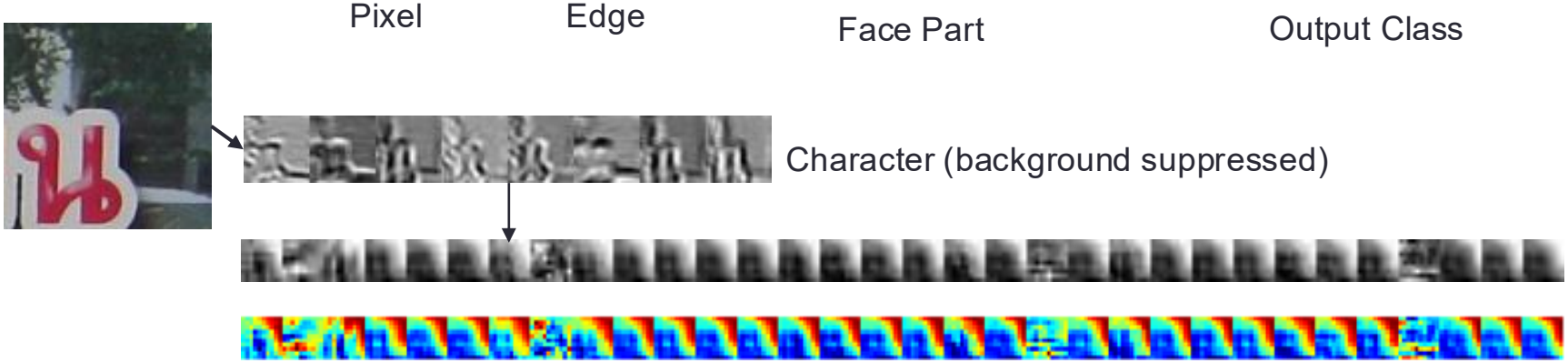
Trainable Feature Extractor Layer **Classification Layer**

Convolution layers followed by sub-sampling (pooling) layers
Output of each layer is called a feature map.

Convolution Neural Network (CNN)

- Hierarchy of representations with increasing level of abstraction
- Each stage is a kind of trainable feature transform
- Image recognition: Pixel →edge → texture → part → object

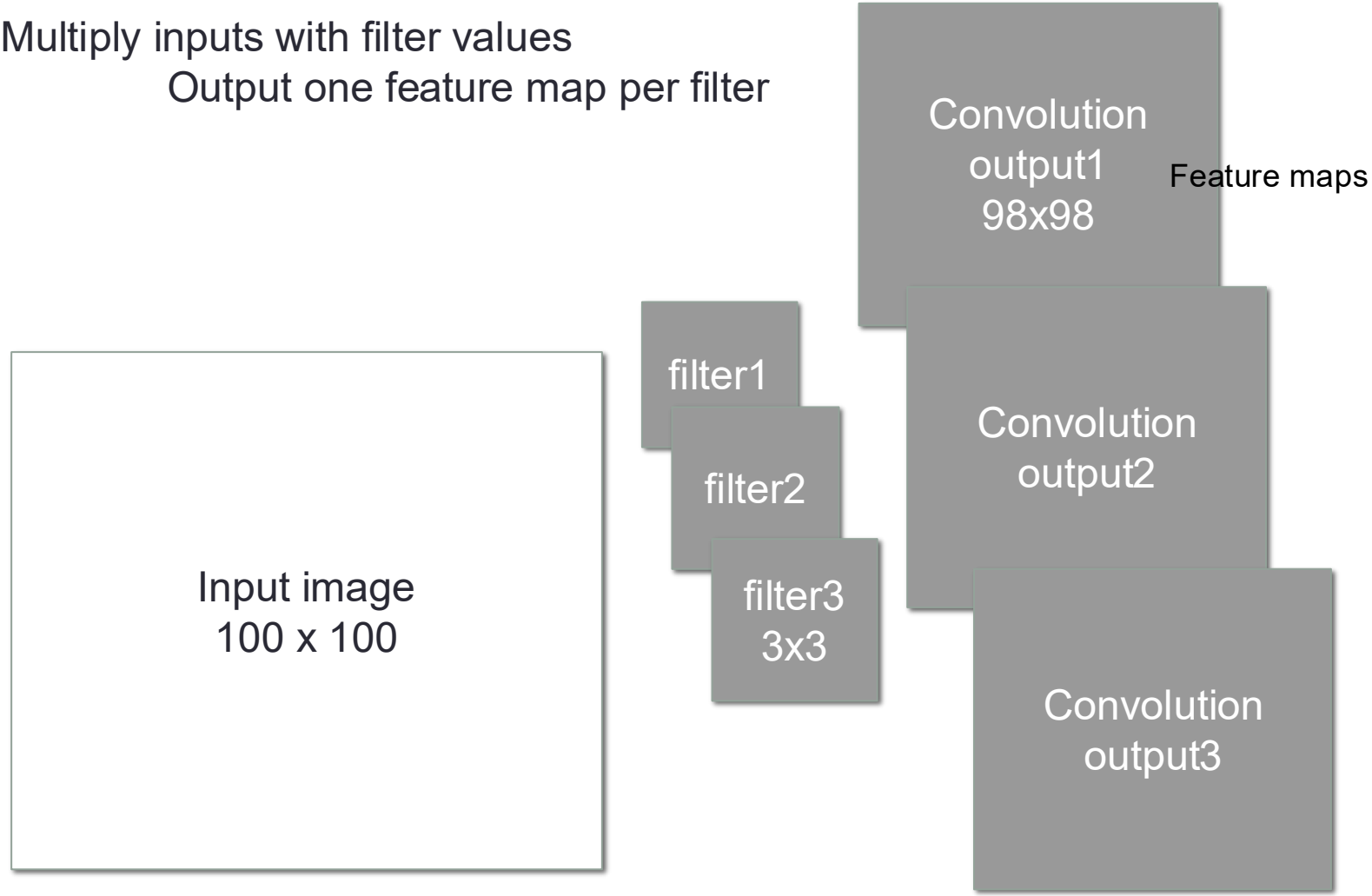




Convolution filters

Multiply inputs with filter values

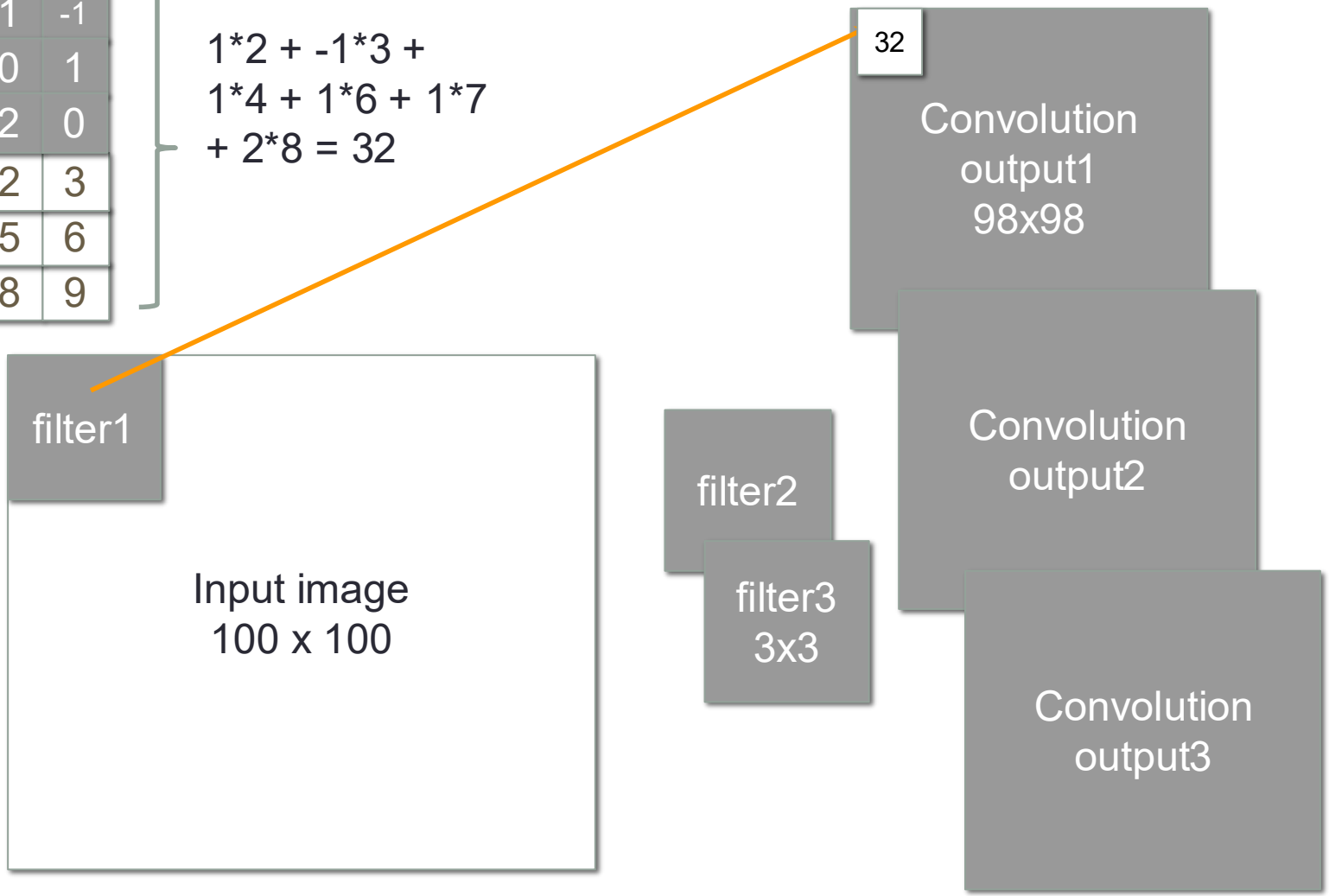
Output one feature map per filter



Convolution filters

0	1	-1
1	0	1
1	2	0
1	2	3
4	5	6
7	8	9

1*2 + -1*3 +
1*4 + 1*6 + 1*7
+ 2*8 = 32

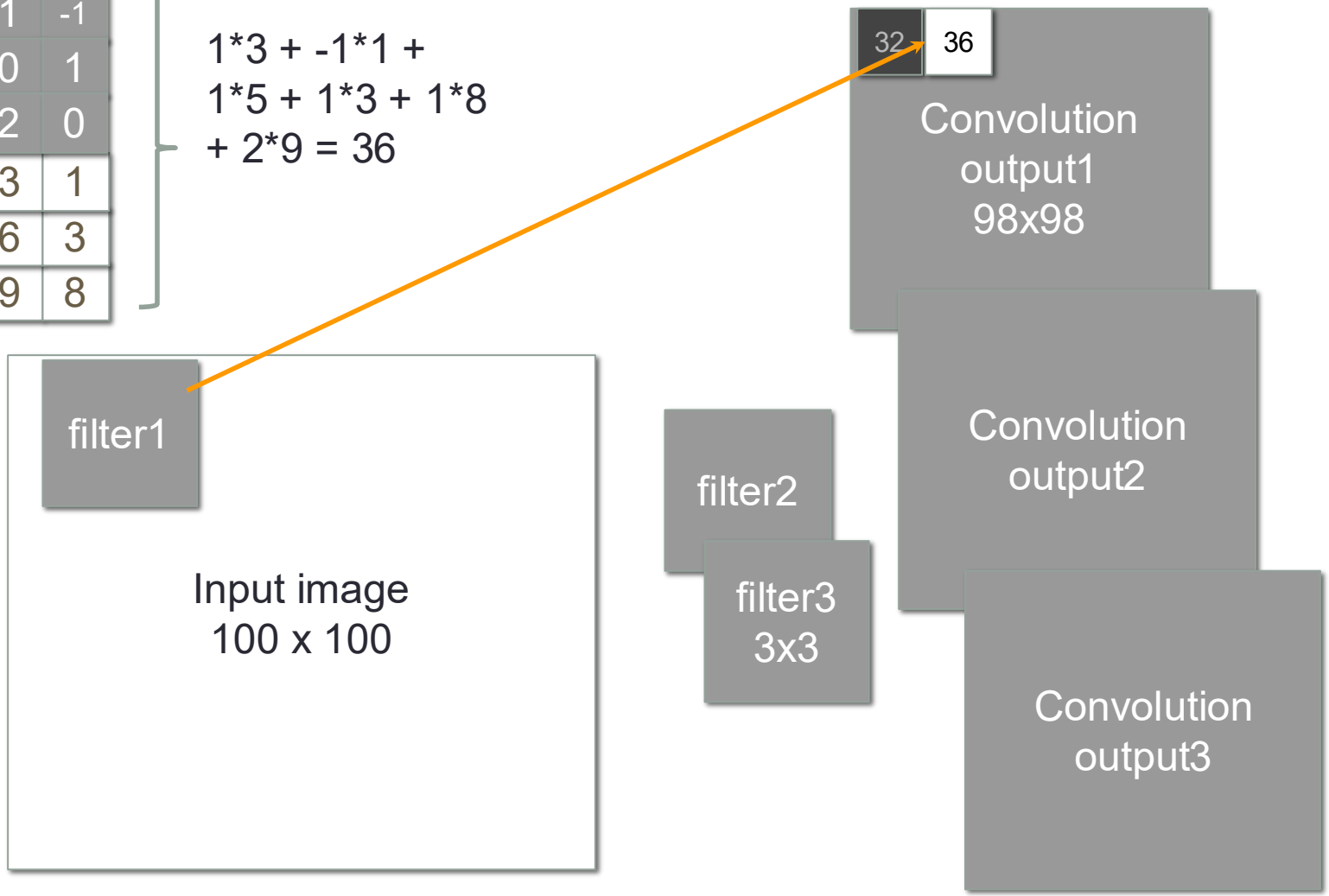


Convolution filters

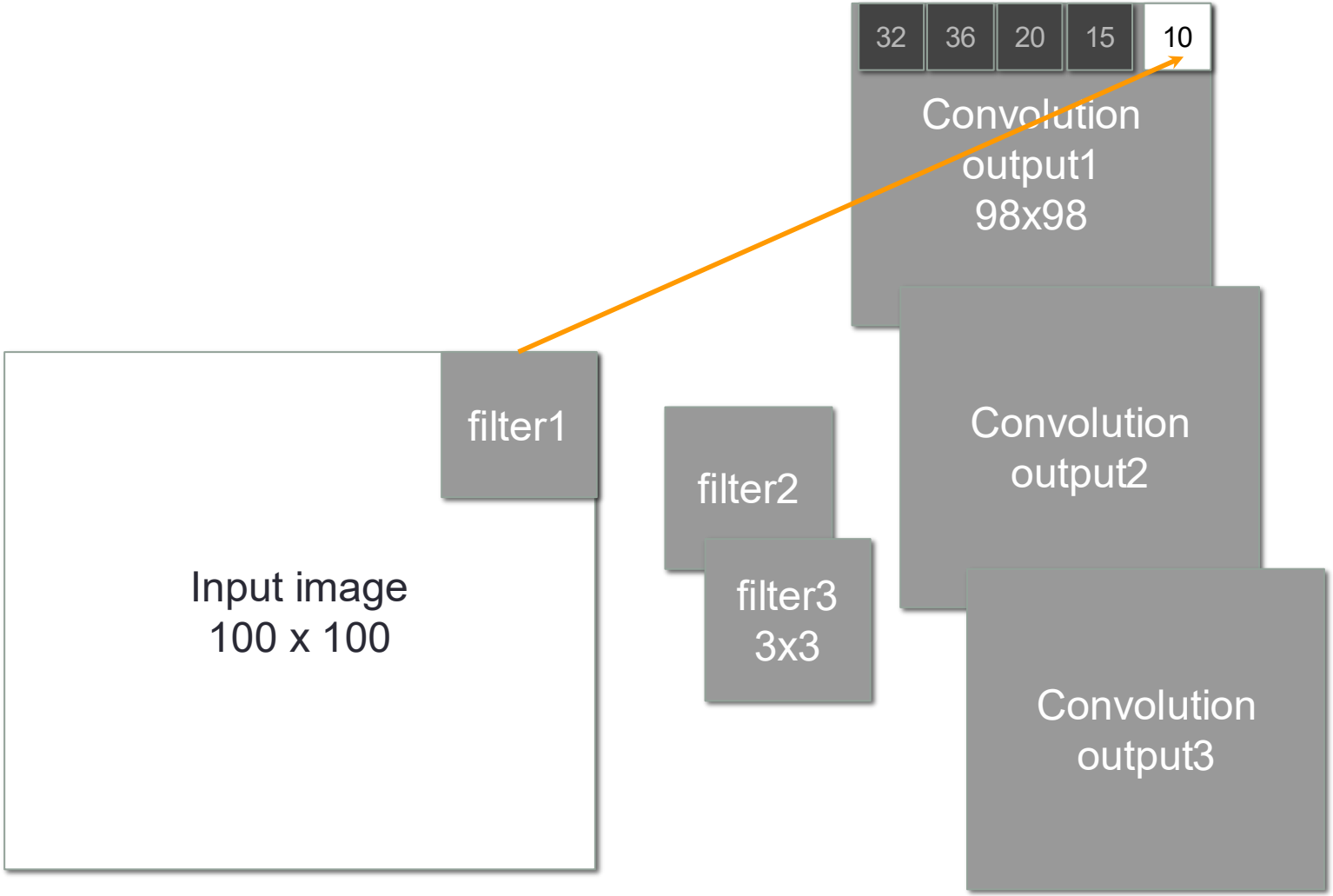
Stride of 1

0	1	-1
1	0	1
1	2	0
2	3	1
5	6	3
8	9	8

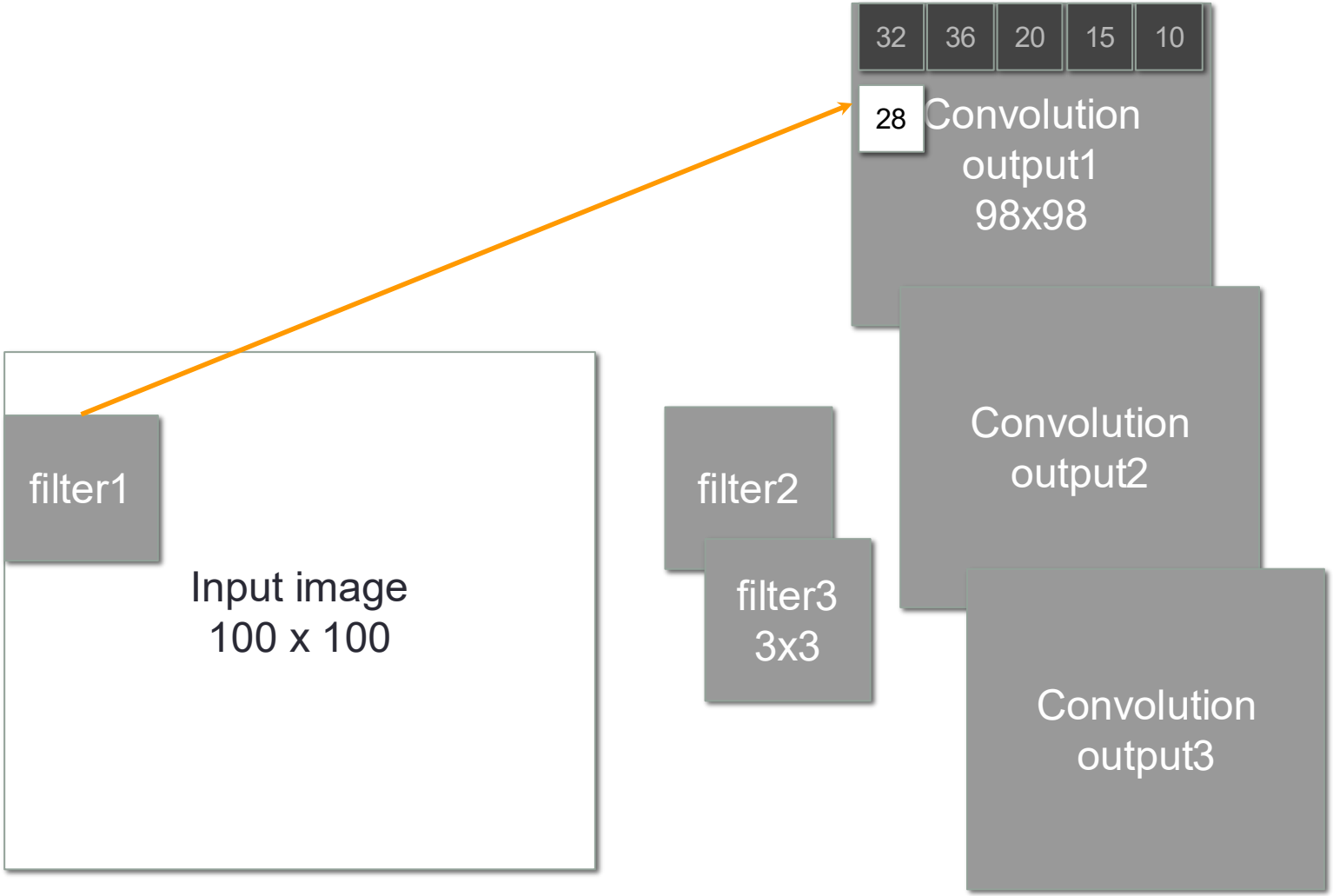
1*3 + -1*1 +
1*5 + 1*3 + 1*8
+ 2*9 = 36



Convolution filters

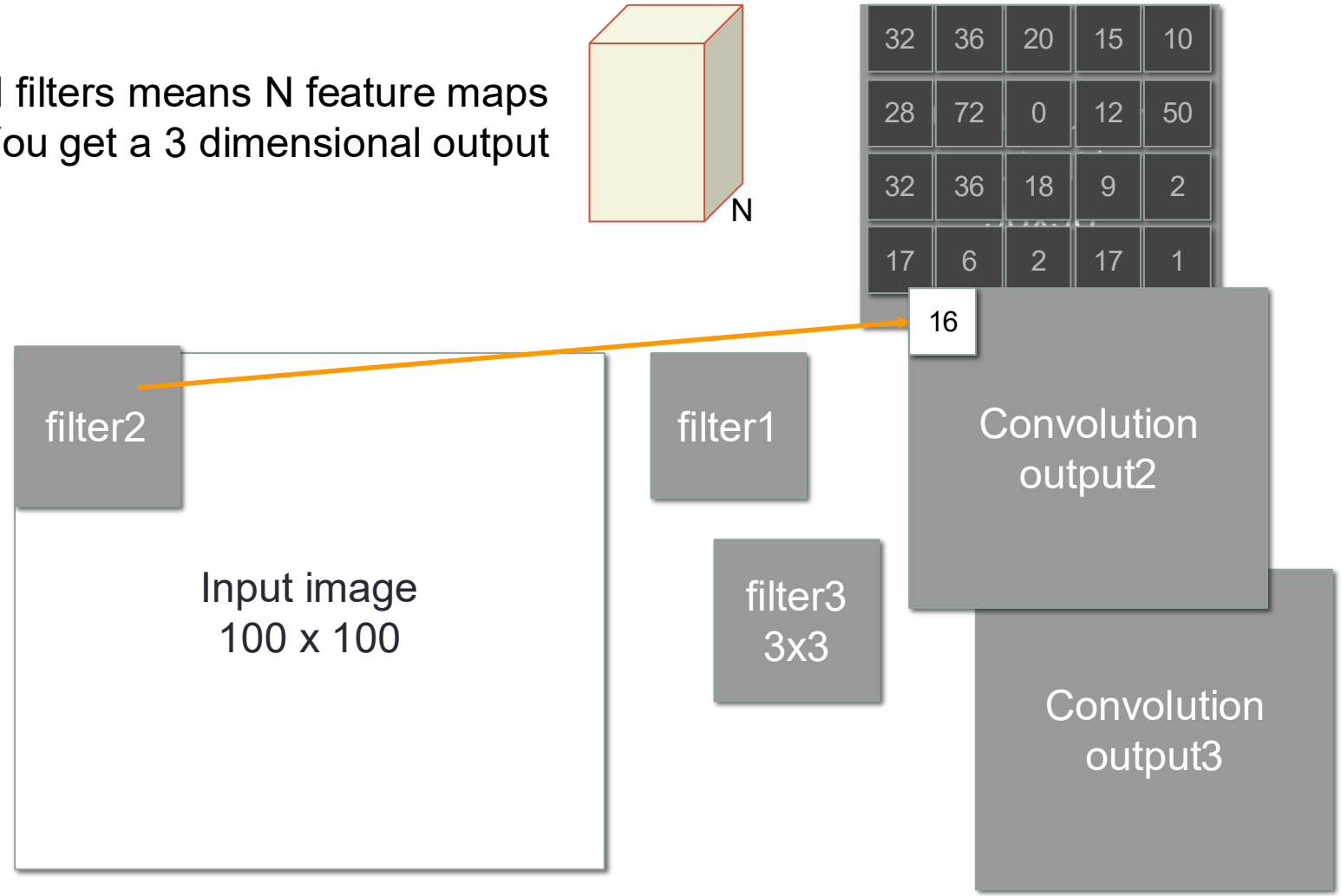


Convolution filters



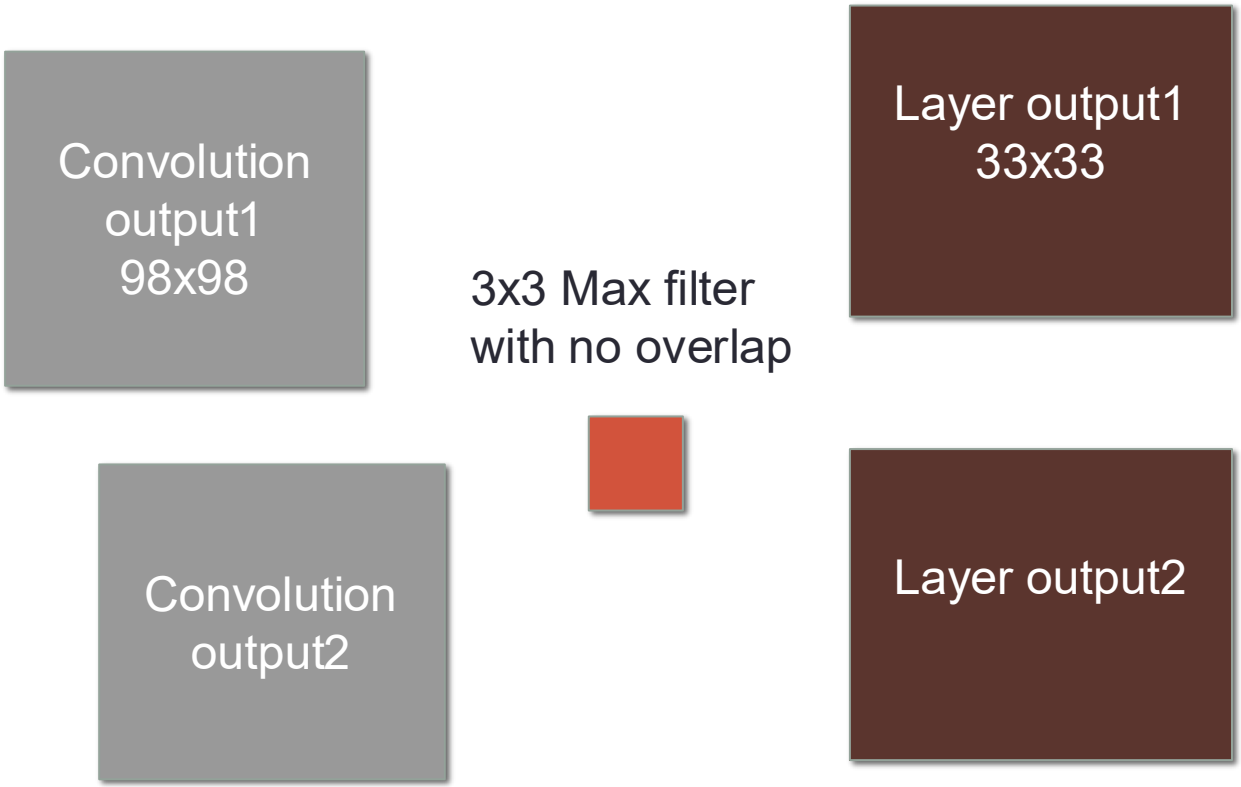
Convolution filters

N filters means N feature maps
You get a 3 dimensional output



Pooling/subsampling

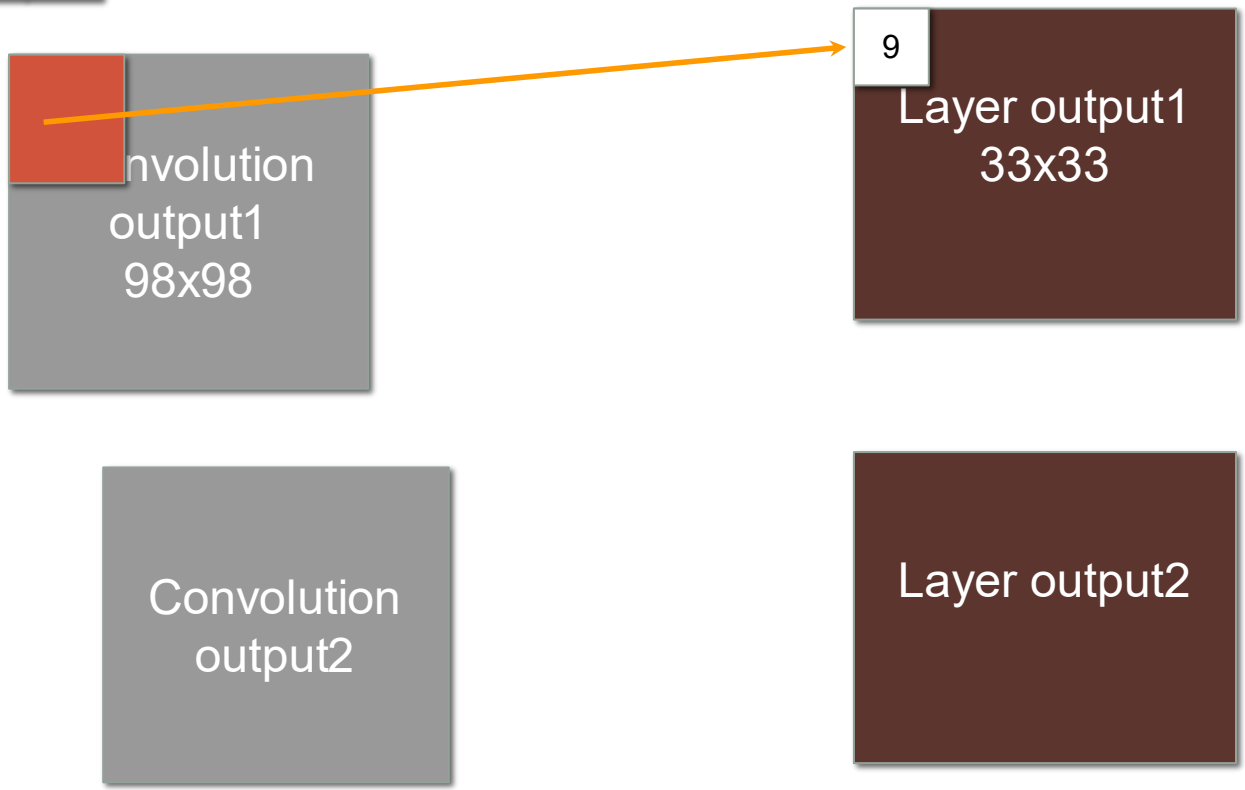
Reduce dimension of the feature maps



Pooling/subsampling

1	2	3
4	5	6
7	8	9

Max = 9

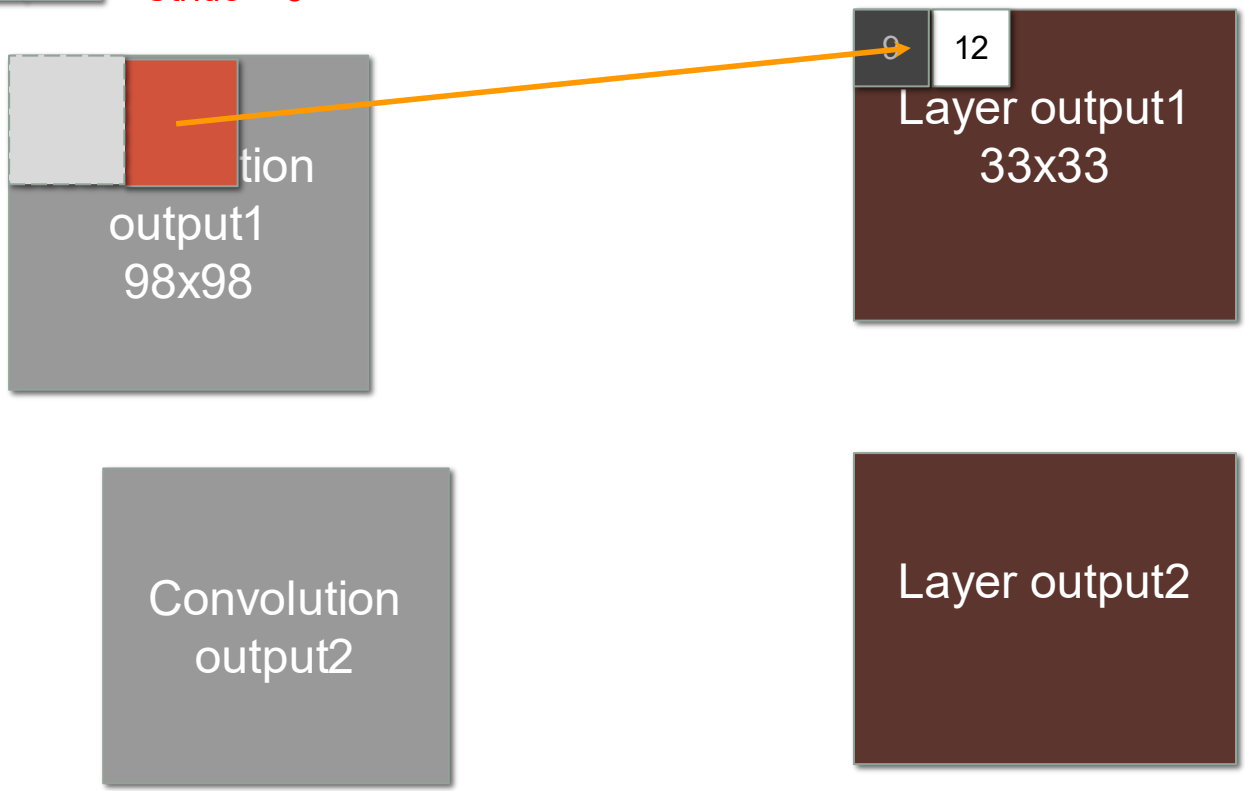


Pooling/subsampling

5	2	1
5	7	1
9	5	12

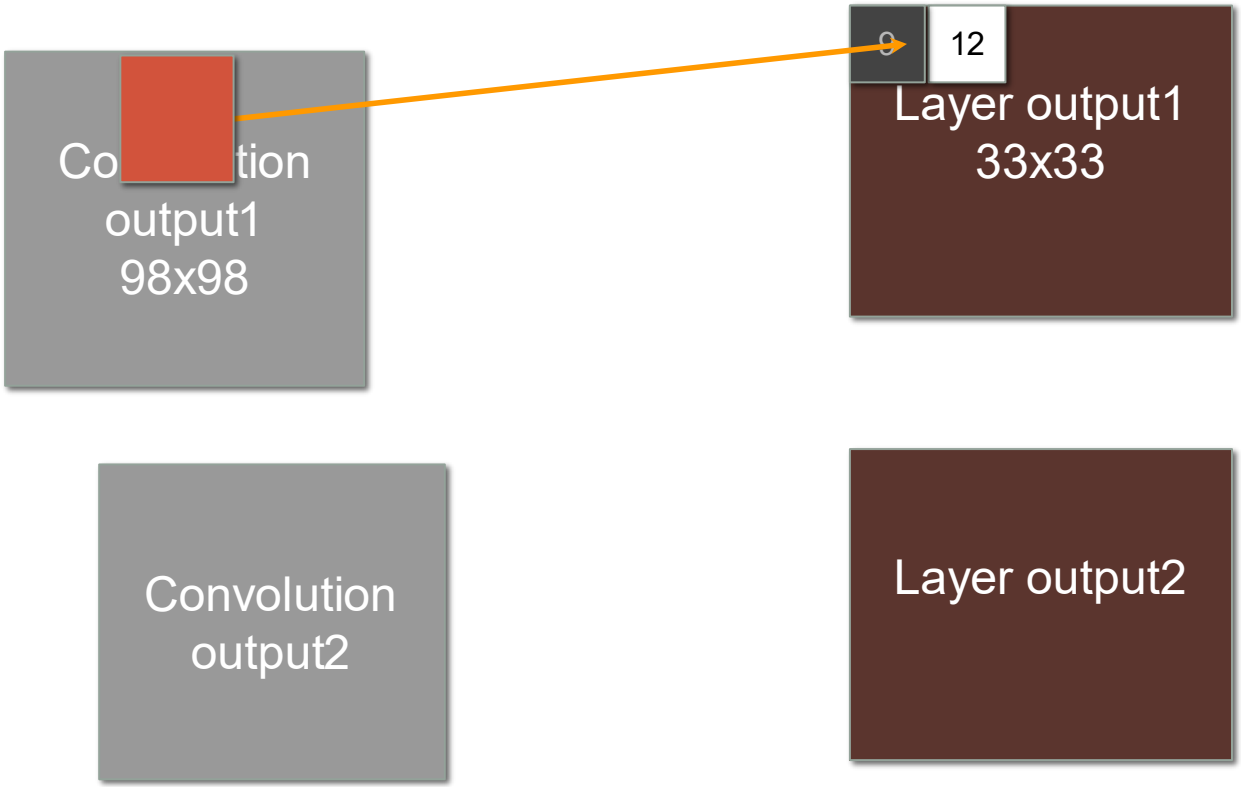
Max = 12

Stride = 3



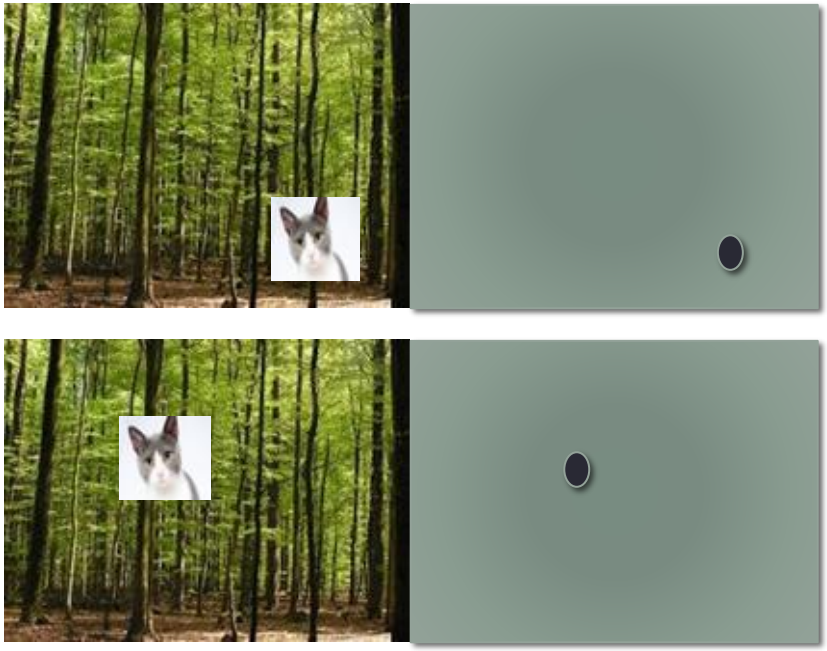
Pooling/subsampling

Can use other functions besides max
Example, average



Notes on convolution

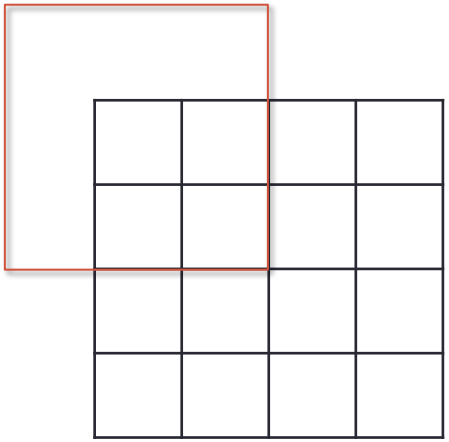
- Convolution layer gives us shift-equivariant (shifting the input, will shift the output equally)
- Using Max pooling (globally) after the convolution layers will give us shift-invariant (shifting the input, will yield the same output)



Convolution puzzle

5 filters 3x3 filter pad, stride 1, pad 1

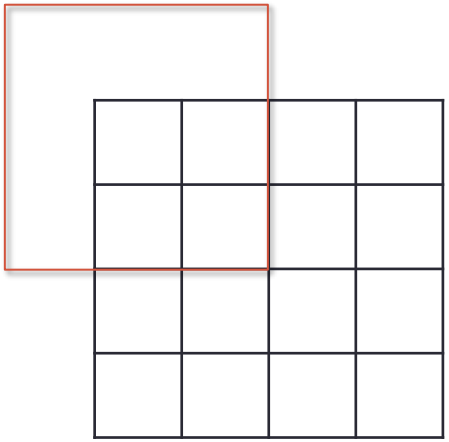
What is the output size?

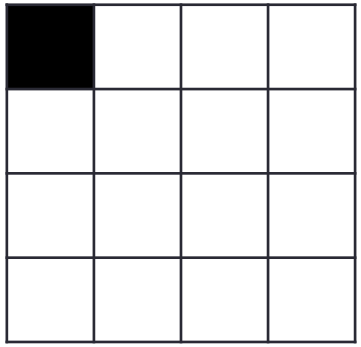


Convolution puzzle

5 filters 3x3 filter pad, stride 1, pad 1

What is the output size?

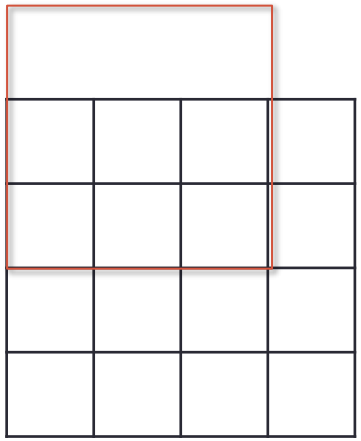


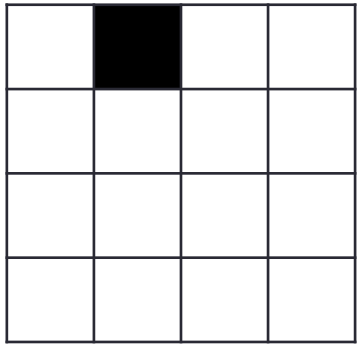


Convolution puzzle

5 filters 3x3 filter pad, stride 1, pad 1

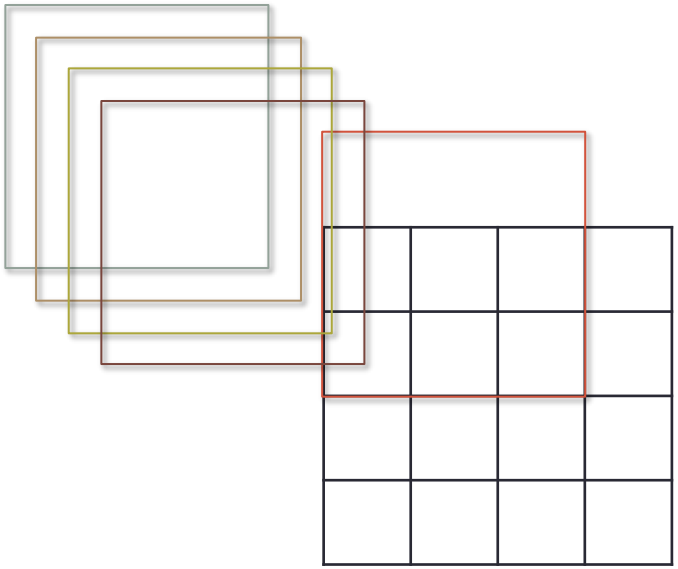
What is the output size?

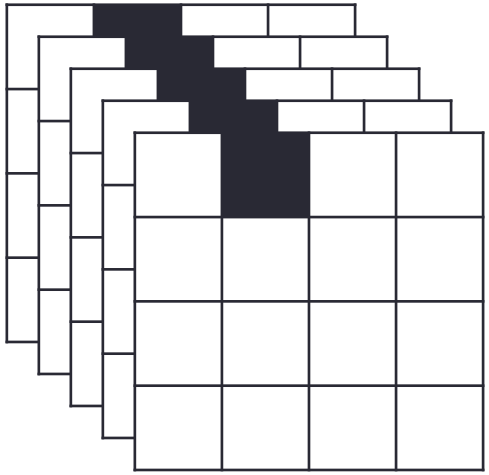




Convolution puzzle

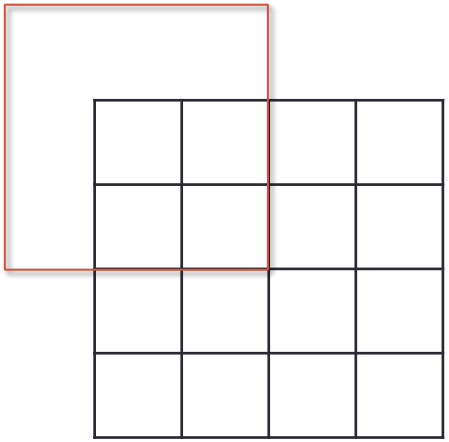
5 filters 3x3 filter pad, stride 1, pad 1





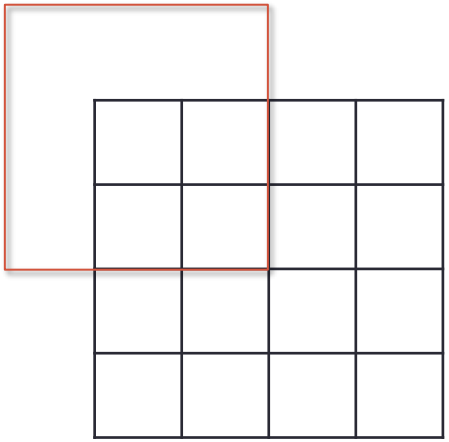
Convolution puzzle

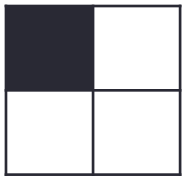
3x3 filter pad, stride 2, pad 1



Convolution puzzle

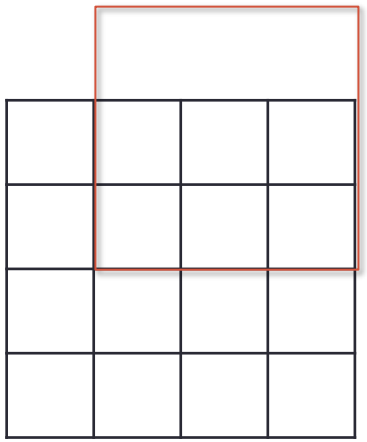
3x3 filter pad, stride 2, pad 1

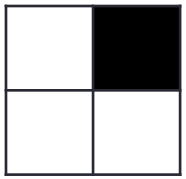




Convolution puzzle

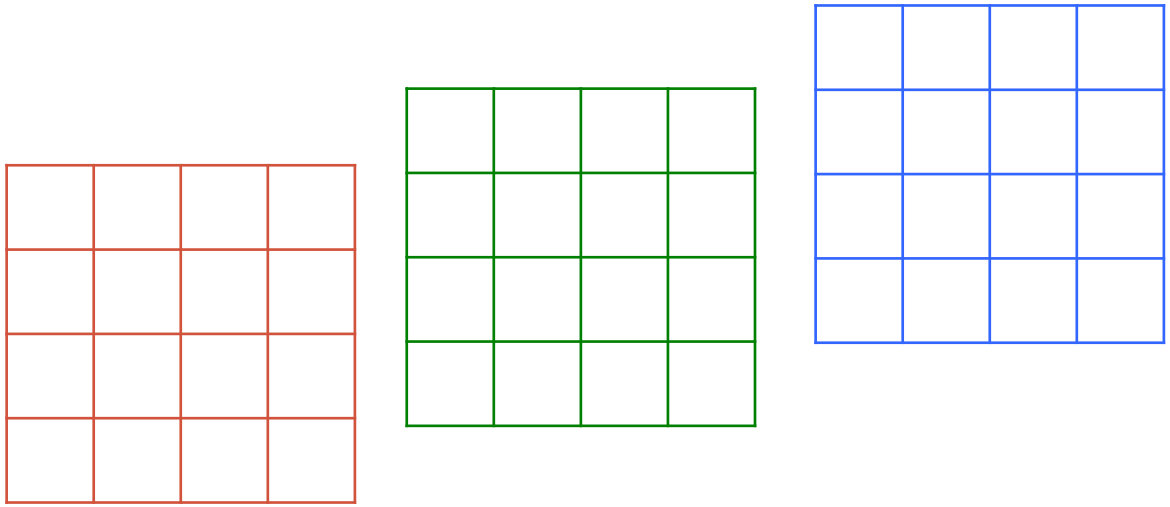
3x3 filter pad, stride 2, pad 1





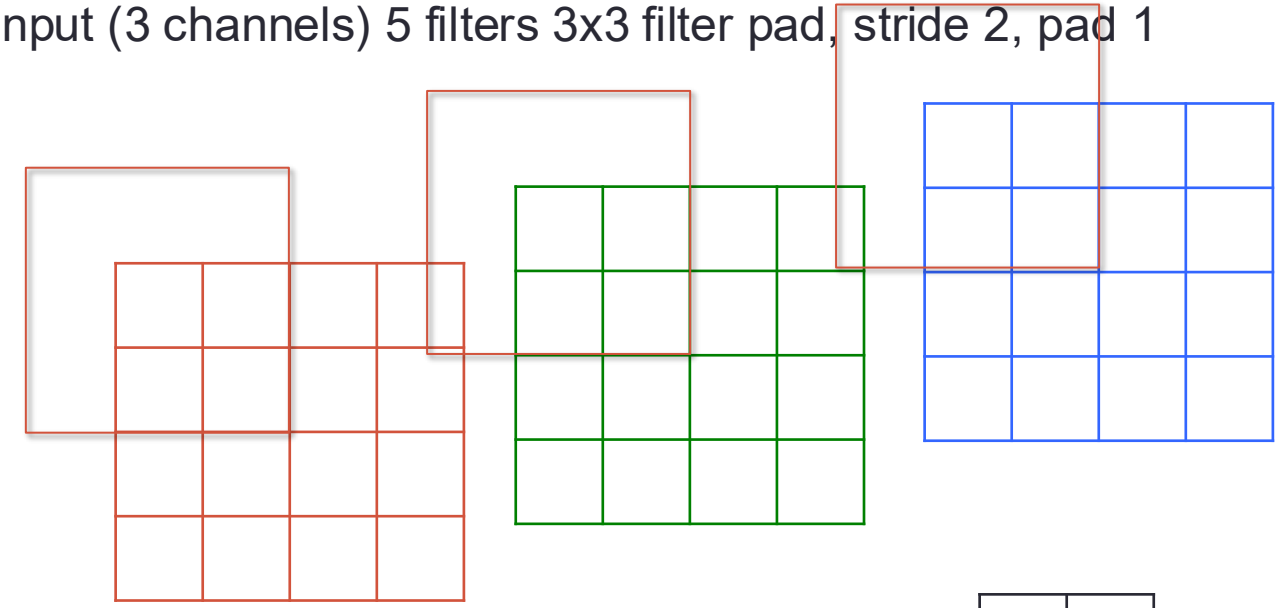
Convolution puzzle

RGB input (3 channels) 5 filters 3x3 filter pad, stride 2, pad 1

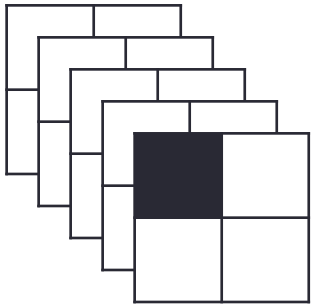


Convolution puzzle

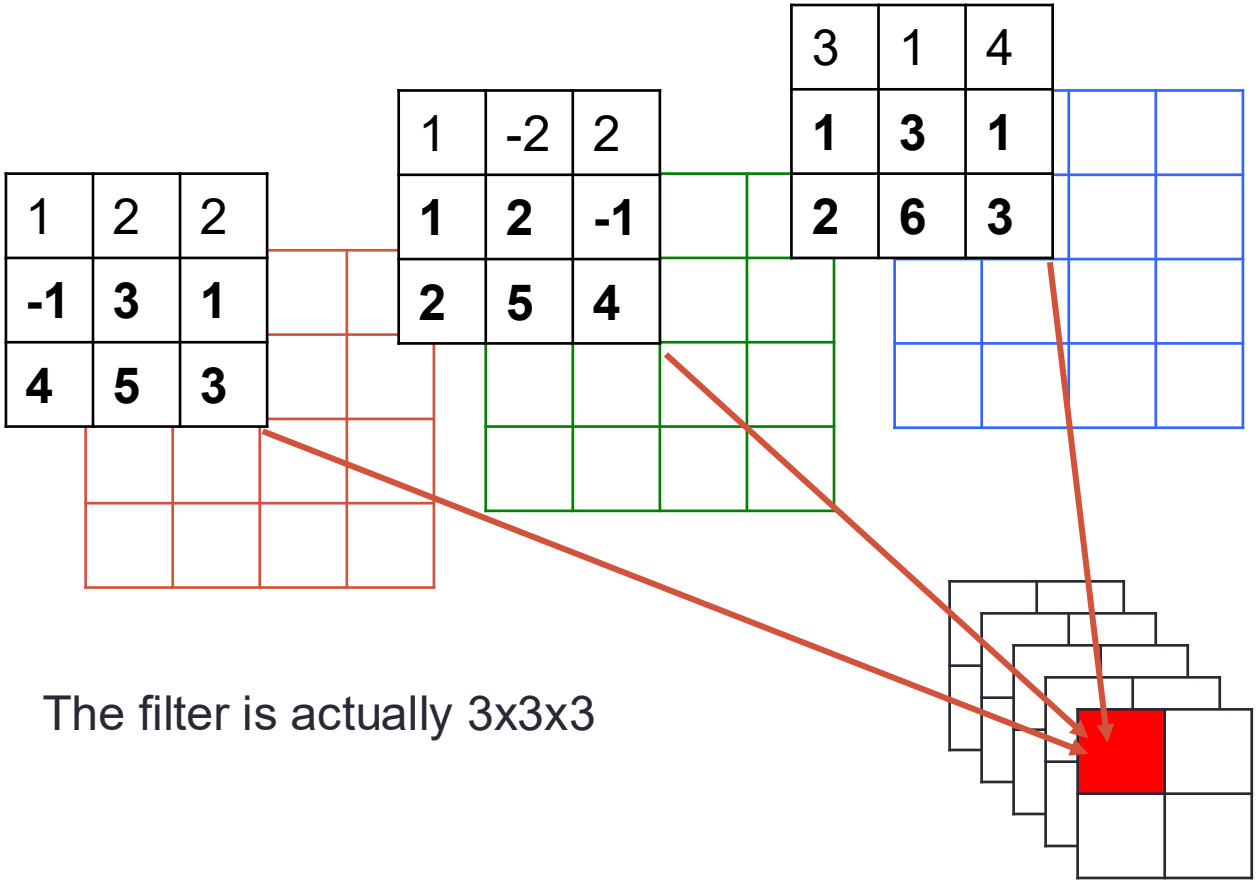
RGB input (3 channels) 5 filters 3x3 filter pad, stride 2, pad 1



The filter is actually 3x3x3



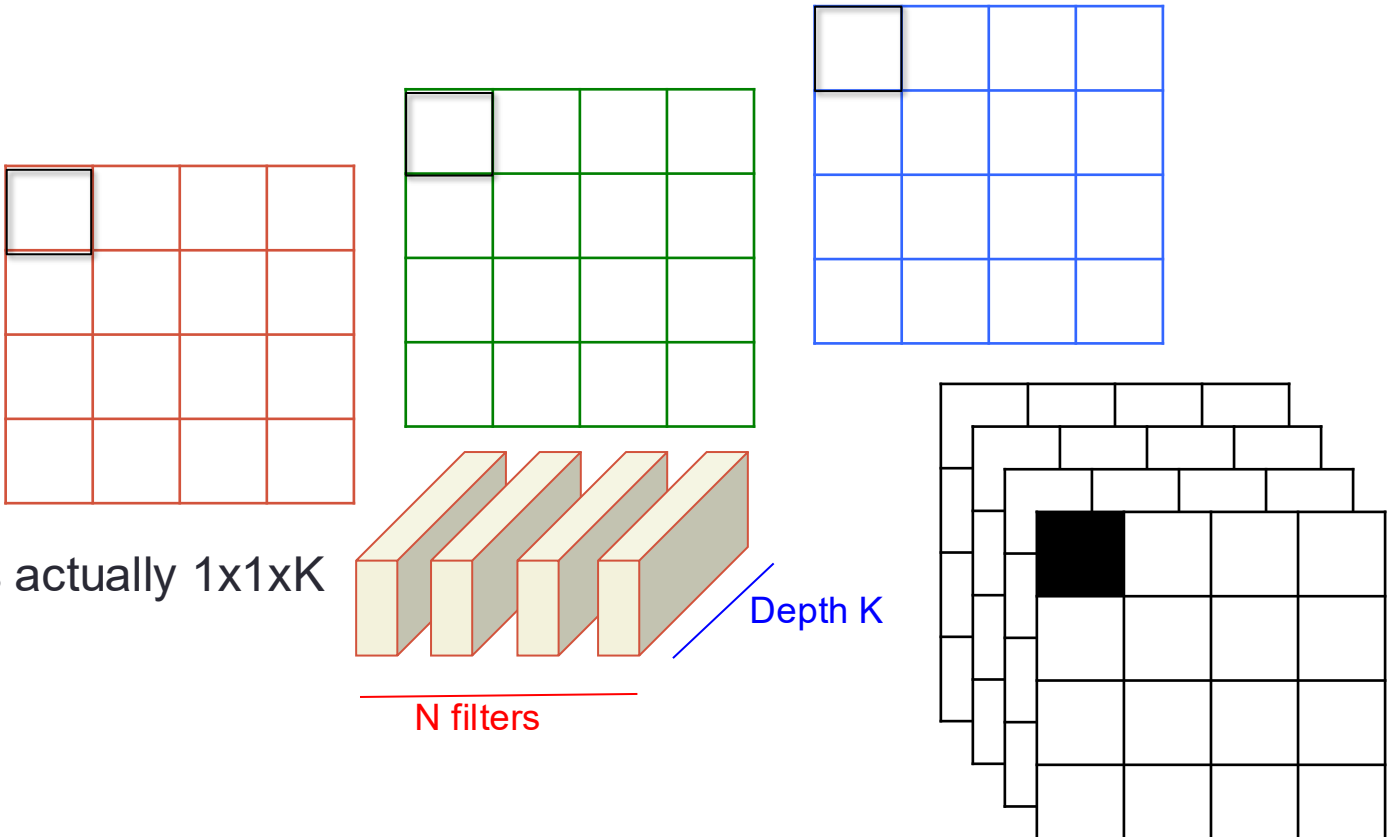
Convolution puzzle



The filter is actually 3x3x3

1x1 convolution

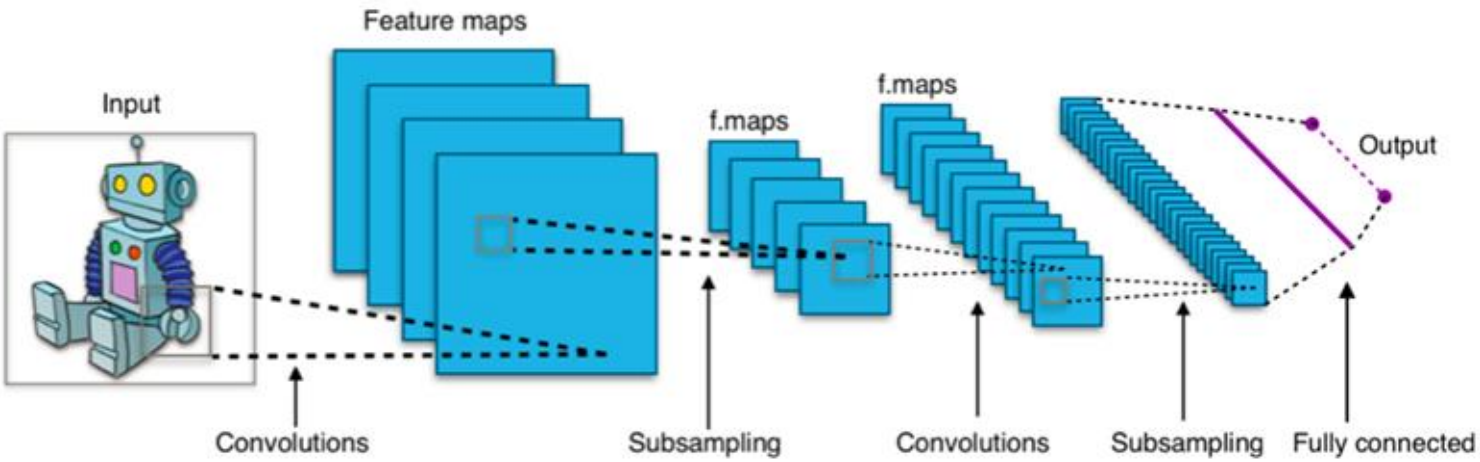
Reduces the dimension of feature maps



The filter is actually 1x1xK

CNN overview

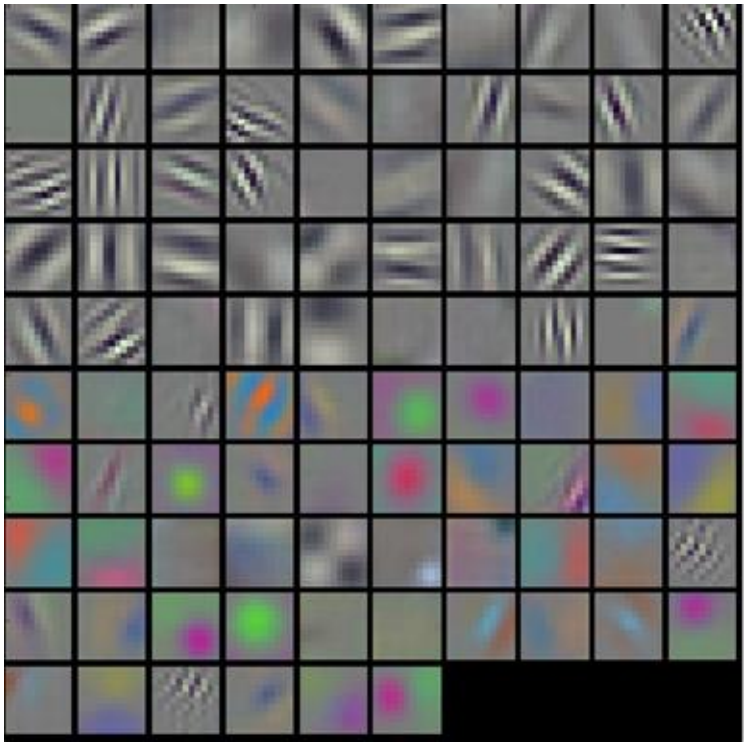
- Filter size, number of filters, filter shifts, and pooling rate are all parameters
- Usually followed by a fully connected network at the end
  - CNN is good at learning low level features
  - DNN combines the features into high level features and classify



https://en.wikipedia.org/wiki/Convolutional_neural_network#/media/File:Typical_cnn.png

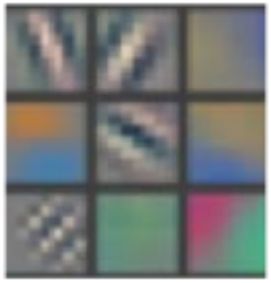
Visualizing convolutional layers

- Just like PCA, we can visualize the weights of a transform
- “Matched filters”

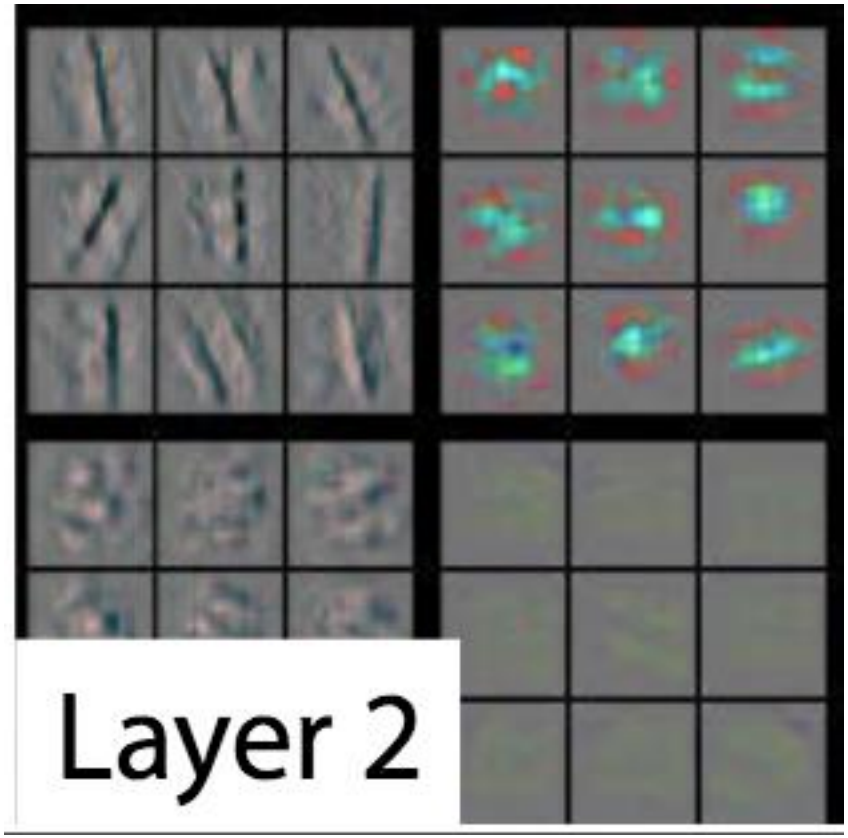


http://cs231n.github.io/understanding-cnn/

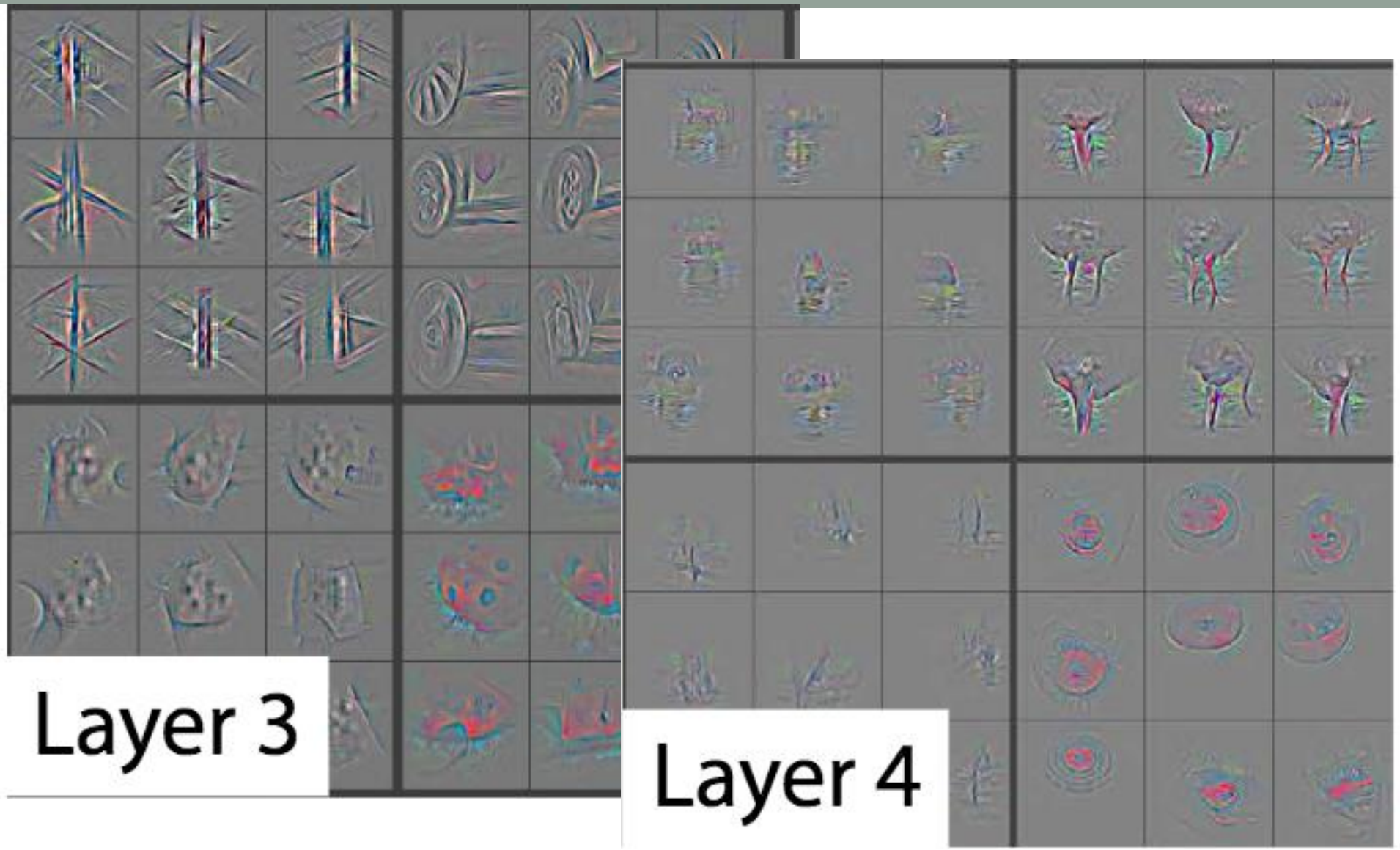
Higher layer captures higher-level concepts



Layer 1

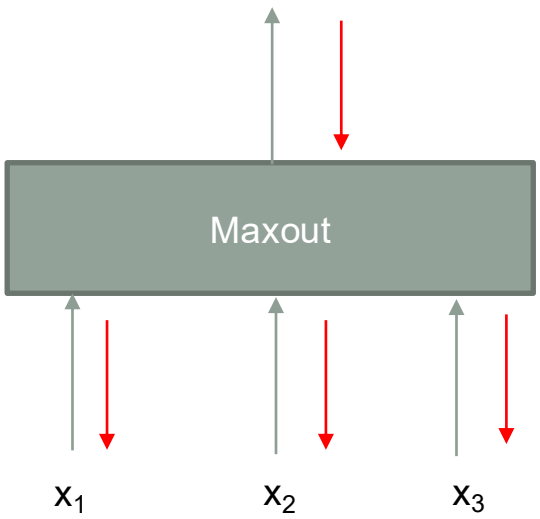


https://arxiv.org/pdf/1311.2901.pdf


Layer 3
Layer 4

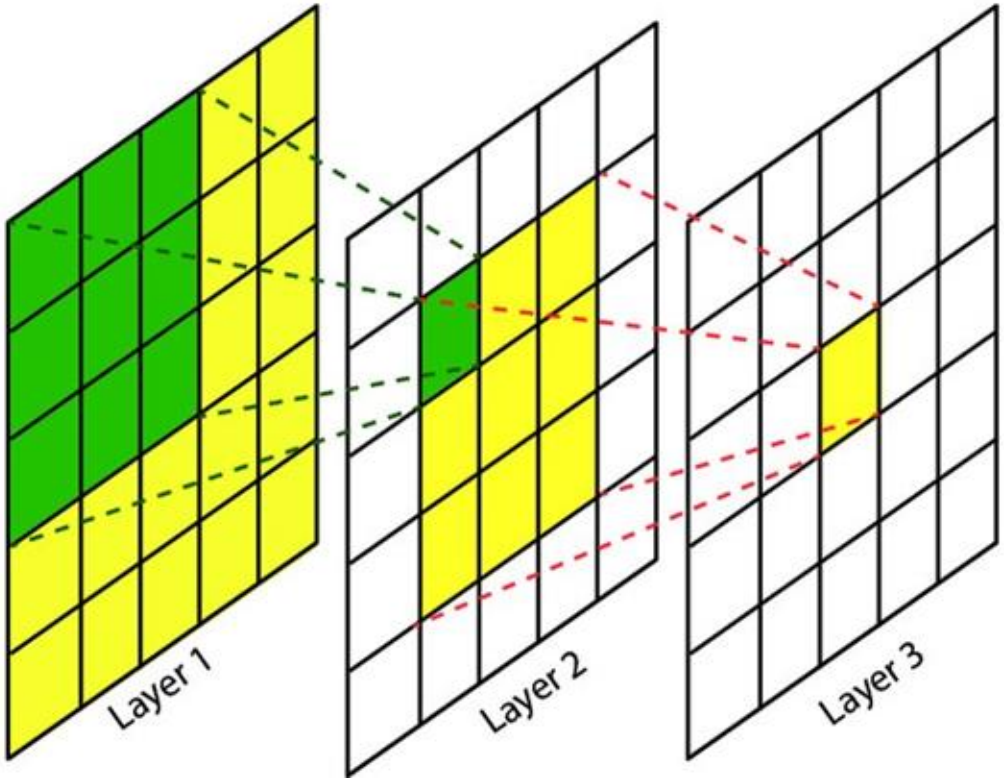
Pooling/subsampling

- Max filter -> Maxout
  - Backward pass?



Receptive field

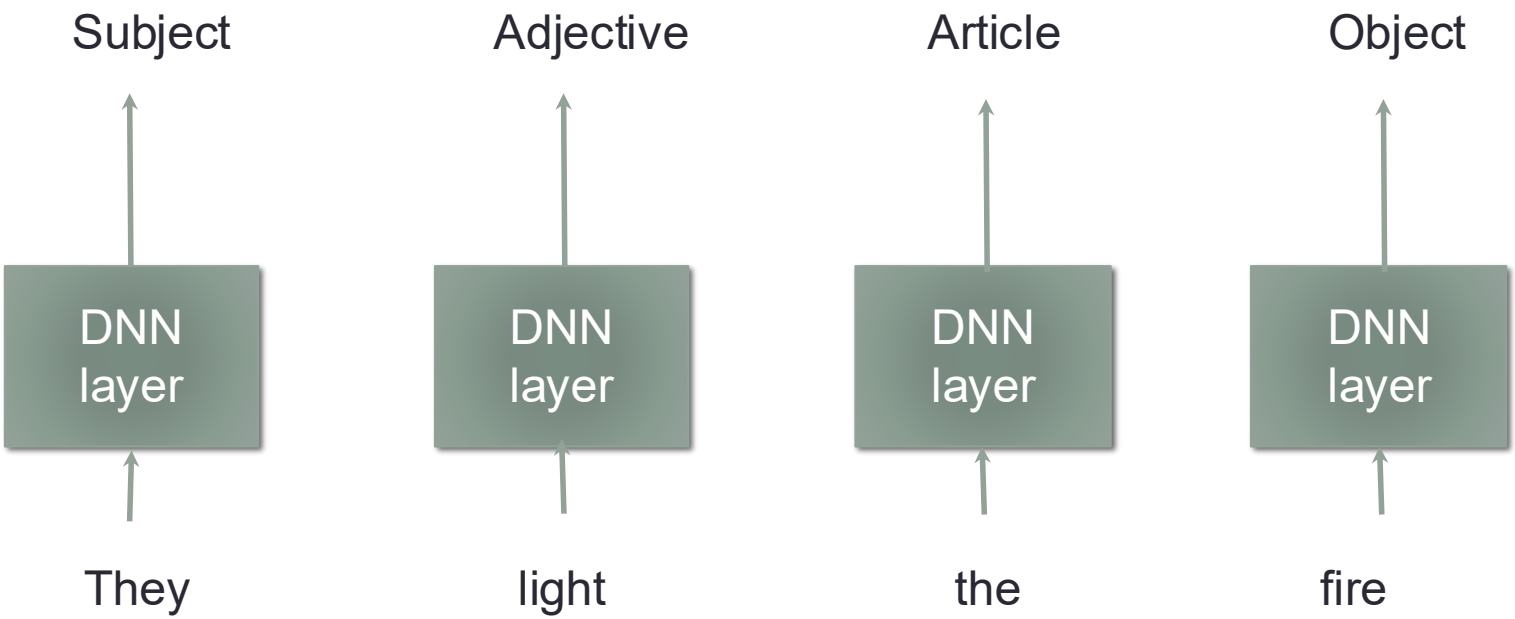
You might want to consider about how large is your pattern when designing your network



source image

Recurrent neural network (RNN)

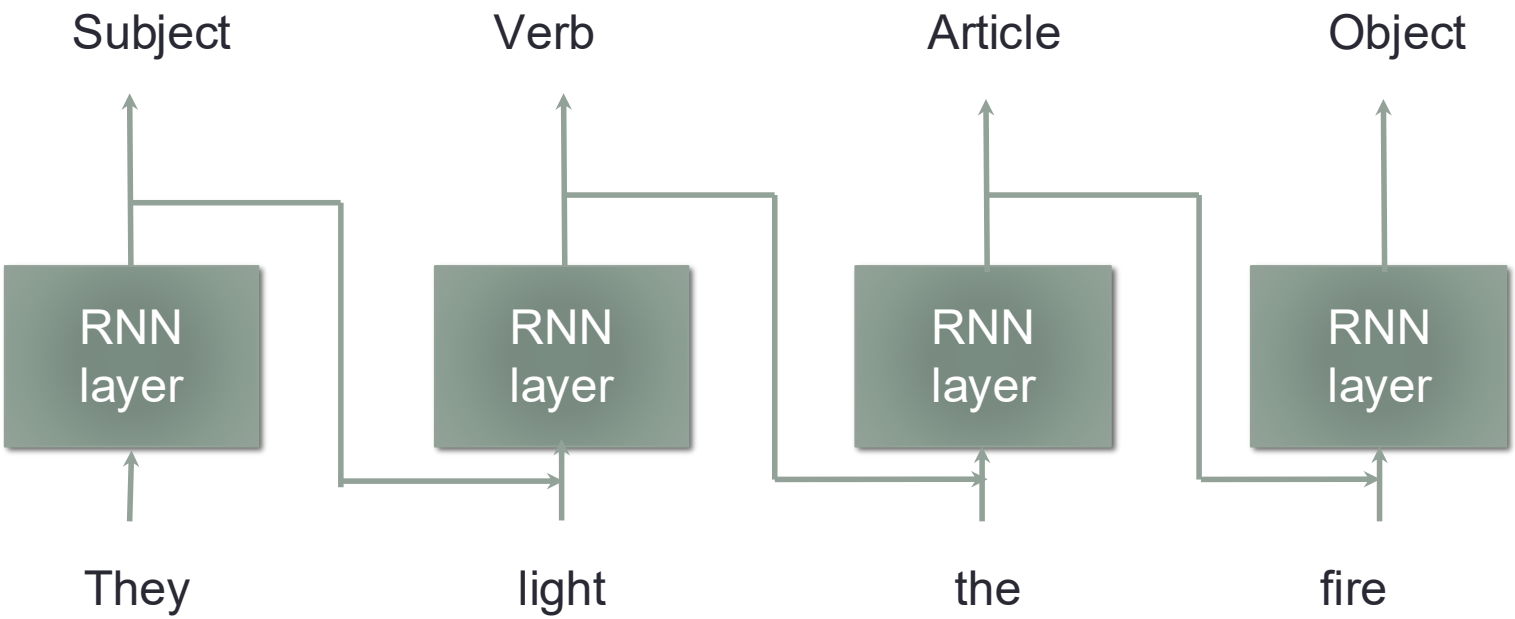
- DNN framework



Problem 1: need a way to remember the past

Recurrent neural network (RNN)

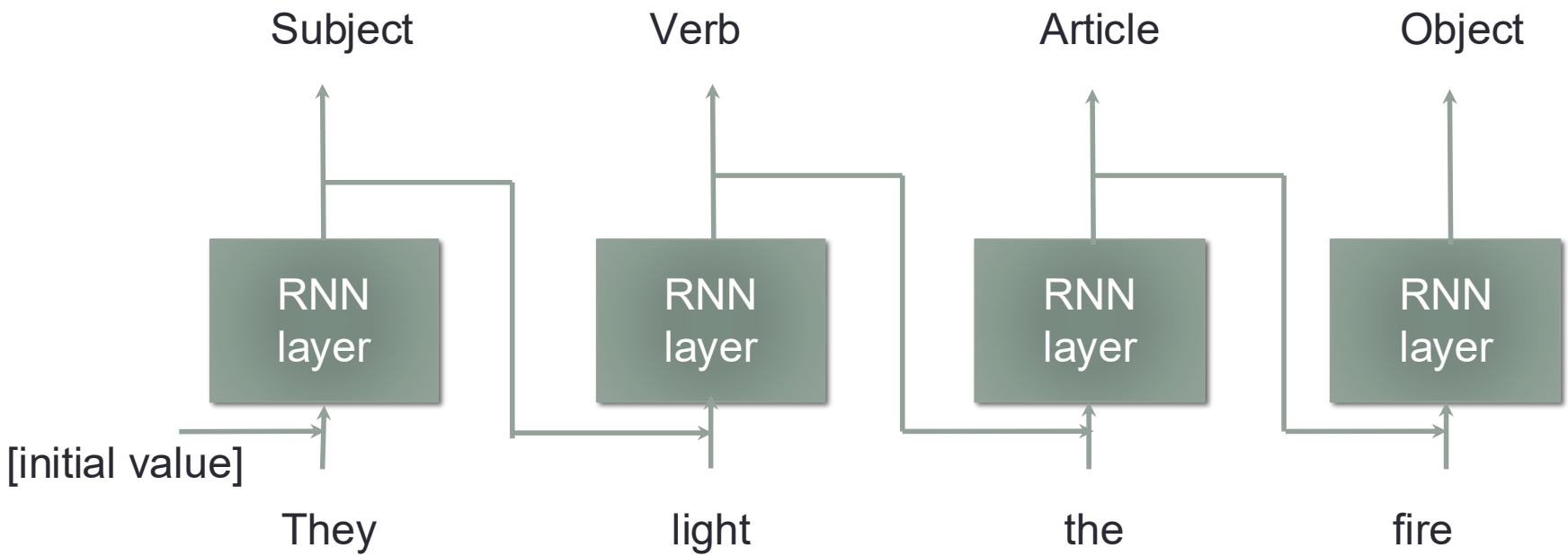
- RNN framework



Output of the layer encodes something meaningful about the past

Recurrent neural network (RNN)

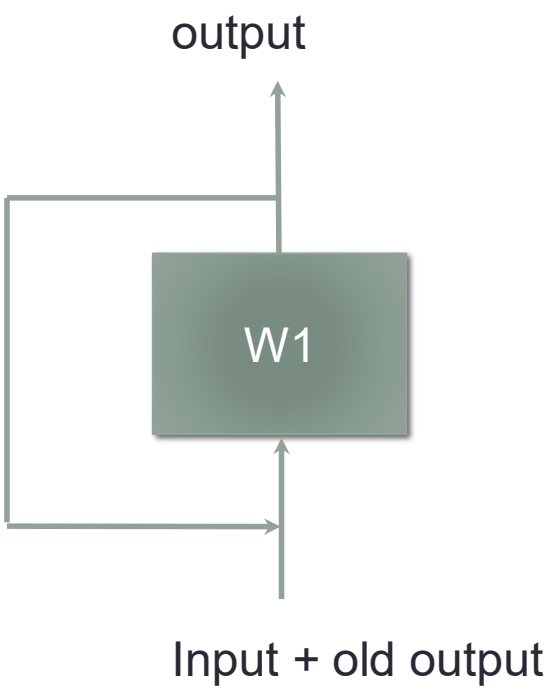
- RNN framework



New input feature = [original input feature, output of the layer at previous time step]

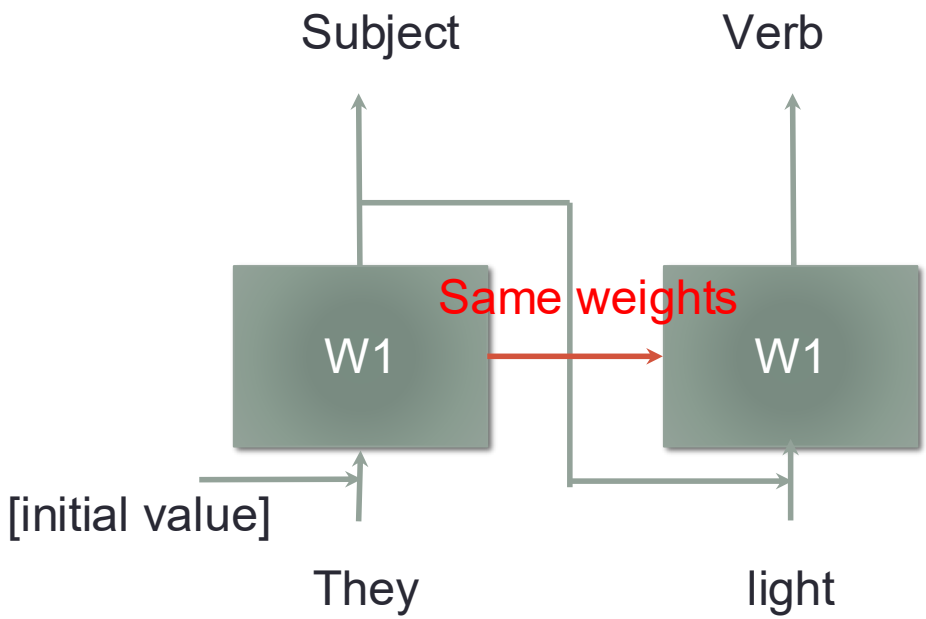
Recurrent neural network (RNN)

- Unrolling of a recurrent layer.



Recurrent neural network (RNN)

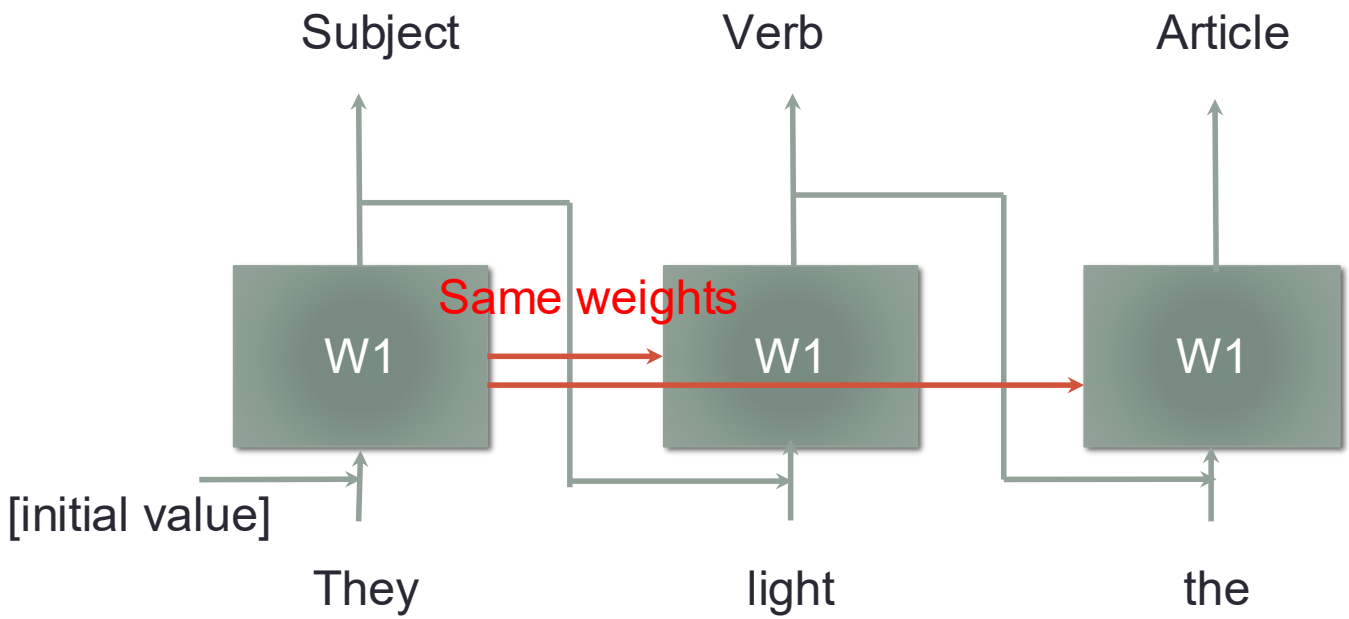
- Unrolling of a recurrent layer.



Parameter sharing across time

Recurrent neural network (RNN)

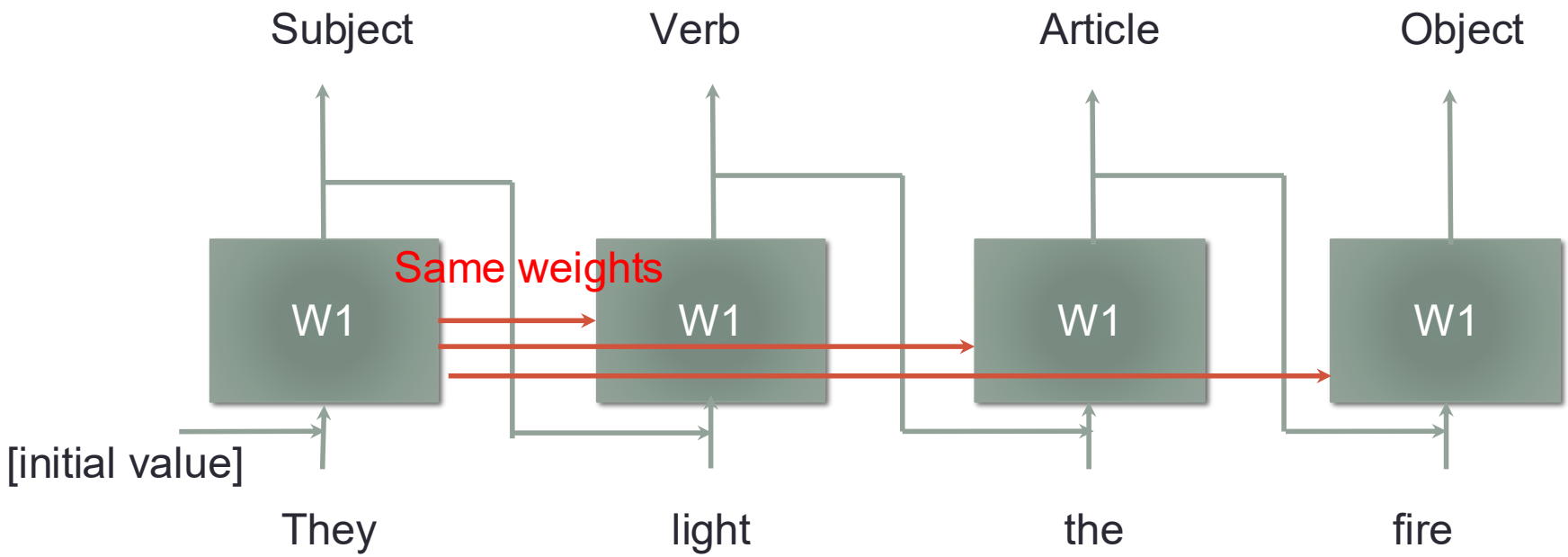
- Unrolling of a recurrent layer.



Parameter sharing across time

Recurrent neural network (RNN)

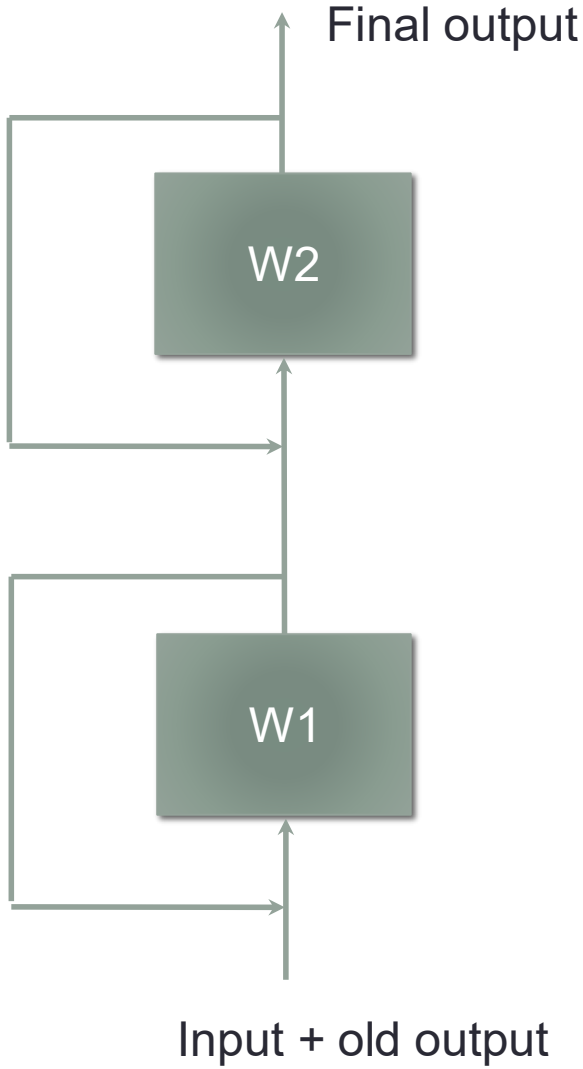
- Unrolling of a recurrent layer.



Parameter sharing across time

Recurrent neural network (RNN)

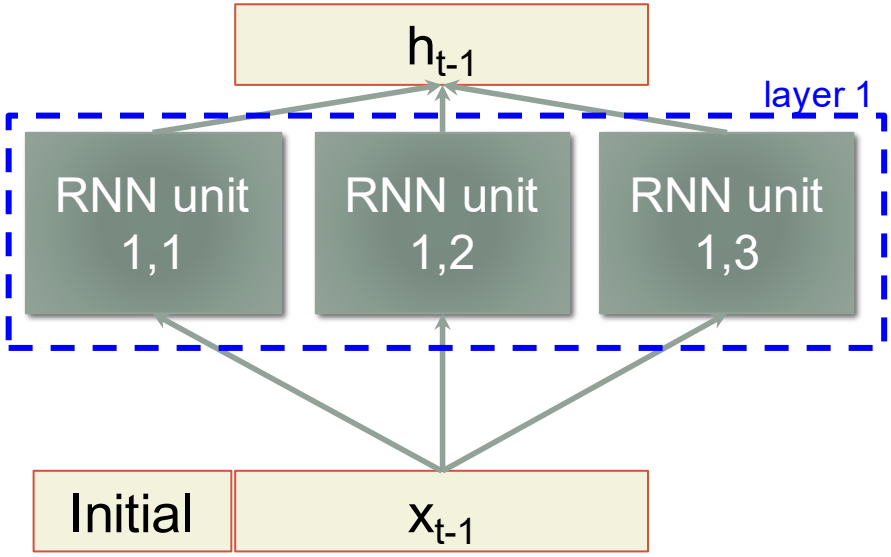
- Stacks of recurrent layer



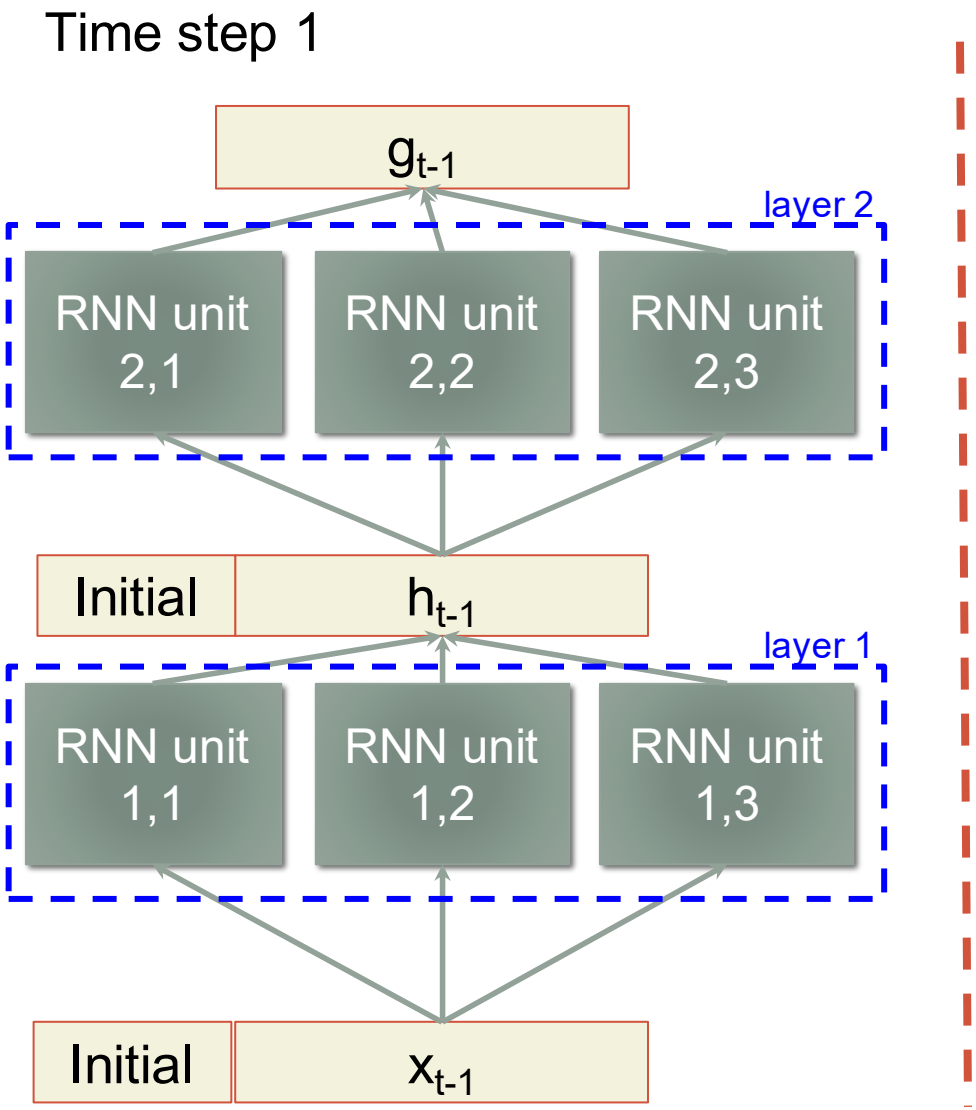
RNN layers (expanded in time)

Time step 1

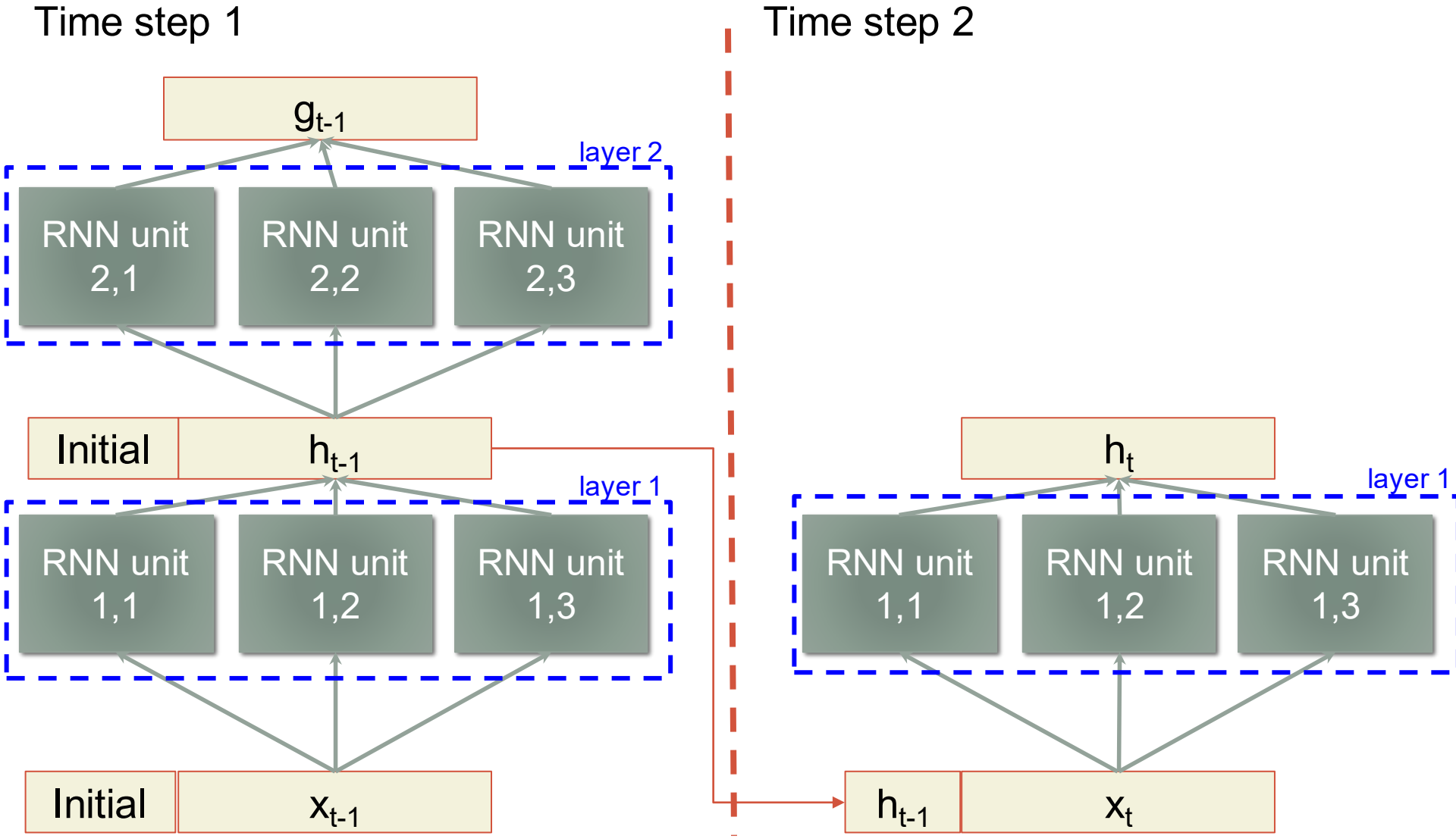
Time step 2



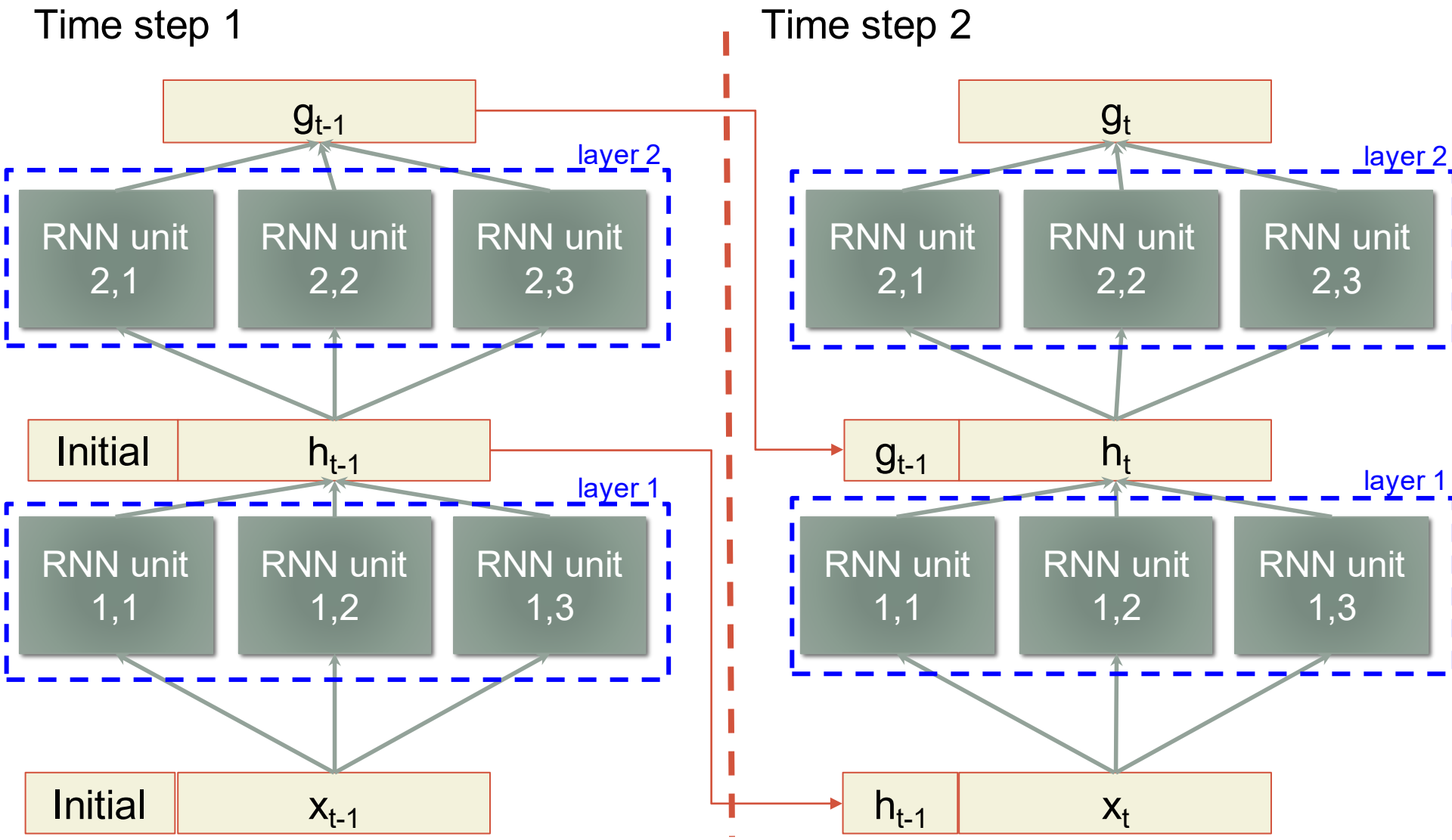
RNN layers (expanded in time)



RNN layers (expanded in time)

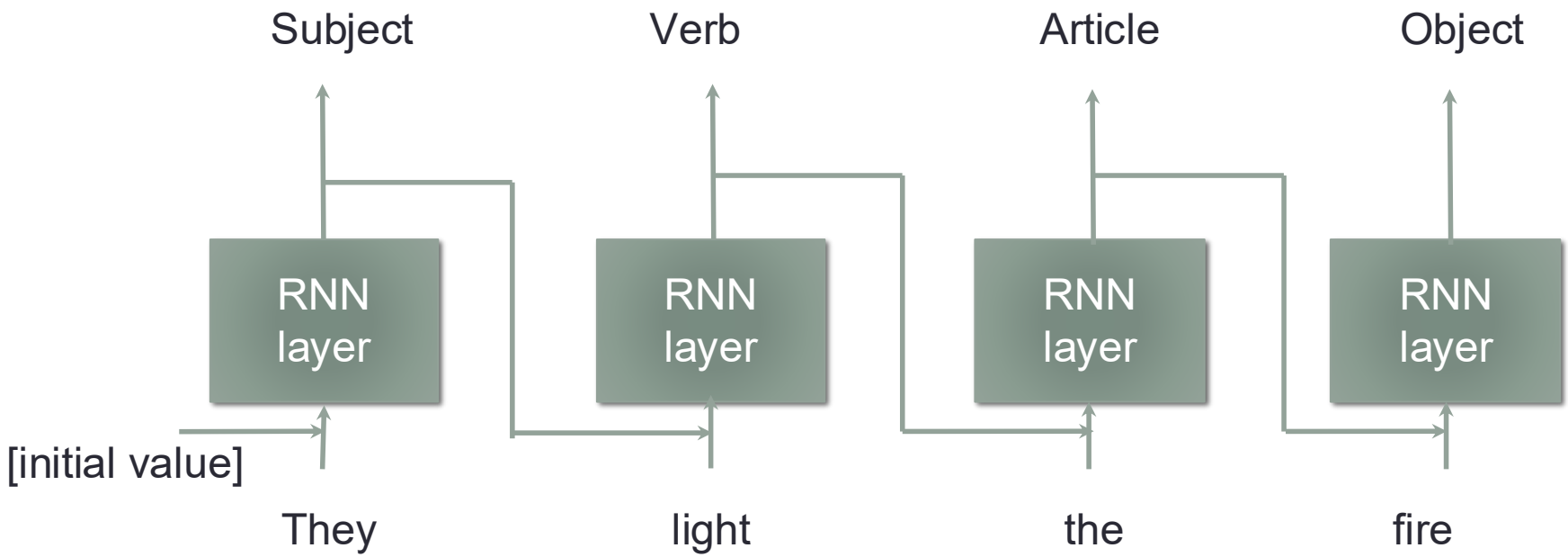


RNN layers (expanded in time)



Training a recurrent neural network

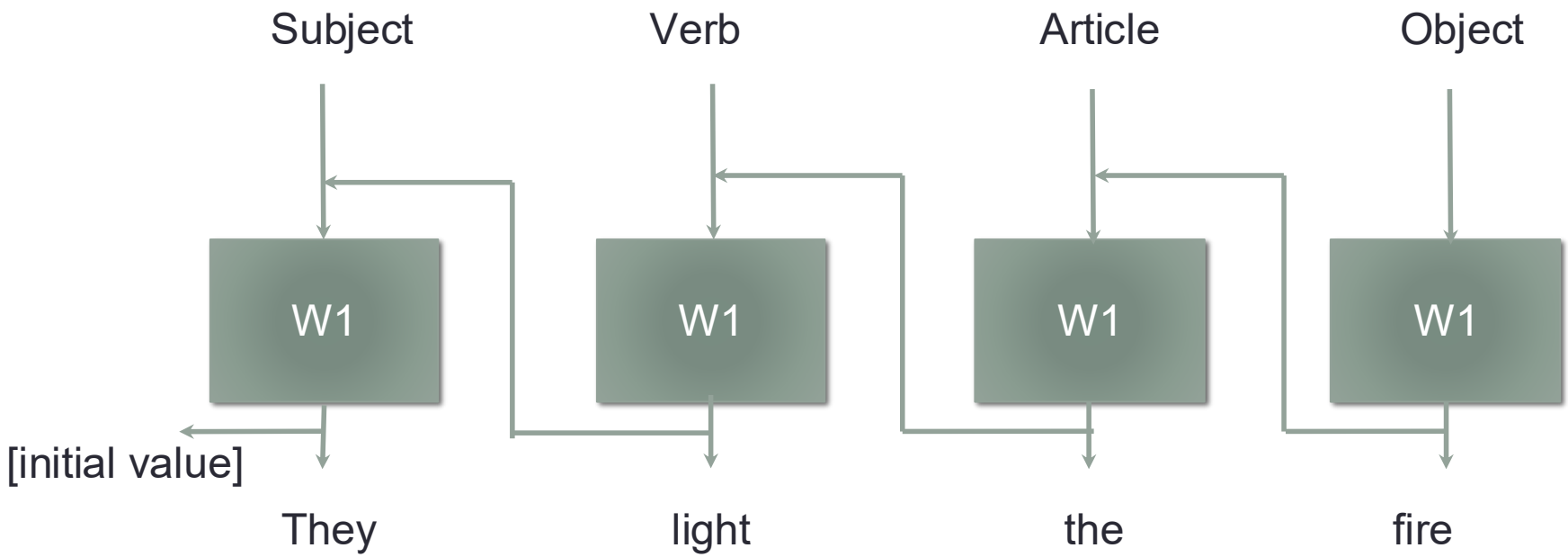
- RNN framework



New input feature = [original input feature, output of the layer at previous time step]

Training a recurrent neural network

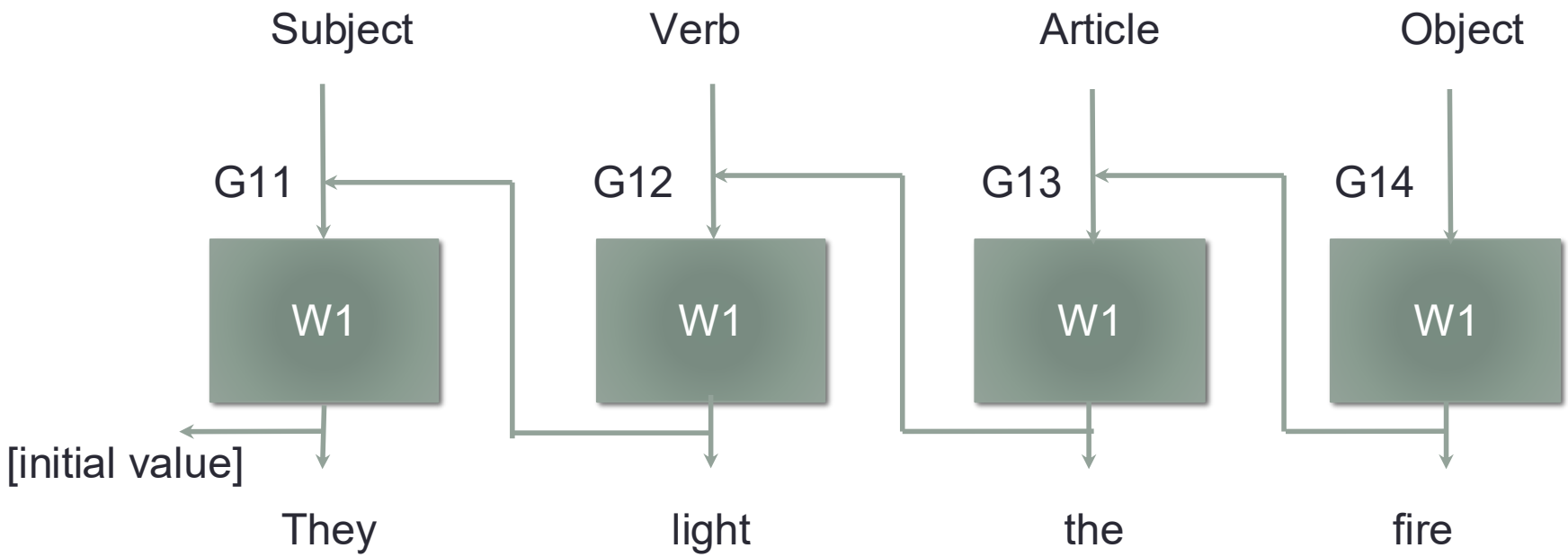
- Backward Computation graph



Backpropagation through time (BPTT)

BPTT

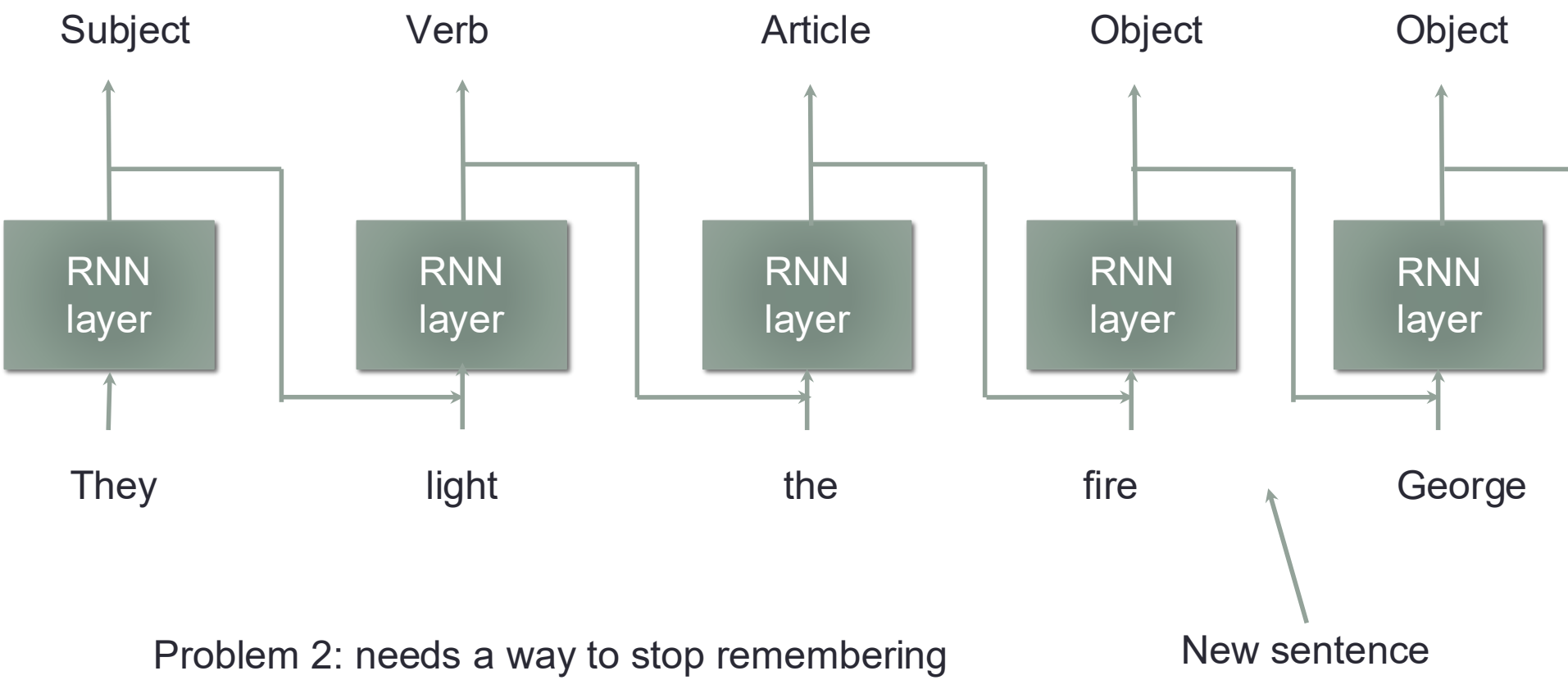
- Backward Computation graph



W1 <- W1 + G11 + G12 + G13 + G14

Problem 1: cannot deal with infinitely long recurrent
Gradient explosion, vanishing gradient

Recurrent neural network (RNN)

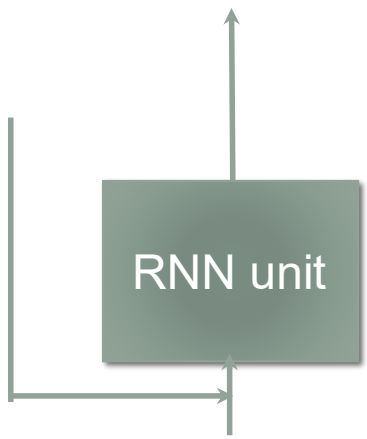


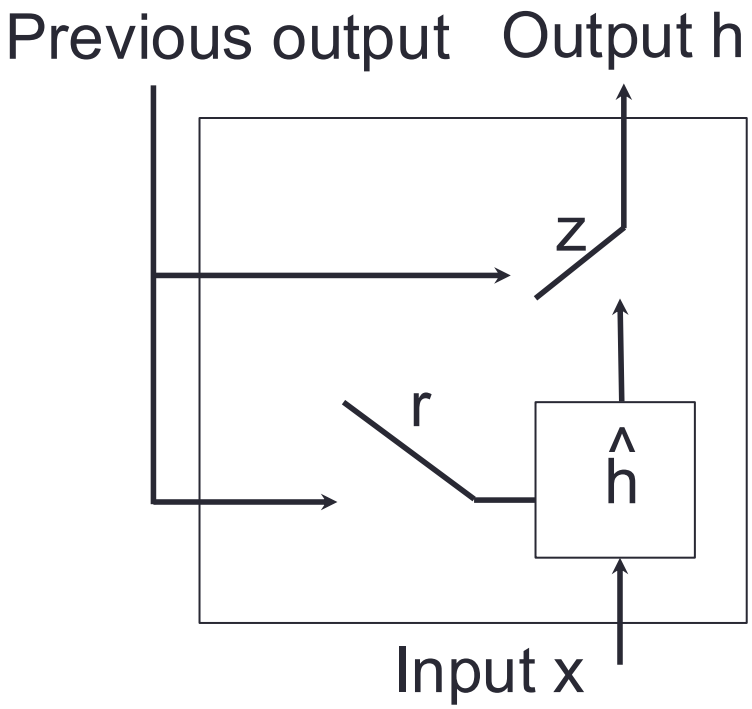
Problem 2: needs a way to stop remembering

Can the network learn when to start and stop remembering things?

Gated Recurrent Unit (GRU)

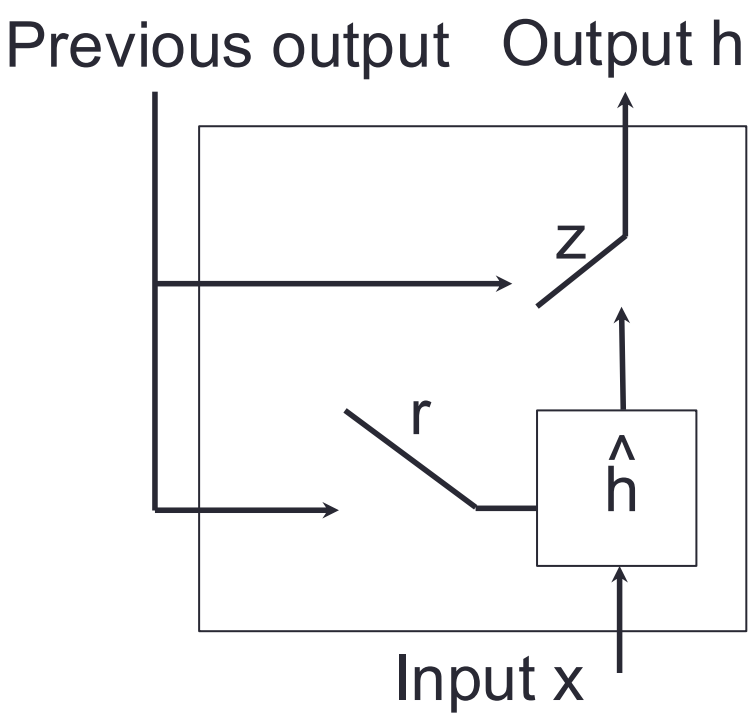
- Forms a Gated Recurrent Neural Networks (GRNN)
- Add gates that can choose to reset (r) or update (z)

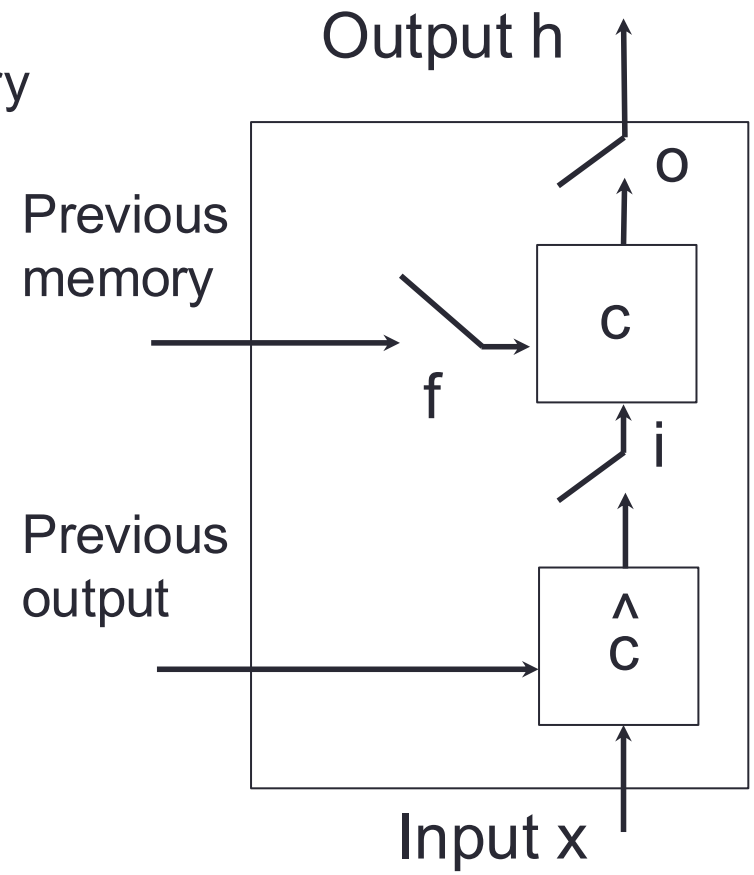




Long Short-Term Memory (LSTM)

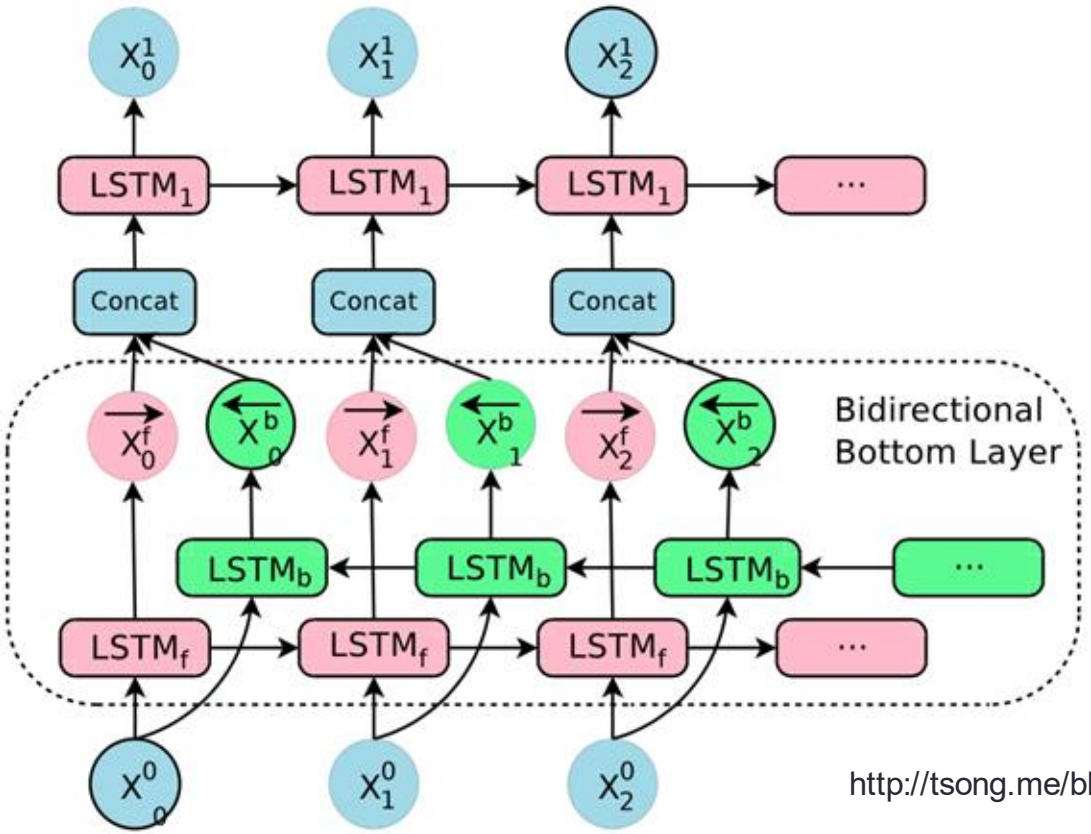
- Have 3 gates, forget (f), input (i), output (o)
- Has an explicit memory cell (c)
  - Does not have to output the memory





Bi-directional LSTM

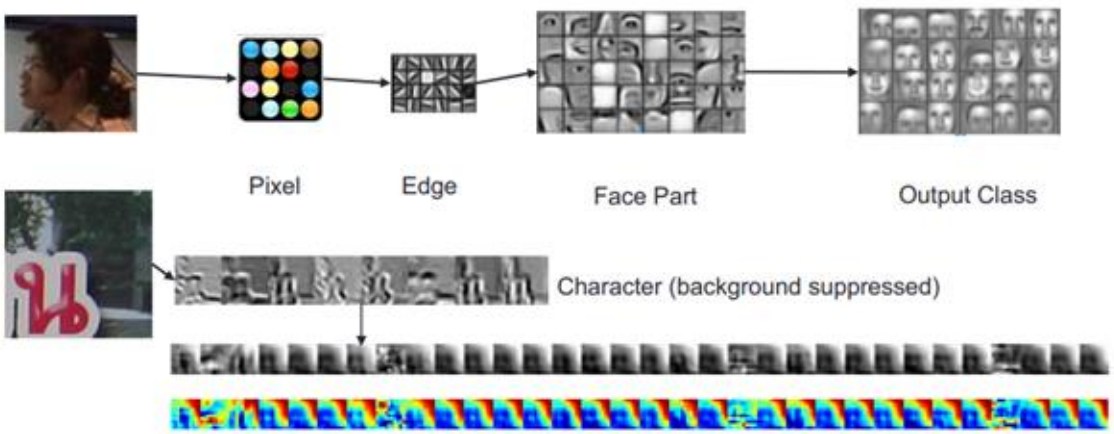
- The previous GRU/LSTM only goes backward in time (uni-directional)
- Most of the time information from the future is useful for predicting the current output

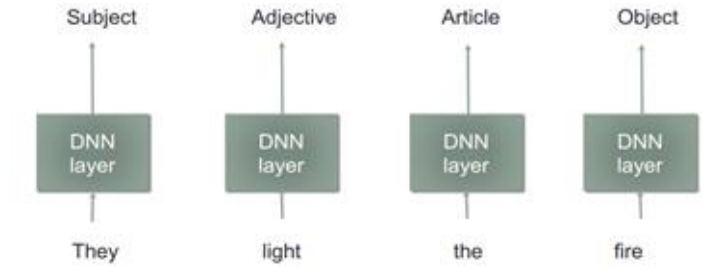


http://tsong.me/blog/google-nmt/

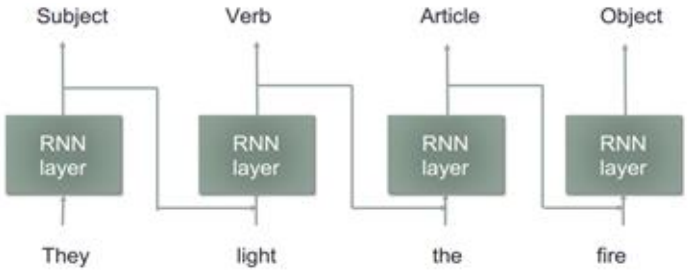
CNN and RNN recap

- CNN
  - Assume that **Spatial** information is useful
  - Convolutional layer has shift-invariant property
- RNN families (RNN, GRU, LSTM)
  - Assume that **Temporal** information is useful





Problem 1: need a way to remember the past

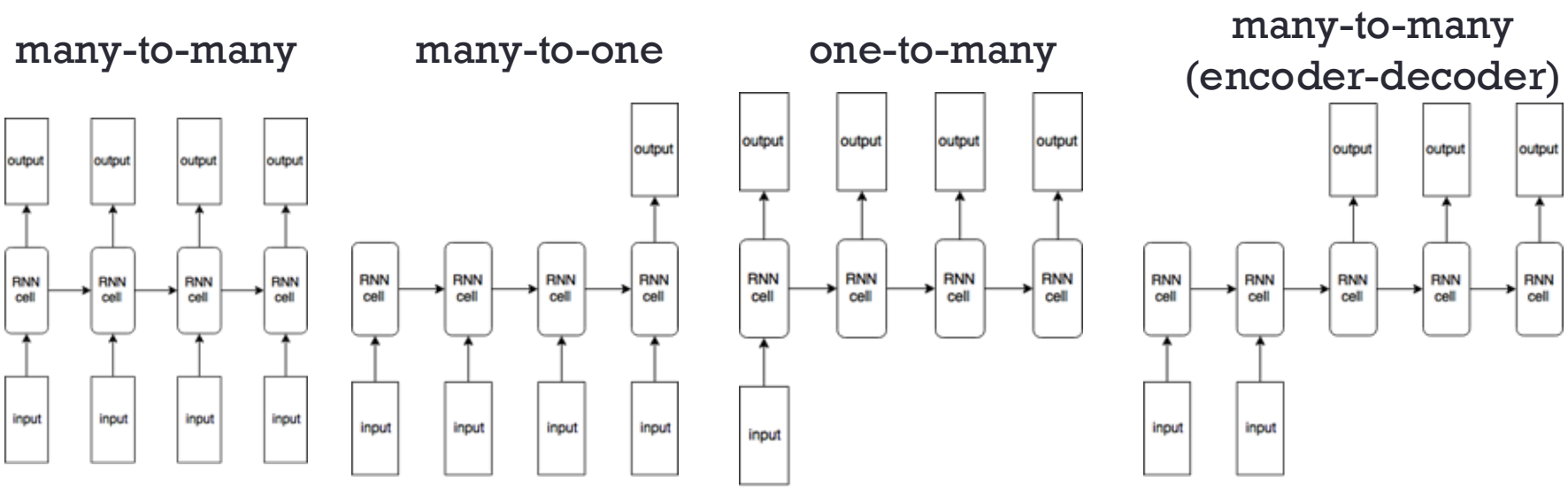


Output of the layer encodes something meaningful about the past

Structure of this lecture

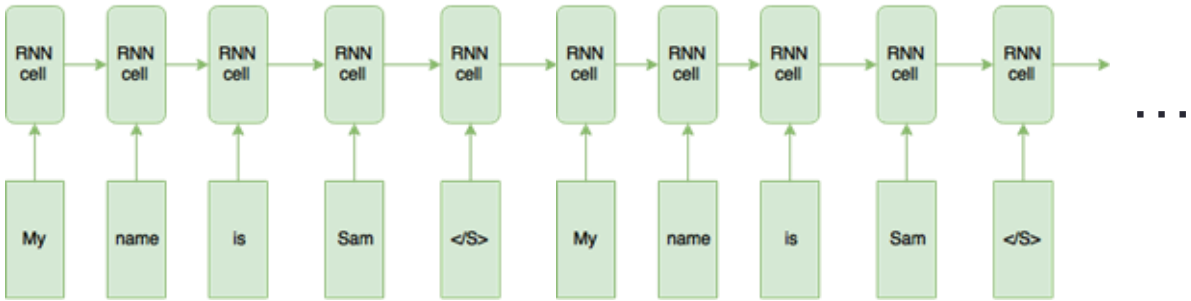
- Attention
- Deep Learning Architectures in Computer Vision

Different recurrent architectures



Drawbacks of CNN and RNN

- CNN
  - Shift-invariant property is actually a double edged sword
    - Does not make much sense in some tasks
      - Order Prediction (Given text “[A, B, C, D]” what is the third element?”)
  - Need a lot of layer to capture long range information
- RNN families (RNN, GRU, LSTM)
  - Poor parallelism
  - Exploding / Vanishing gradient
    - Not very of good at understanding very long sequence input.



…

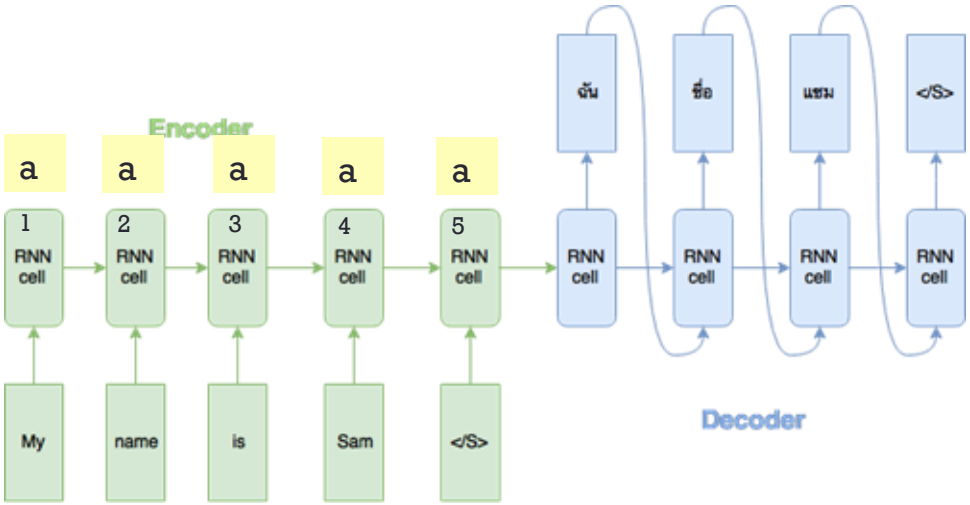
Shouldn’t there be some mechanism to mitigate this issue?

Attention Mechanism

Solution : A mechanism to capture long range information

Attention is commonly used in sequence-to-sequence model, it allows **<u>the decoder part</u>** of the network to focus/**attend** on a different part of **<u>the encoder outputs</u>** for every step of the decoder's own outputs.

This is what we want you to think about: How can information travel from one end to another in neural networks?



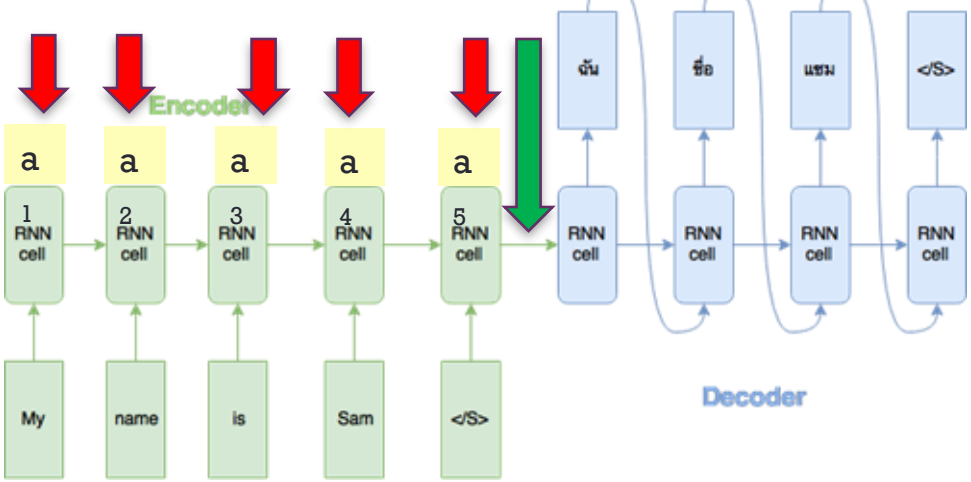
Machine Translation Problem: English to Thai

Attention Mechanism (cont.)

Why attention?

Main idea: We can use multiple vectors based on the length of the sentence instead of one.

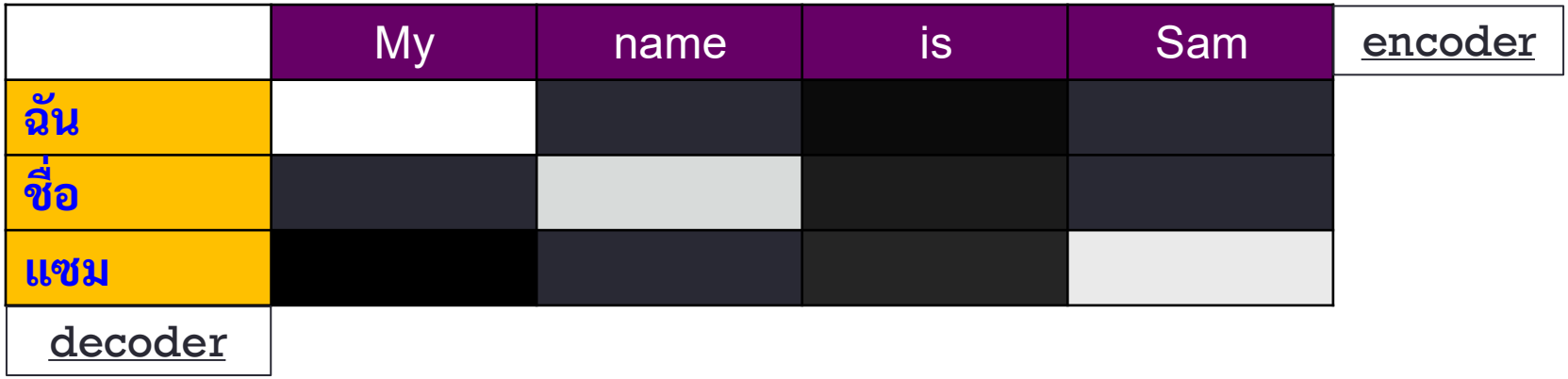
Attention mechanism = Instead of encoding all the information into a fixed-length vector, the decoder gets to decide parts of the input source to pay attention. -> **Access to Global information**



Machine Translation Problem: English to Thai

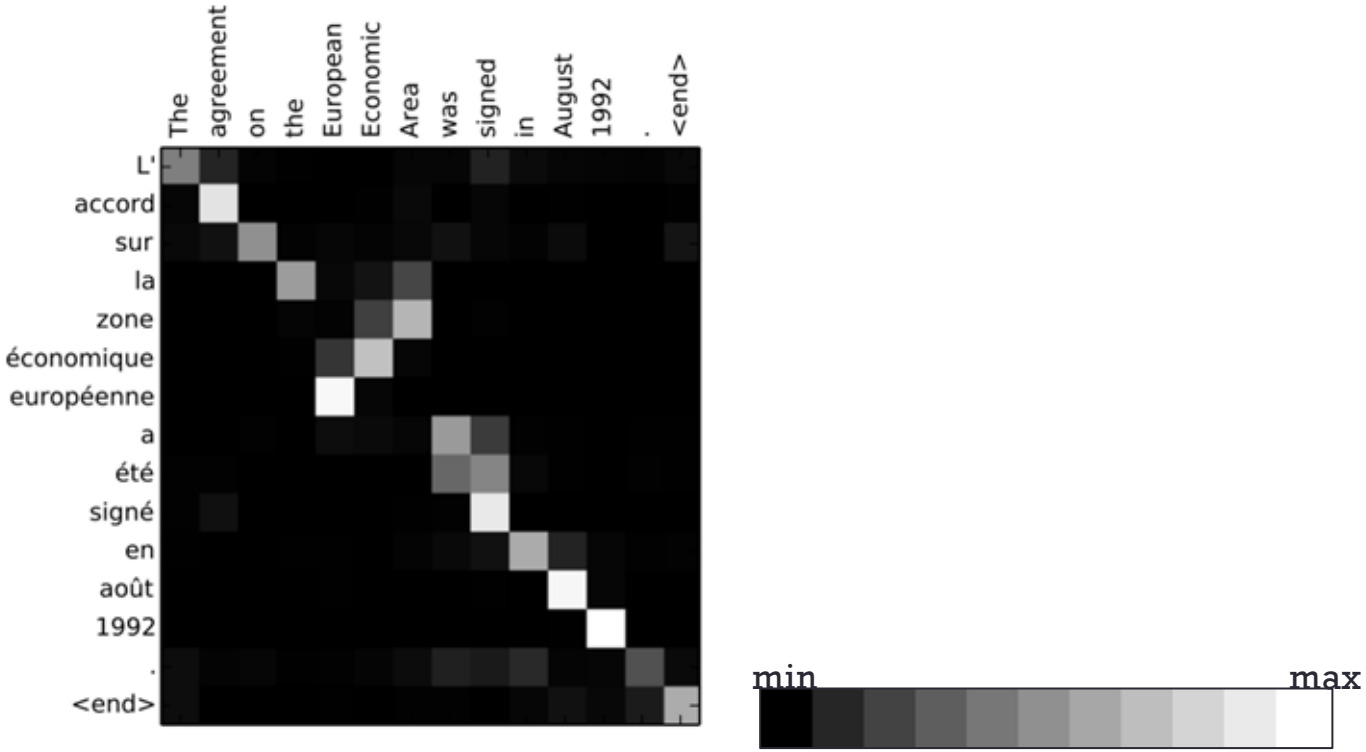
Graphical Example: English-to-Thai machine translation

- This is a rough estimate of what might occur for English-to-Thai translation



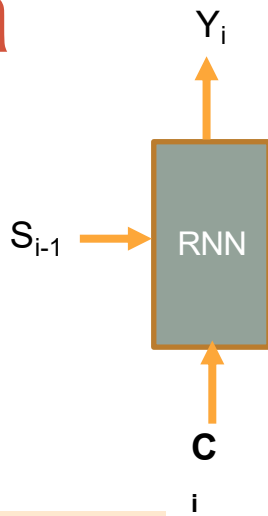
Machine Translation Problem: English to Thai

Graphical Example: English-to-French machine translation



Reference: Bahdanau, Dzmitry, Kyunghyun Cho, and Yoshua Bengio. "Neural machine translation by jointly learning to align and translate." ICLR(2015).

Attention Mechanism: Basic Idea



- Encode each word in the sequence into a vector
- When DECODING, perform a linear combination of these encoded vectors from the encoding step with their corresponding "attention weights".

$$\mathbf{c}_i = \sum_j a_{ij}\mathbf{h}_j$$

j = each encoder's input
i = each decoder's input

- A vector formed by this linear combination is called **"context vector"**
- Use context vectors as inputs for the decoding step

Reference: Bahdanau, Dzmitry, Kyunghyun Cho, and Yoshua Bengio. "Neural machine translation by jointly learning to align and translate." ICLR(2015).

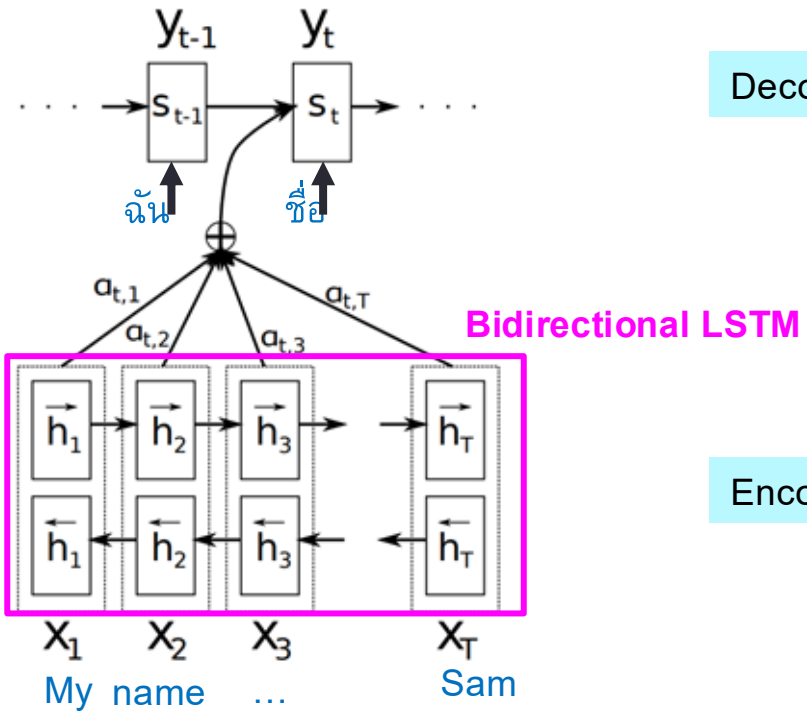


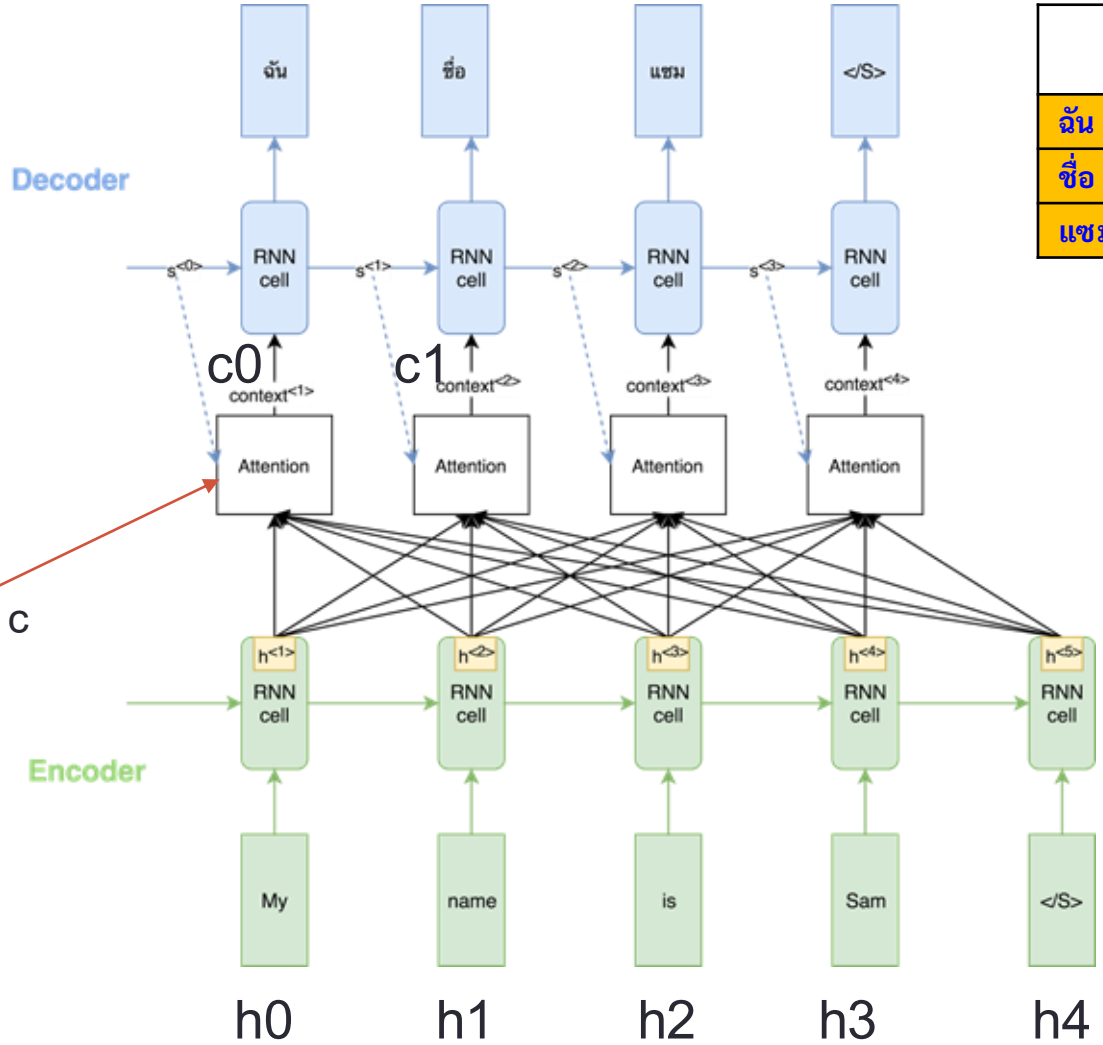
Figure 1: The graphical illustration of the proposed model trying to generate the t-th target word y_t given a source sentence $(x_1, x_2, \ldots, x_T)$.

target word = decoder

source = encoder

Reference: Bahdanau, D., Cho, K., & Bengio, Y.. Neural Machine Translation by Jointly Learning to Align and Translate. ICLR 2015

RNN and attention mechanism



receive s, h return c

	My	name	is	Sa m
ฉัน				
ชื่อ				
แซม				

Attention Mechanism (1): C_i

We want to calculate a context vector ***c*** based on hidden states $\mathbf{s}_0 \ldots \mathbf{s}_{m-1}$ that can be used with the current state $\mathbf{h}_j$ for prediction. The context vector $\mathbf{c}_i$ at position "i" is calculated as an average of the previous states weighted with the attention scores $\mathbf{a}_i$.

$$\mathbf{c}_i = \sum_j a_{ij} \mathbf{h}_j$$

context vector

attention score

encoder state at index j

i = decoder index
j = encoder index

$$a_{ij} = softmax(f_{att}(\mathbf{s}_{i-1}, \mathbf{h}_j)) \rightarrow \frac{e^{f_{attn}(\mathbf{s}_{i-1}, \mathbf{h}_j)}}{\sum_j e^{f_{attn}(\mathbf{s}_{i-1}, \mathbf{h}_j)}}$$

previous hidden state/ **decoder** state

attention score(weight vector) of encoder state at index j

encoder state at index j

Reference: http://ruder.io/deep-learning-nlp-best-practices/index.html#attention

Attention Mechanism (2): f_{att}

sum(a_ij) should be 1

$$\mathbf{c}_i = \sum_j a_{ij} \mathbf{h}_j$$

context vector

attention score

encoder state at index j

The attention function $f_{att}(\mathbf{s}_{i-1}, \mathbf{h}_j)$ calculates an unnormalized alignment score between the current hidden state $\mathbf{s}_{i-1}$ and the previous hidden state $\mathbf{h}_j$.

There are many variants of the attention function f_{att}.

i = decoder index
j = encoder index

$$a_{ij} = softmax(f_{att}(\mathbf{s}_{i-1}, \mathbf{h}_j)) \rightarrow \frac{e^{f_{attn}(\mathbf{s}_{i-1}, \mathbf{h}_j)}}{\sum_j e^{f_{attn}(\mathbf{s}_{i-1}, \mathbf{h}_j)}}$$

previous hidden state/
decoder state

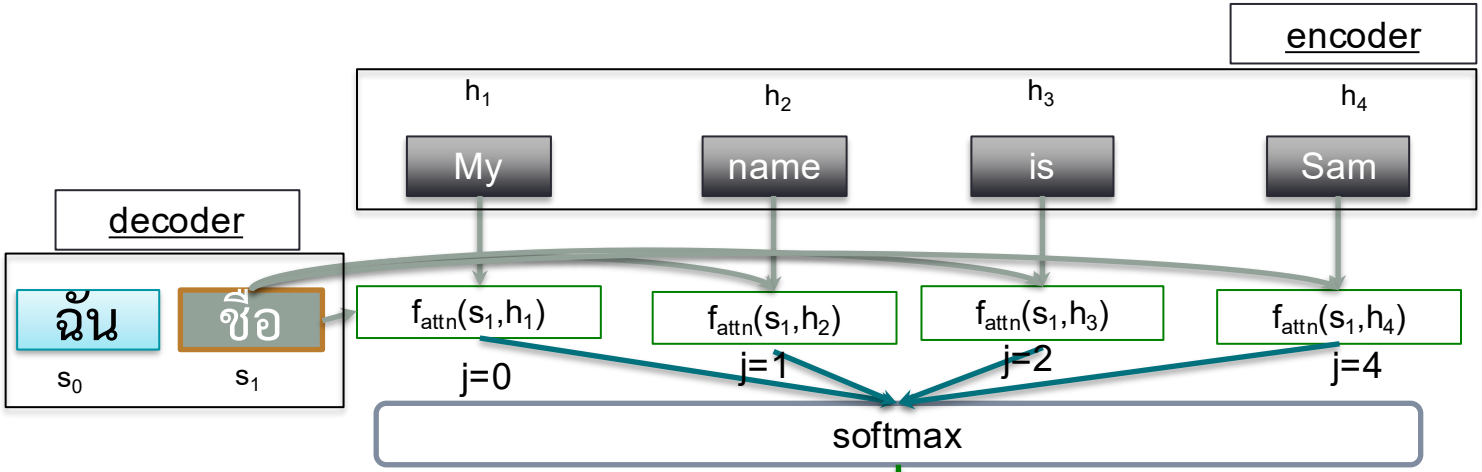
attention score(weight vector)
of encoder state at index j

encoder state at index j

Reference: http://ruder.io/deep-learning-nlp-best-practices/index.html#attention

Attention Calculation Example (1): Attention Scores

Now we want to predict แซม **(i=2)**



$$a_{ij} = softmax(f_{att}(\mathbf{s}_{i-1}, \mathbf{h}_j))$$

$\mathbf{a}_2=[a_{2,1}\ a_{2,2}\ a_{2,3}\ a_{2,4}]$

i = decoder index
j = encoder index

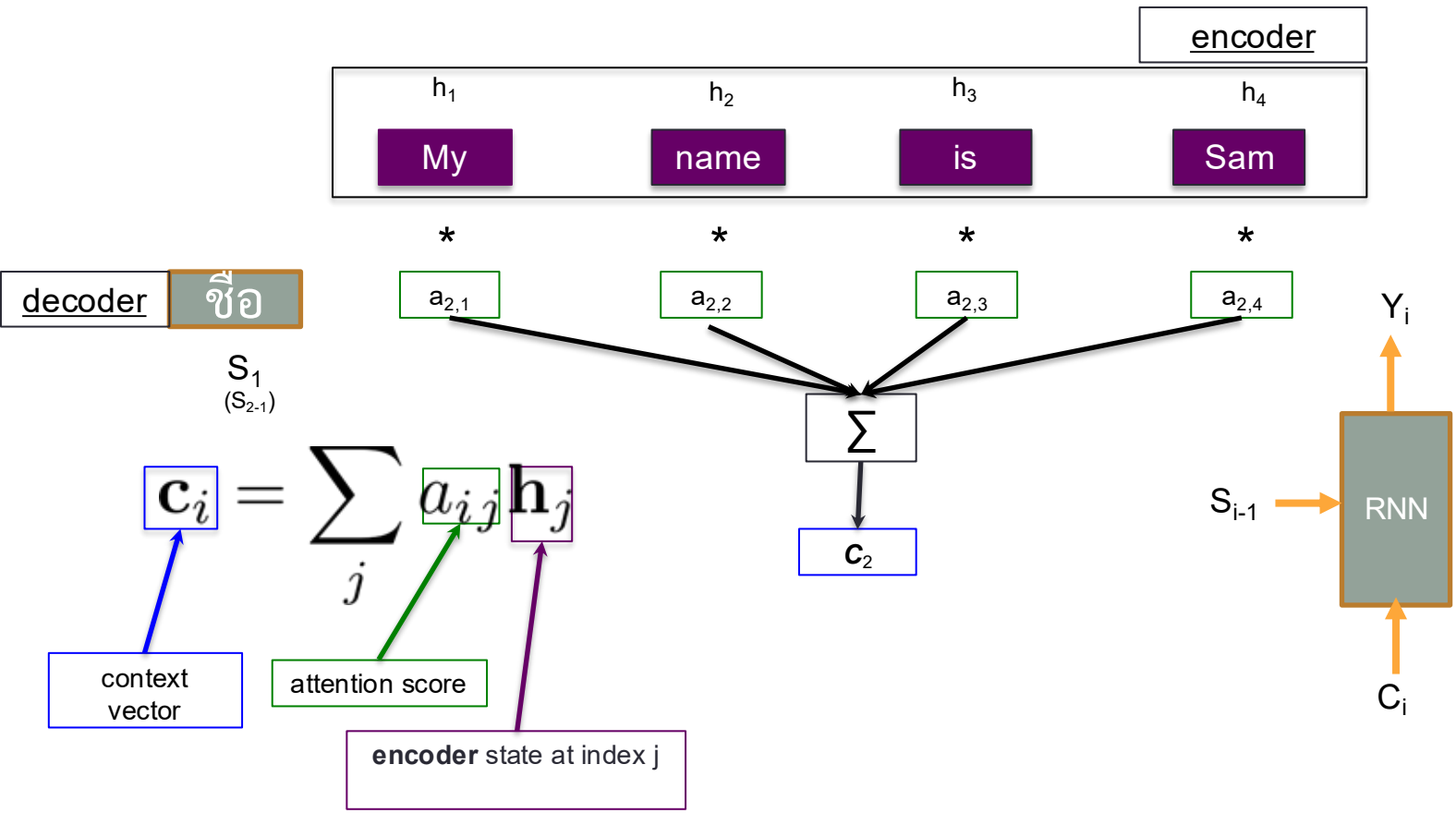
attention scores

previous hidden state/ decoder state

encoder state at index j

$$\frac{e^{f_{attn}(\mathbf{s}_{i-1},\mathbf{h}_j)}}{\sum_j e^{f_{attn}(\mathbf{s}_{i-1},\mathbf{h}_j)}}$$

Attention Calculation Example (2): Context Vector



$$\mathbf{c}_i = \sum_j a_{ij} \mathbf{h}_j$$

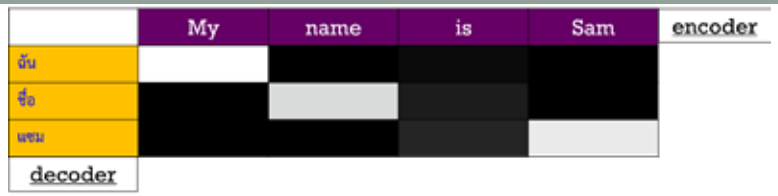
context vector

attention score

encoder state at index j

$$a_{ij} = softmax(f_{att}(\mathbf{s}_{i-1}, \mathbf{h}_j))$$

Type of Attention mechanisms



(Remember that there are many variants of attention function f_{attn})

Additive attention: The original attention mechanism (Bahdanau et al., 2015) uses a one-hidden layer feed-forward network to calculate the attention alignment:

h_i = encoder index
s_j = decoder index

$$f_{attn}(\mathbf{s}_{i-1}, \mathbf{h}_j) = v_a^T tanh(\mathbf{W}_a[\mathbf{s}_{i-1}; \mathbf{h}_j])$$

Multiplicative attention: Multiplicative attention (Luong et al., 2015) simplifies the attention operation by calculating the following function:

$$f_{attn}(\mathbf{s}_{i-1}, \mathbf{h}_j) = \mathbf{s}_{i-1}^\top \mathbf{W}_a \mathbf{h}_j$$

Self-attention: Without any additional information, however, we can still extract relevant aspects from the sentence by allowing it to attend to itself using self-attention (Lin et al., 2017)

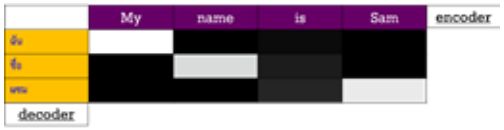
$$\mathbf{a} = softmax(\mathbf{w}_{s_2} tanh(\mathbf{W}_{s_1} \mathbf{H}^T))$$

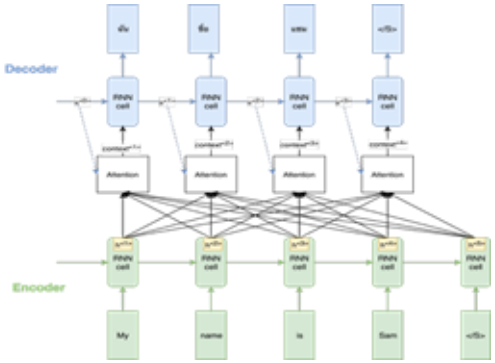
Key-value attention: key-value attention (Daniluk et al., 2017) is a recent attention variant that separates form from function by keeping separate vectors for the attention calculation.

Reference: http://ruder.io/deep-learning-nlp-best-practices/index.html#attention

$$a_{ij} = softmax(f_{att}(\mathbf{s}_{i-1}, \mathbf{h}_j))$$

Additive Attention





- The original attention mechanism (Bahdanau et al., 2015) uses a one-hidden layer feed-forward network to calculate the attention alignment:

concatenation

$$f_{attn}(\mathbf{s}_{i-1}, \mathbf{h}_j) = v_a^T tanh(\mathbf{W}_a[\mathbf{s}_{i-1}; \mathbf{h}_j])$$

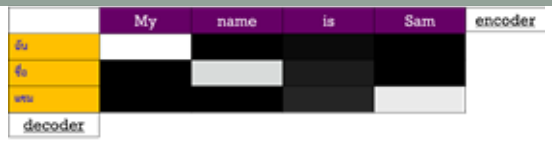
One-hidden layer (Dense)

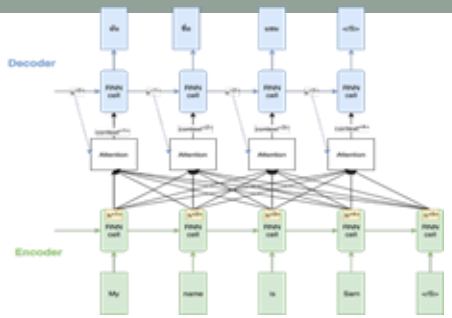
- Where $\mathbf{W}_a$ are learned attention parameters. Analogously, we can also use matrices $\mathbf{W}_1$ and $\mathbf{W}_2$ to learn separate transformations for $\mathbf{s}_{i-1}$ and $\mathbf{h}_j$ respectively, which are then summed (hence the name additive):

$$f_{attn}(\mathbf{s}_{i-1}, \mathbf{h}_j) = v_a^T tanh(\mathbf{W}_1\mathbf{s}_{i-1} + \mathbf{W}_2\mathbf{h}_j)$$

Reference: http://ruder.io/deep-learning-nlp-best-practices/index.html#attention

$$a_{ij} = softmax(f_{att}(\mathbf{s}_{i-1}, \mathbf{h}_j))$$





Multiplicative Attention

- Multiplicative attention (Luong et al., 2015) [16] simplifies the attention operation by calculating the following function:

$$f_{attn}(\mathbf{s}_{i-1}, \mathbf{h}_j) = \mathbf{s}_{i-1}^{\top} \mathbf{W}_a \mathbf{h}_j$$

- Faster, more efficient than additive attention BUT additive attention performs better for larger dimensions
- One way to mitigate this is to scale f_{attn} by $\frac{1}{\sqrt{d_s}}$

d_s = #dimensions of hidden states in LSTM (context vector; latent factors)

  - Dot product of high dimensional vectors has high variance -> softmax is peaky -> small gradient -> harder to train

Additive attention

$$f_{attn}(\mathbf{s}_{i-1}, \mathbf{h}_j) = v_a^T tanh(\mathbf{W}_a[\mathbf{s}_{i-1}; \mathbf{h}_j]) \longrightarrow f_{attn}(\mathbf{h}_i, \mathbf{h}_j) = v_a^T tanh(\mathbf{W}_a[\mathbf{h}_i; \mathbf{h}_j])$$

Self Attention (1)

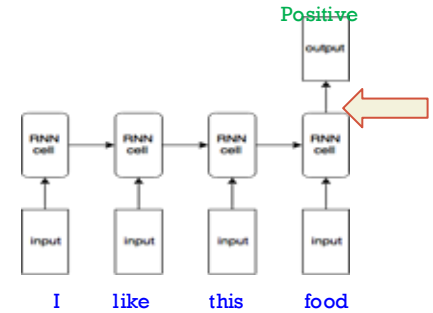
- Without any additional information, we can still extract relevant aspects from the sentence by allowing it to attend to itself using self-attention (Lin et al., 2017)

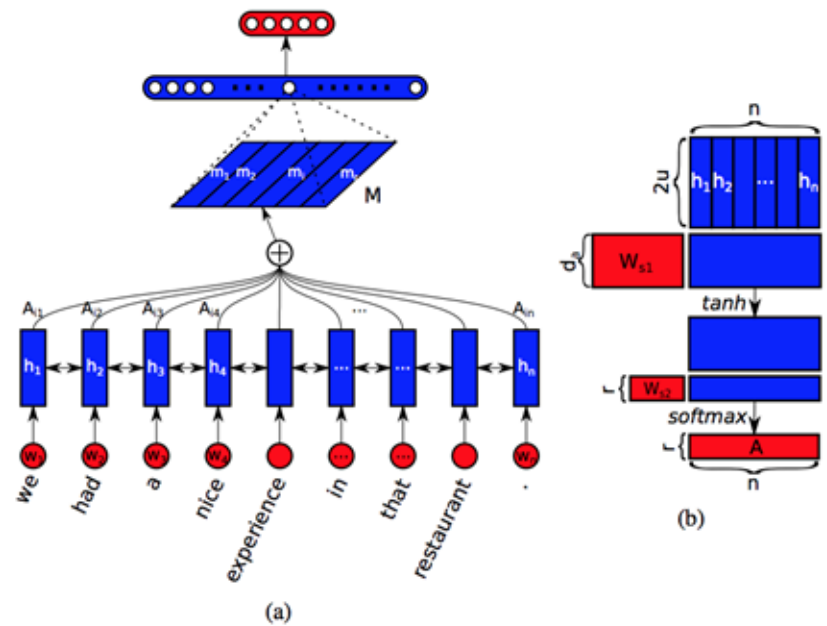
$$H = (\mathbf{h_1}, \mathbf{h_2}, \cdots \mathbf{h_n})$$

$$\mathbf{a} = \overbrace{softmax(\mathbf{w}_{s_2}}^{\text{Fully connected layer}} \underbrace{tanh(\mathbf{W}_{s_1}}_{\text{One-hidden layer (Dense)}} \mathbf{H}^T)$$

- w_{s1} is a weight matrix, w_{s2} is a vector of parameters. Note that these parameters are tuned by the neural networks.

- **The objective is to improve a quality of embedding vector by adding context information.**





Self-attention (2)

- if I can give this restaurant a 0 I will we be just ask our waitress leave because someone with a reservation be wait for our table my father and father-in-law be still finish up their coffee and we have not yet finish our dessert I have never be so humiliated do not go to this restaurant their food be mediocre at best if you want excellent Italian in a small intimate restaurant go to dish on the South Side I will not be go back
- this place suck the food be gross and taste like grease I will never go here again ever sure the entrance look cool and the waiter can be very nice but the food simply be gross taste like cheap 99cent food do not go here the food shot out of me quick then it go in
- everything be pre cook and dry its crazy most Filipino people be used to very cheap ingredient and they do not know quality the food be disgusting I have eat at least 20 different Filipino family home this not even mediocre
- seriously f *** this place disgust food and shitty service ambience be great if you like dine in a hot cellar engulf in stagnate air truly it be over rate over price and they just under deliver forget try order a drink here it will take forever get and when it finally do arrive you will be ready pass out from heat exhaustion and lack of oxygen how be that a head change you do not even have pay for it I will not disgust you with the detailed review of everything I have try here but make it simple it all suck and after you get the bill you will be walk out with a sore ass save your money and spare your self the disappointment
- i be so angry about my horrible experience at Medusa today my previous visit be amaze 5/5 however my go to out of town and I land an appointment with Stephanie I go in with a picture of roughly what I want and come out look absolutely nothing like it my hair be a horrible ashy blonde not anywhere close to the platinum blonde I request she will not do any of the pop of colour I want and even after specifically tell her I do not like blunt cut my hair have lot of straight edge she do not listen to a single thing I want and when I tell her I be unhappy with the colour she basically tell me I be wrong and I have do it this way no no I do not if I can go from Little Mermaid red to golden blonde in 1 sitting that leave my hair fine I shall be able go from golden blonde to a shade of platinum blonde in 1 sitting thanks for ruin my New Year's with 1 the bad hair job I have ever have

(a) 1 star reviews

- I really enjoy Ashley and Ami salon she do a great job be friendly and professional I usually get my hair do when I go to MI because of the quality of the highlight and the price the price be very affordable the highlight fantastic thank Ashley i highly recommend you and ill be back
- love this place it really be my favorite restaurant in Charlotte they use charcoal for their grill and you can taste it steak with chimichurri be always perfect Fried yucca cilantro rice pork sandwich and the good tres lech I have had.The desert be all incredible if you do not like it you be a mutant if you will like diabeetus try the Inca Cola
- this place be so much fun I have never go at night because it seem a little too busy for my taste but that just prove how great this restaurant be they have amazing food and the staff definitely remember us every time we be in town I love when a waitress or waiter come over and ask if you want the cab or the Pinot even when there be a rush and the staff be run around like crazy whenever I grab someone they instantly smile acknowlegde us the food be also killer I love when everyone know the special and can tell you they have try them all and what they pair well with this be a first last stop whenever we be in Charlotte and I highly recommend them
- great food and good service .... what else can you ask for everything that I have ever try here have be great
- first off I hardly remember waiter name because its rare you have an unforgettable experience the day I go I be celebrate my birthday and let me say I leave feel extra special our waiter be the best ever Carlos and the staff as well I be with a party of 4 and we order the potato salad shrimp cocktail lobster amongst other thing and boy be the food great the lobster be the good lobster I have ever eat if you eat a dessert I will recommend the cheese cake that be also the good I have ever have it be expensive but so worth every penny I will definitely be back there go again for the second time in a week and it be even good ..... this place be amazing

(b) 5 star reviews

Figure 2: Heatmap of Yelp reviews with the two extreme score.

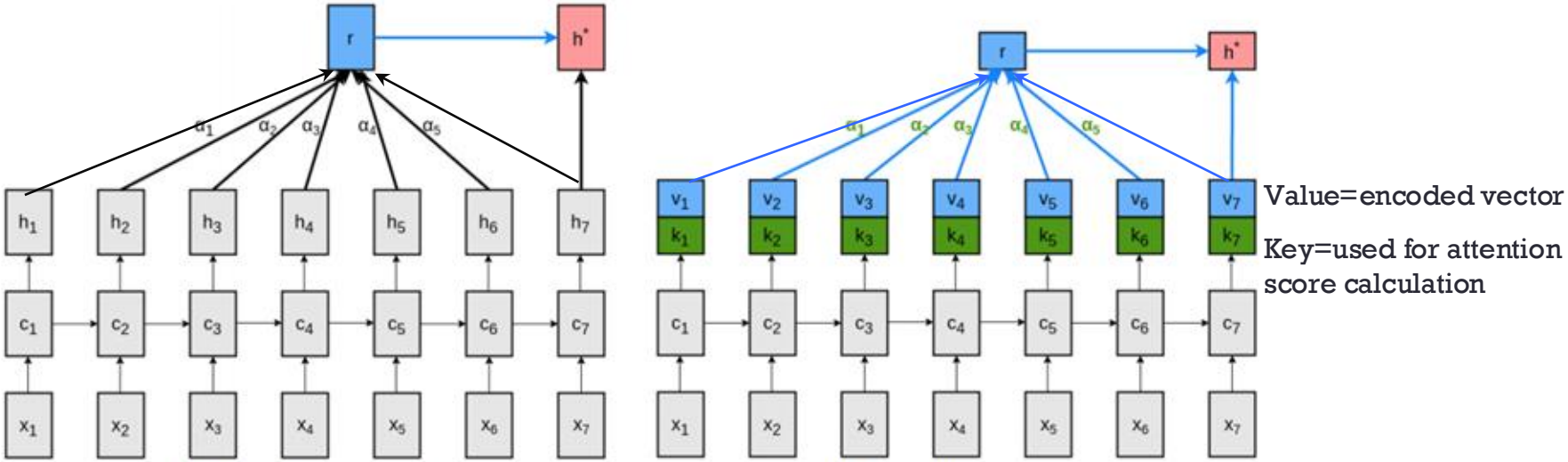
$$a_{ij} = softmax(f_{att}(\mathbf{s}_{i-1}, \mathbf{h}_j))$$

$$\mathbf{c}_i = \sum_j a_{ij}\mathbf{h}_j$$

Key-value attention (1)

Separate **h** in to **k**, **v** to specialize the neurons

k for learning attentions, **v** for sending information



(a) Neural language model with attention.

(b) Key-value separation.

Reference: Daniluk, M., Rockt, T., Welbl, J., & Riedel, S. (2017). Frustratingly Short Attention Spans in Neural Language Modeling. In ICLR 2017.

$$f_{attn}(\mathbf{s}_{i-1}, \mathbf{h}_j) = v_a^T tanh(\mathbf{W}_a[\mathbf{s}_{i-1}; \mathbf{h}_j])$$

Key-value attention (2)

previous outputs (memory) **with window L**

current output

k - output dimension
L - output length

key

value

$$\begin{bmatrix} \boldsymbol{k}_t \\ \boldsymbol{v}_t \end{bmatrix} = \boldsymbol{h}_t \quad \in \mathbb{R}^{2k}$$

$$\boldsymbol{M}_t = \tanh(\boldsymbol{W}^Y [\boldsymbol{k}_{t-L} \cdots \boldsymbol{k}_{t-1}] + (\boldsymbol{W}^h \boldsymbol{k}_t)\mathbf{1}^T) \quad \in \mathbb{R}^{k \times L}$$

Attention score

$$\boldsymbol{\alpha}_t = \text{softmax}(\boldsymbol{w}^T \boldsymbol{M}_t) \quad \in \mathbb{R}^{1 \times L}$$

Context vector (v_t not included)

$$\boldsymbol{r}_t = [\boldsymbol{v}_{t-L} \cdots \boldsymbol{v}_{t-1}]\boldsymbol{\alpha}^T \quad \in \mathbb{R}^k$$

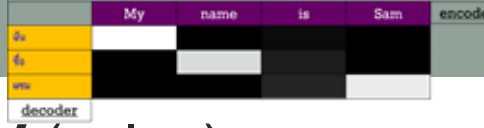
$$\boldsymbol{h}_t^* = \tanh(\boldsymbol{W}^r \boldsymbol{r}_t + \boldsymbol{W}^x \boldsymbol{v}_t) \quad \in \mathbb{R}^k$$

The final representation

Context vector (v_t not included)

Original vector at index t

$$\mathbf{c}_i = \sum_j a_{ij} \mathbf{h}_j$$



$$a_{ij} = softmax(f_{att}(\overset{Q}{\mathbf{s}_{i-1}}, \overset{K,V}{\mathbf{h}_j}))$$

QKV attention

Q (Query), **K** (Key), **V** (value)
V for sending information
Q, K for interaction

$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$

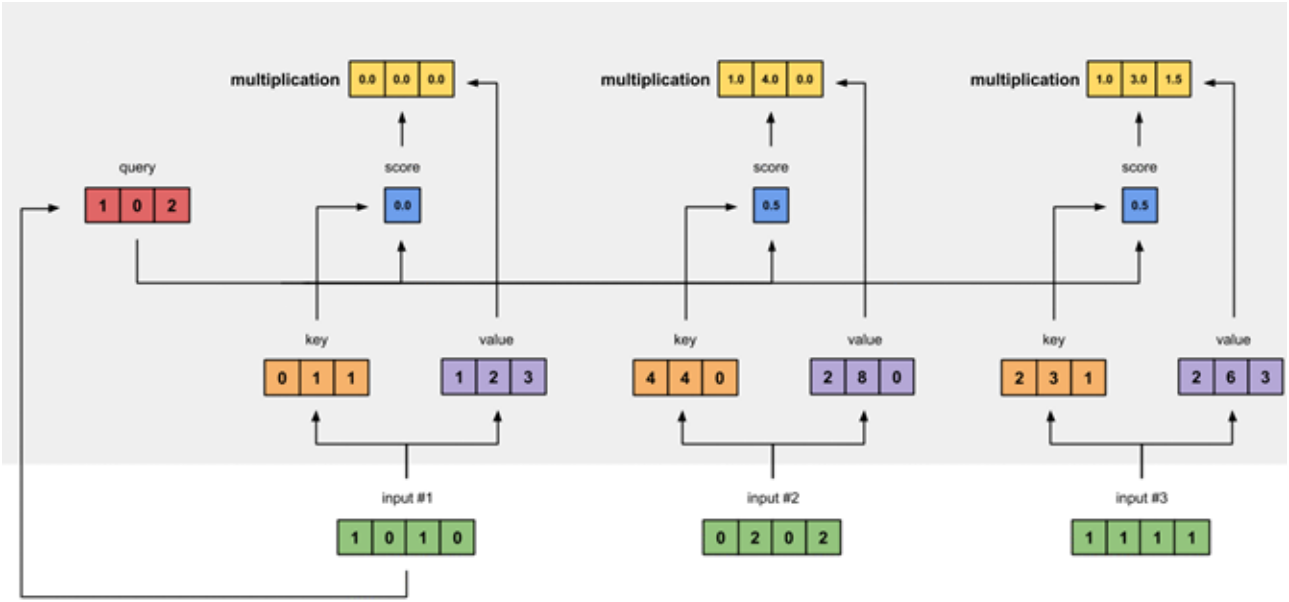
$$Q = W_Q s \quad K = W_K h \quad V = W_V h$$

$$a_{ij} = softmax(\frac{Q_i K_j^T}{\sqrt{d_k}})$$

$$y_i = \sum_j a_{ij} V_i$$

$$Q = W_Q h$$

For self-attention



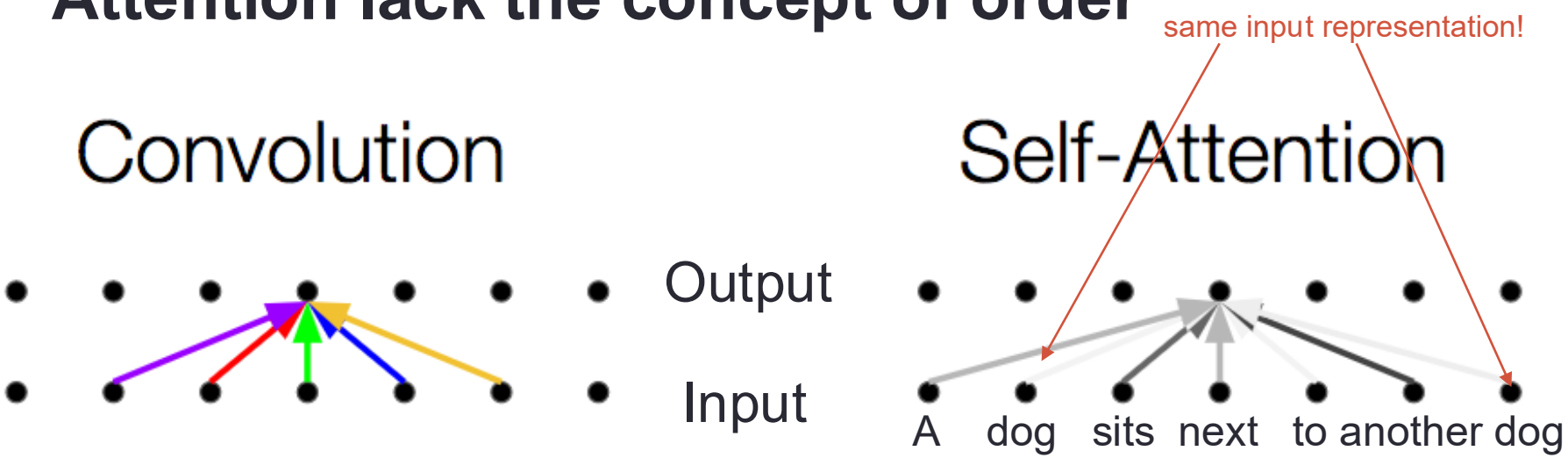
https://towardsdatascience.com/illustrated-self-attention-2d627e33b20a

Convolution vs Self attention

Limited context vs Wide context
- Convolution has limited scope (filter size)
- Self-attention has **different learnable weight** for each output
- **Downside: O(n^2) (length^2) when computing attention weights**

Fixed weights vs weights changes according to input

Attention lack the concept of order

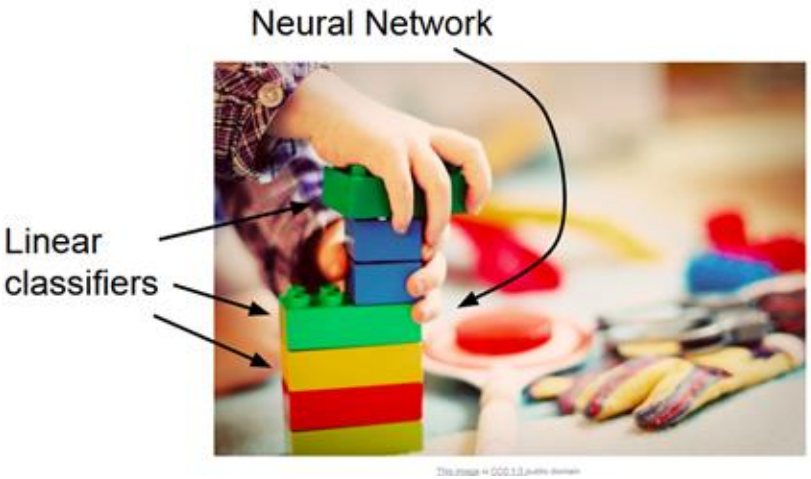


https://nlp.stanford.edu/seminar/details/lkaiser.pdf

Neural Network Architectures

DNN Legos

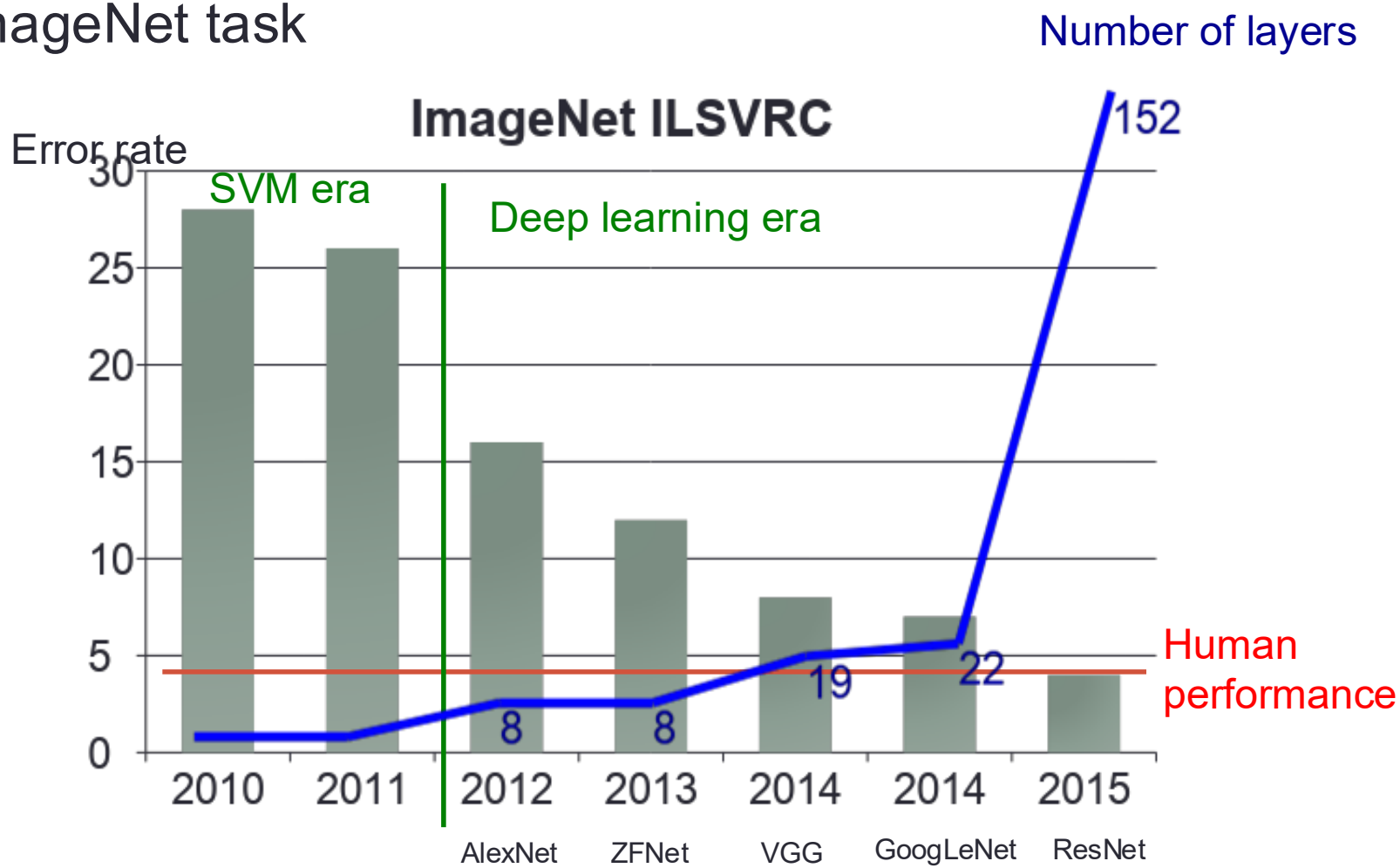
- Typical models now consists of all 3 types
  - CNN: local structure in the feature. Used for feature learning.
  - RNN (GRU/LSTM) : remembering longer term structure or across time
  - DNN: Good for mapping features for classification. Usually used in final layers
  - Attention : remember long term structure / global information
- DNN structures encode inductive bias



https://www.analyticsvidhya.com/blog/2017/08/10-advanced-deep-learning-architectures-data-scientists/

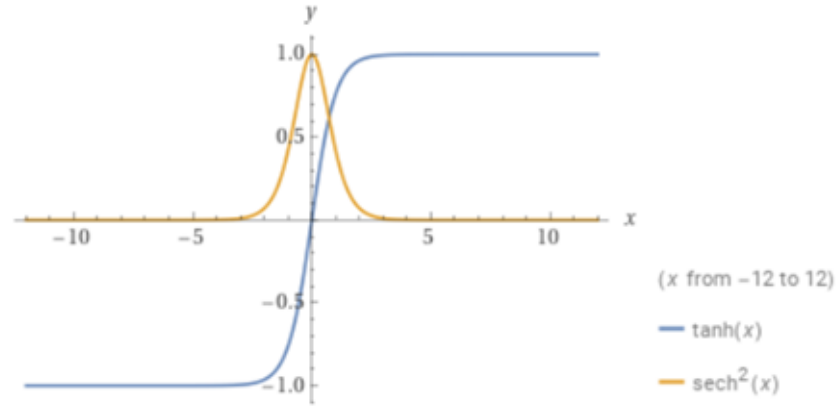
A brief history of imagenet architectures

- ImageNet task



Olga Russakovsky, ImageNet Large Scale Visual Recognition Challenge, 2014 https://arxiv.org/abs/1409.0575

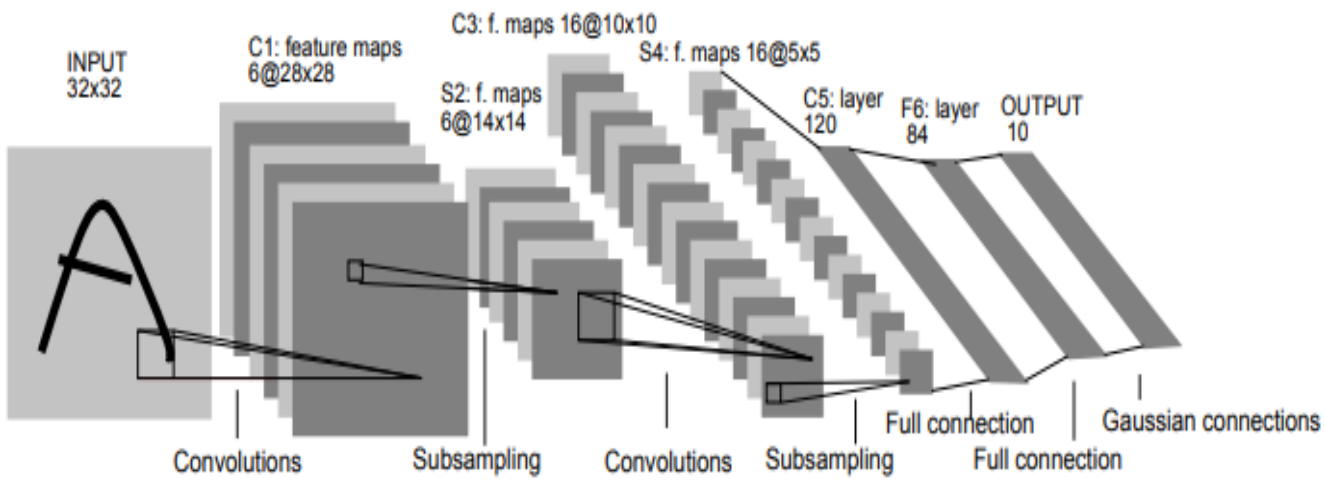
LeNet



Convolutions and poolings followed by fully connected layers

Tanh activations (zero-centering)

Ability to handle larger images limited by compute



Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner. “Gradient-based learning applied to document recognition,” 1998.

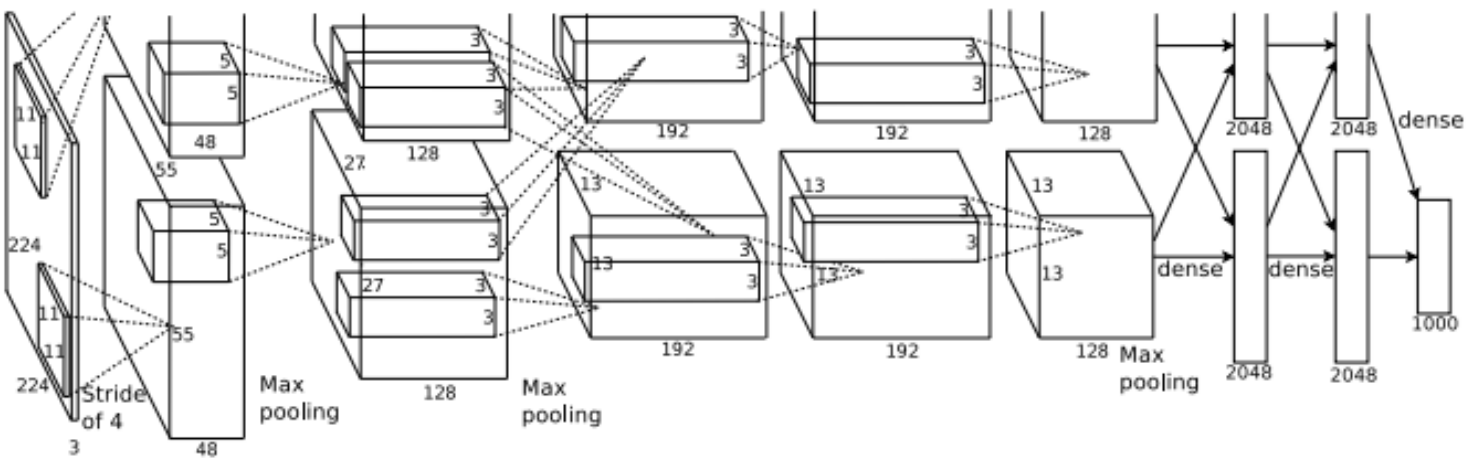
AlexNet

Doesn't make very much sense in computer vision

non-zero centering (potential gradient explosion)

Convolutions, max pooling, dropout, data augmentation, ReLU activations, SGD with momentum

Two pipelines to fit into two GPUs (GTX 580, 3GB, 2010)



Alex Krizhevsky, et al. "ImageNet Classification with Deep Convolutional Neural Networks", 2012.

Fun facts

Apparently, Schmidhuber did it on GPU first (2011)

Flexible, High Performance Convolutional Neural Networks for Image Classification

Dan C. Cireşan, Ueli Meier, Jonathan Masci, Luca M. Gambardella, Jürgen Schmidhuber
IDSIA, USI and SUPSI
Galleria 2, 6928 Manno-Lugano, Switzerland
{dan,ueli,jonathan,luca,juergen}@idsia.ch

Abstract

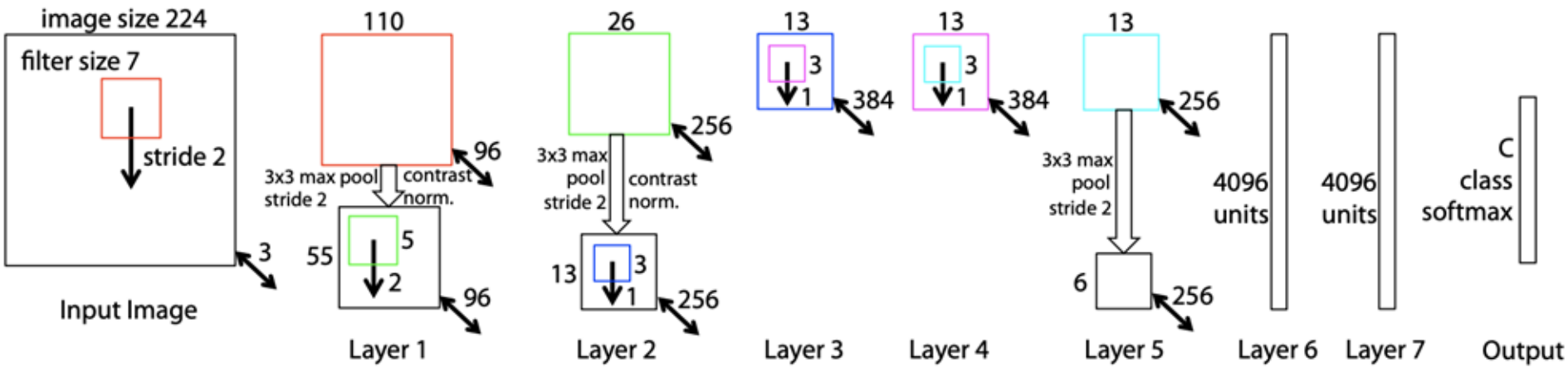
We present a fast, fully parameterizable GPU implementation of Convolutional Neural Network variants. Our feature extractors are neither carefully designed nor pre-wired, but rather learned in a supervised way. Our deep hierarchical architectures achieve the best published results on benchmarks for object classification (NORB, CIFAR10) and handwritten digit recognition (MNIST), with error rates of 2.53%, 19.51%, 0.35%, respectively. Deep nets trained by simple back-propagation perform better than more shallow ones. Learning is surprisingly rapid. NORB is completely trained within five epochs. Test error rates on MNIST drop to 2.42%, 0.97% and 0.48% after 1, 3 and 17 epochs, respectively.

(CNNs) [LeCun *et al.*, 1998; Behnke, 2003; Simard *e*
2003], whose weights (filters) are randomly initialized
changed in a supervised way using back-propagation (B

Despite the hardware progress of the past decades, co
tational speed is still a limiting factor for CNN architec
characterized by many building blocks typically set by
and error. To systematically test the impact of various a
tectures on classification performance, we present a fast
implementation on Graphics Processing Units (GPUs). P
ous GPU implementations of CNNs [Chellapilla *et al.*, 2
Uetz and Behnke, 2009; Strigl *et al.*, 2010] were hard-c
to satisfy GPU hardware constraints or use general pu
libraries, whereas our implementation is flexible and full
line (i.e., weight updates after each image). A notable e
tion is [Jarrett *et al.*, 2009] who performed a thorough a
sis of the influence of all building blocks of a multistag
chitecture on recognition performance. Our implement

ZFNet

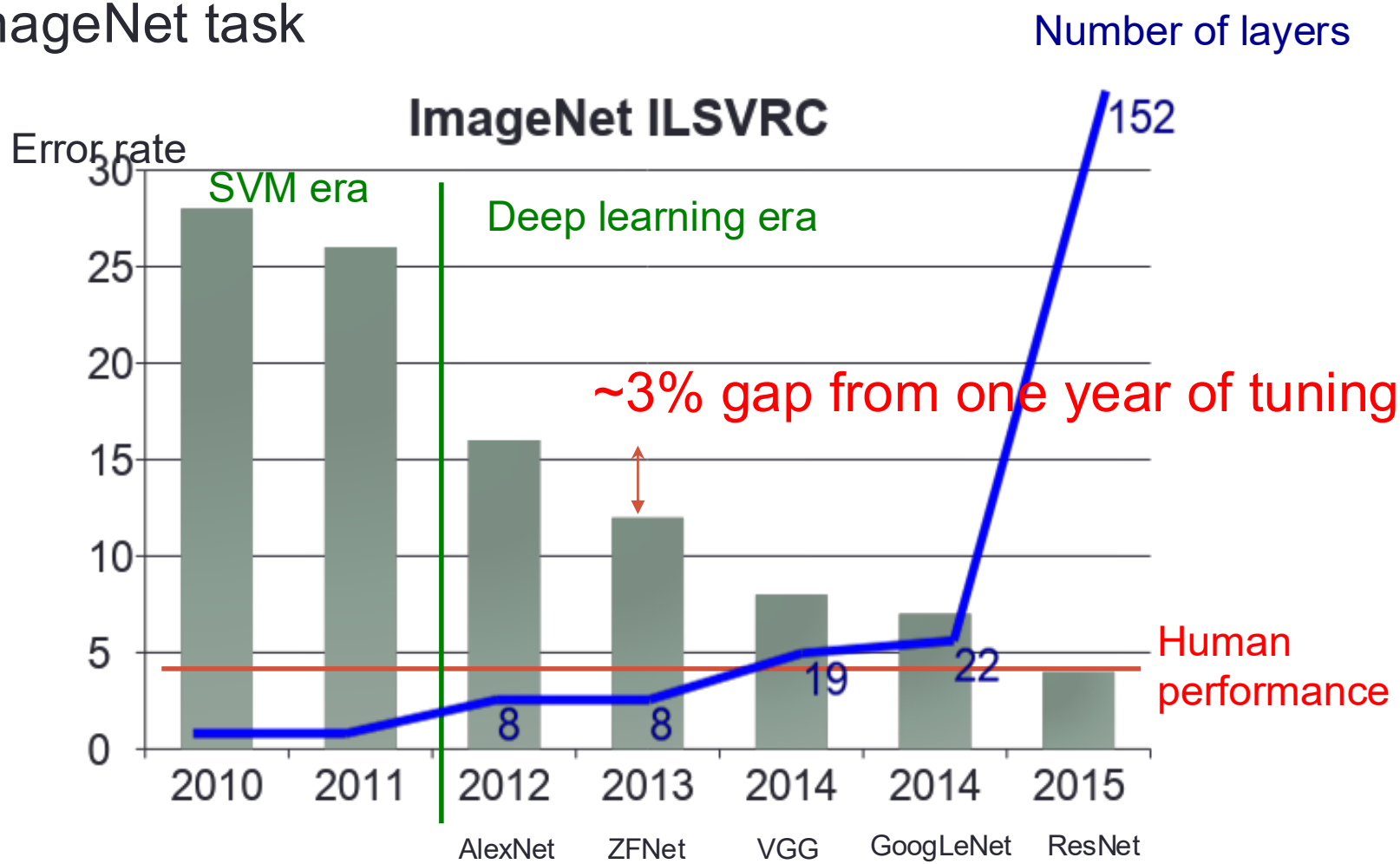
- Tweaking hyperparameters
- same GPU (GTX 580) but use only one



Matthew D. Zeiler, Rob Fergus, “Visualizing and Understanding Convolutional Networks,” 2013

A brief history of imagenet architectures

- ImageNet task



Olga Russakovsky, ImageNet Large Scale Visual Recognition Challenge, 2014 https://arxiv.org/abs/1409.0575

5x5 filter used 2.8x times more parameter than 3x3 one

VGG

Uniform 3x3 convolutional filters **(simplistic design)**

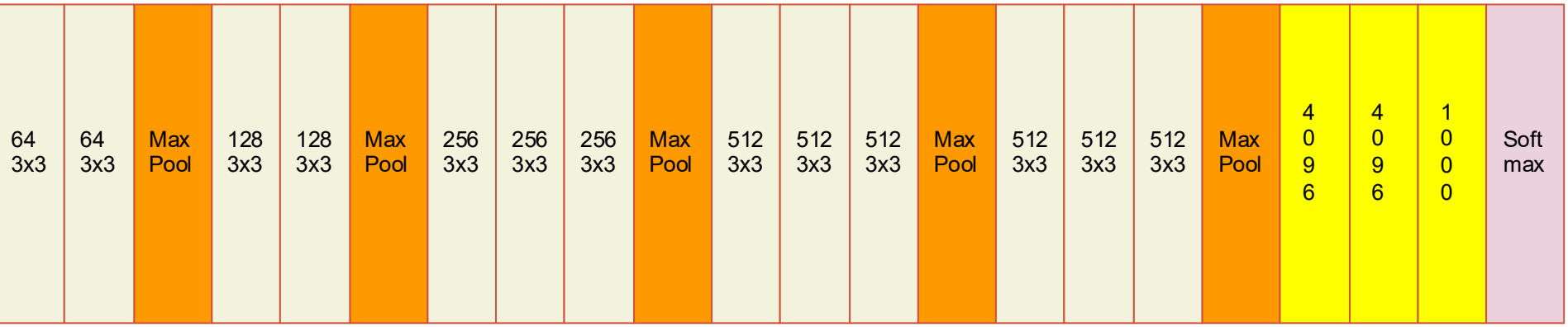
19 layers! Pushing the limits of conventional wisdom at that time.

- Initialization is very important! (Before good initializer like He, Glorot)

Used by many since pre-train weights are publically available

Trained using 4 GTX TITAN Black ! (6 GB each)

VGG16



Karen Simonyan, Andrew Zisserman"Very Deep Convolutional Networks for Large-Scale Image Recognition" 2014

AlexNet Recap

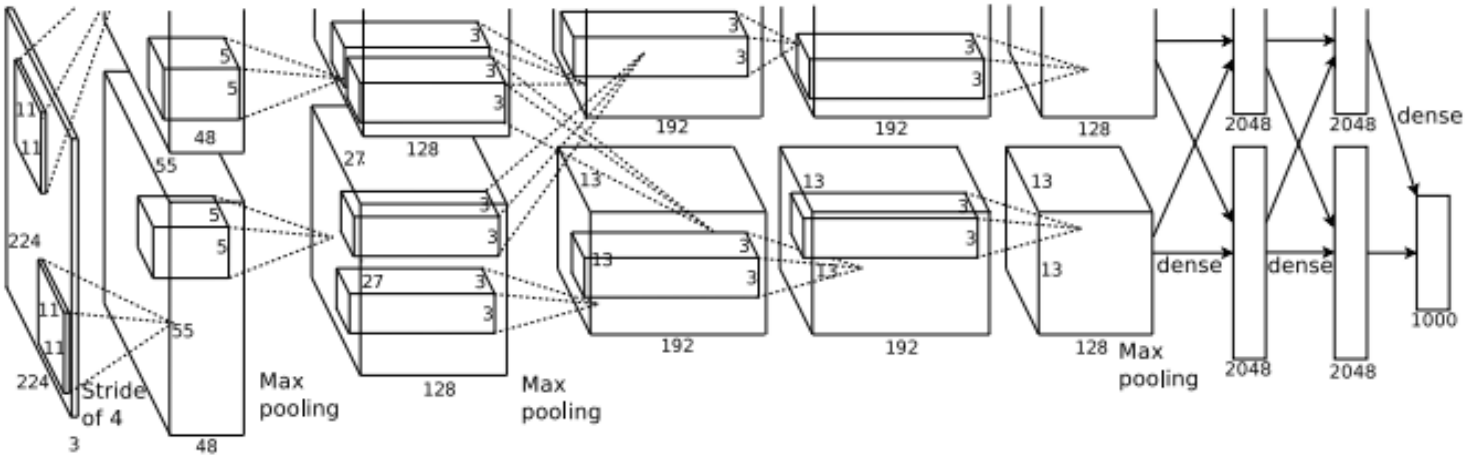
Replaced by Conv with strides

Doesn't make very much sense in computer vision

non-zero centering (potential gradient explosion)

Convolutions, max pooling, dropout, data augmentation, ReLU activations, SGD with momentum

Two pipelines to fit into two GPUs (GTX 580, 3GB, 2010)

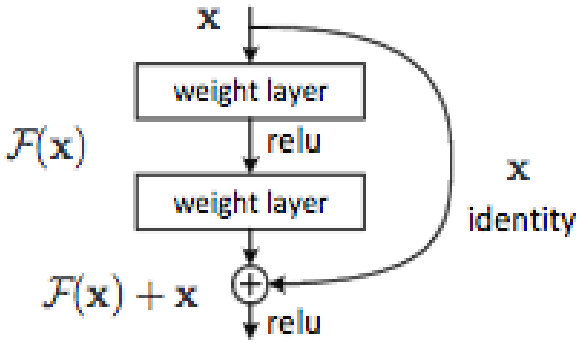


Alex Krizhevsky, et al. "ImageNet Classification with Deep Convolutional Neural Networks", 2012.

Vanishing/Exploding gradient

- Backprop introduces many multiplications down chain
- The gradient value gets smaller and smaller
  - The deeper the network the smaller the gradient in the lower layers
  - Lower layers changes too slowly (or not at all)
  - Hard to train very deep networks (>6 layers)
- The opposite can also be true. The gradient explodes from repeated multiplication
  - Put a maximum value for the gradient (Gradient clipping)

- How to deal with this?
  - Residual connection
  - Gradient injection



https://arxiv.org/abs/1512.03385

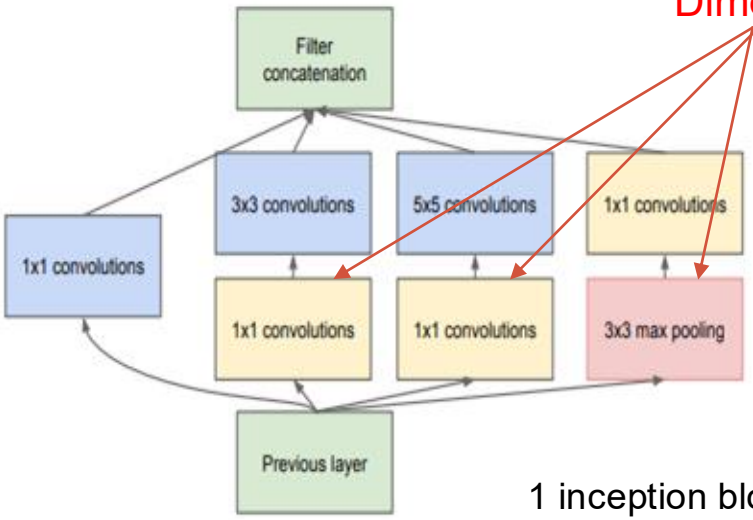
GoogLeNet (Inception v1)

Multiple filter sizes per layer (objects come in different scales)

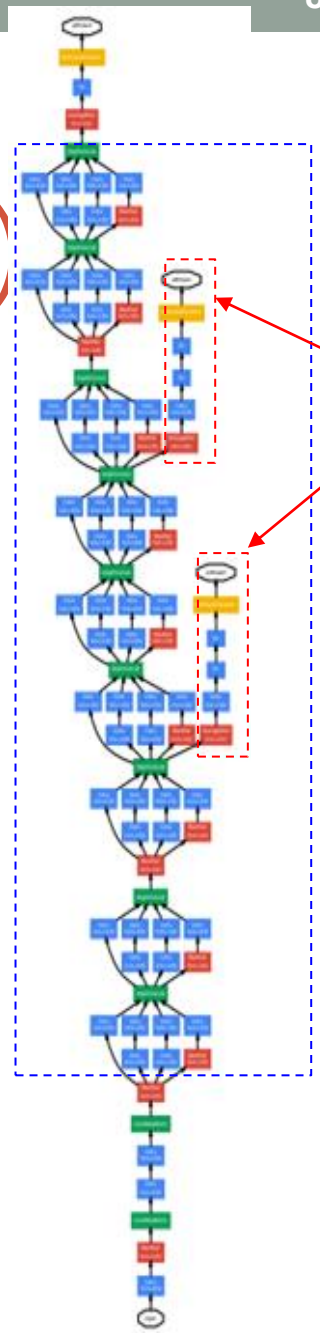
Dimensionality reduction via 1x1 convolution

Multiple softmax losses to help the gradient problem

Dimensionality reduction



1 inception block

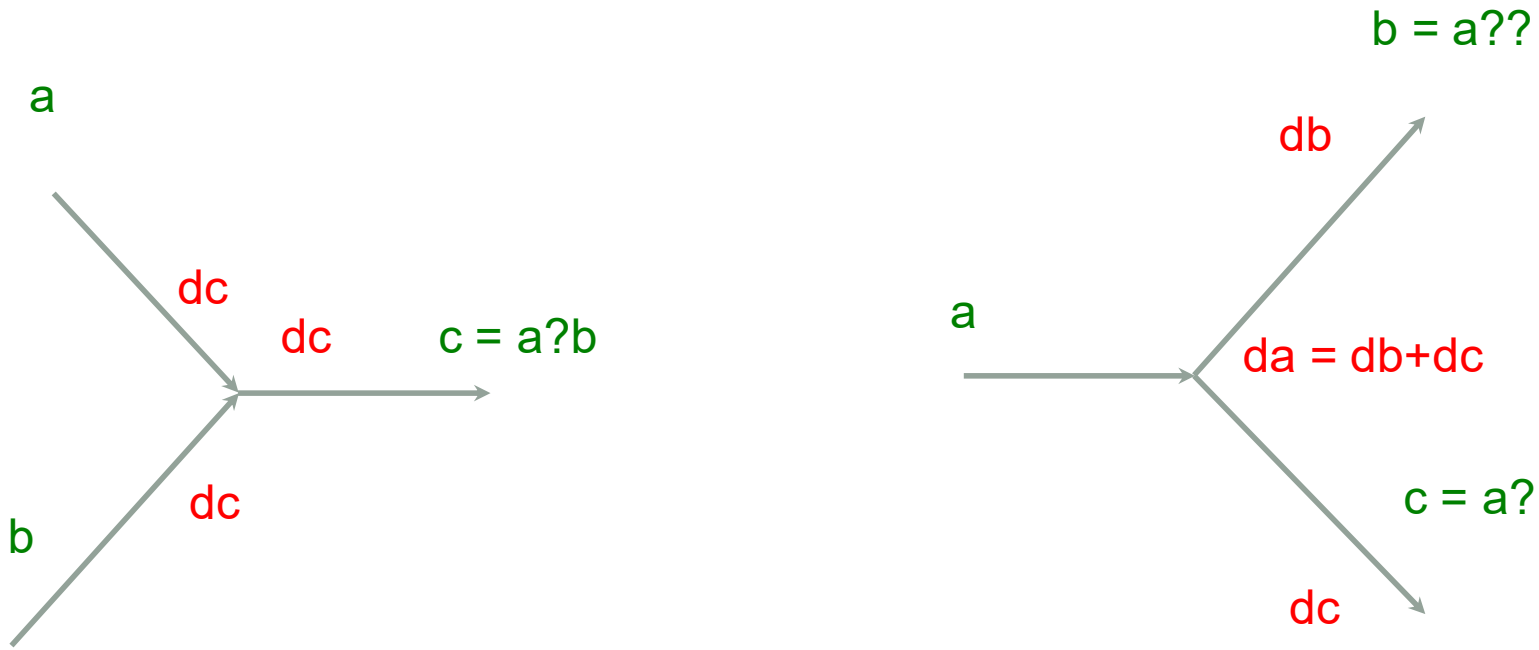


Extra softmax losses

Inception blocks

Christian Szegedy, et al. “Going Deeper with Convolutions,” 2014

Gradient flow at forks

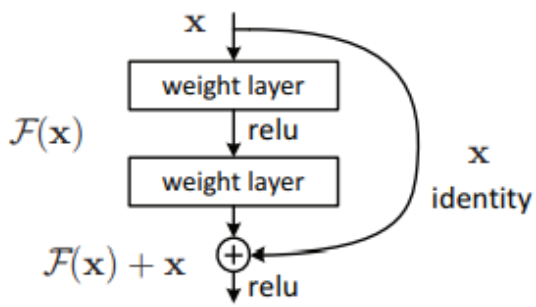


Forward and backward pass acts differently at forks

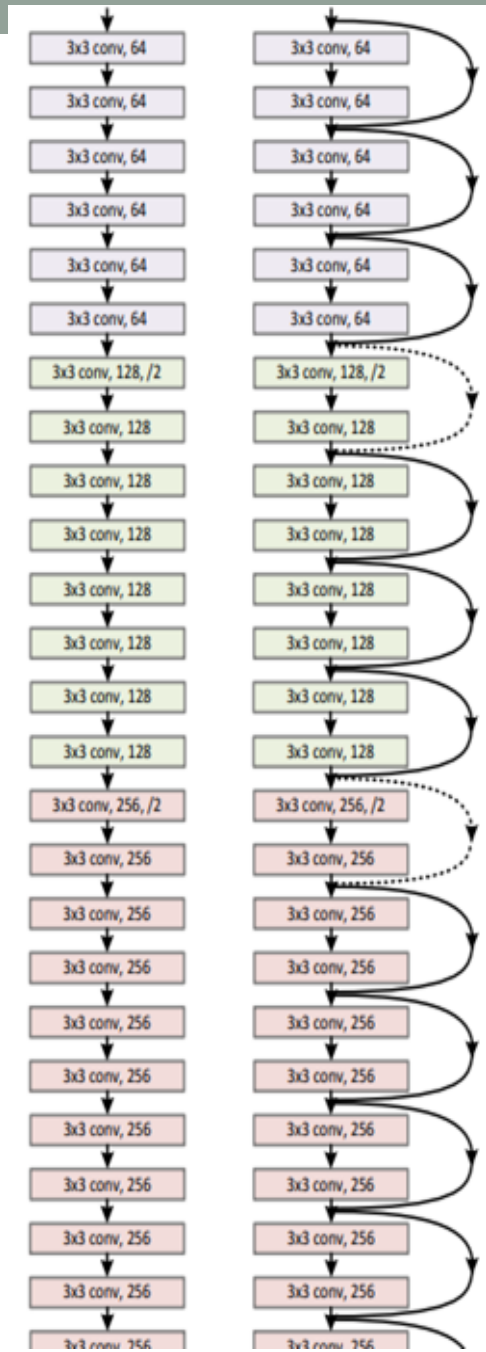
ResNet (Residual Network)

Batch norm

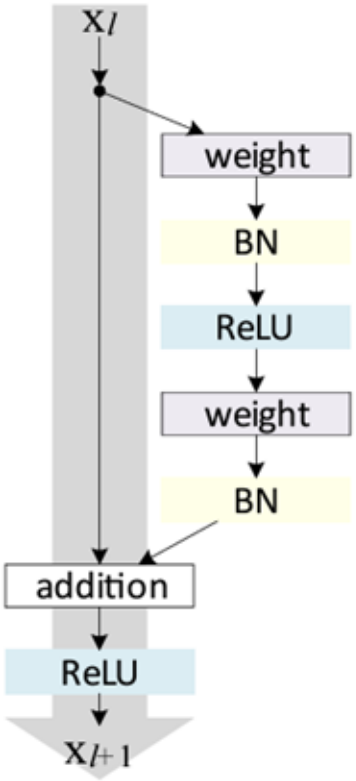
Extra “skip connections” to reduce the vanishing gradient problem



$$y = F(x) + x \rightarrow F(x) = y - x$$



Need extra operations to make the size the same



(a) original

Kaiming He, et al. “Deep Residual Learning for Image Recognition” 2015

Batch normalization

- Recent technique for (implicit) regularization
- Normalize every mini-batch at various batch norm layers to standard Gaussian (different from global normalization of the inputs)
- Place batch norm layers before (or after?) non-linearities
- Faster training (put numbers in expected range – can use larger learning rates) and better generalizations

For each mini-batch that goes through batch norm

1. Normalize by the mean and variance of the mini-batch for each dimension
2. Shift and scale by learnable parameters

$$\hat{x} = \frac{x - \mu_b}{\sigma_b}$$

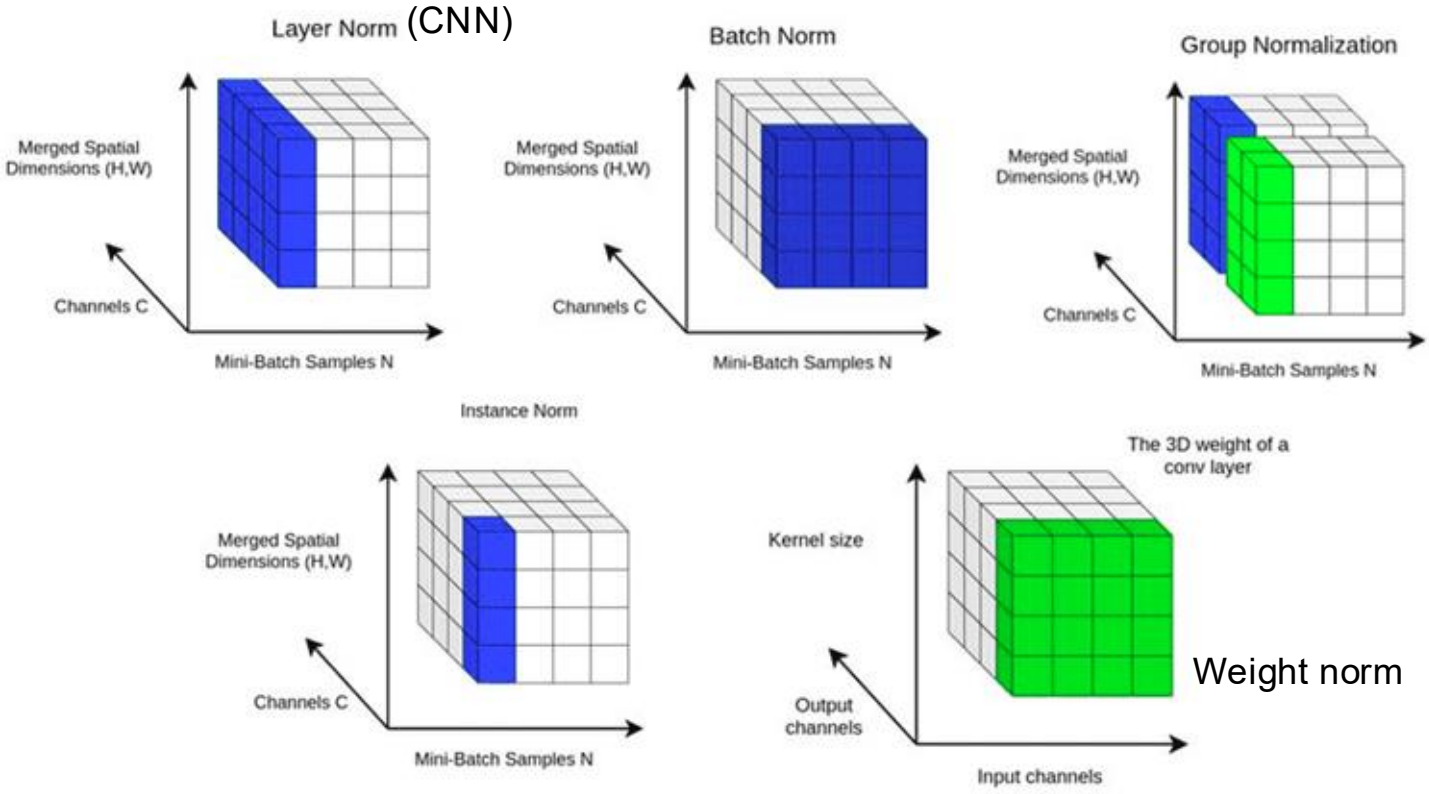
$$y = \alpha \hat{x} + \beta$$

Replaces dropout in some networks
https://arxiv.org/abs/1502.03167

Mean changes every minibatch – a form of noise (regularization)

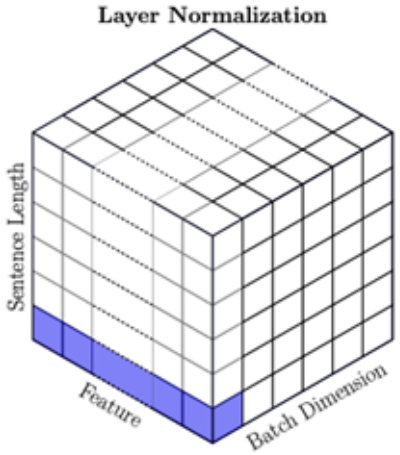
Other normalizations

- Other normalizations are out there



Weight norm

Layer norm in transformers

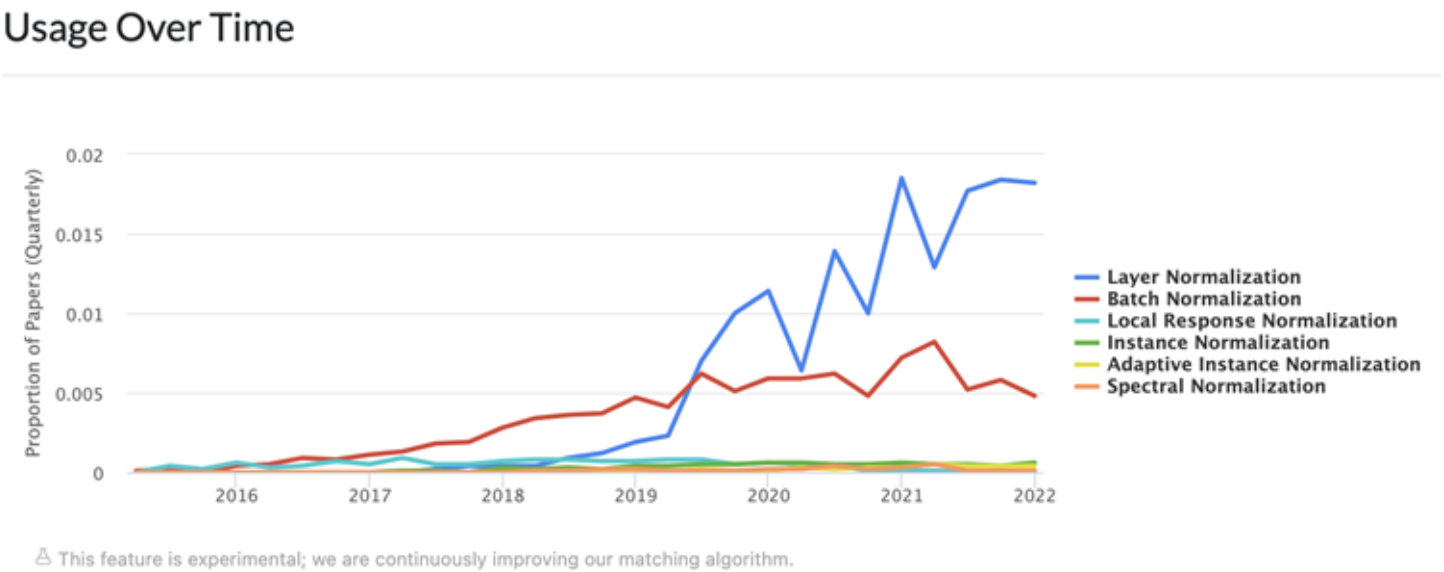


https://proceedings.mlr.press/v119/shen20e/shen20e.pdf

https://theaisummer.com/normalization/

Other normalizations

Trend of normalization used in papers



NLP – layer norm
Image – Batch norm/layer norm
Small batches - Group norm (batch norm is not reliable in small batches)

https://paperswithcode.com/method/layer-normalization

Training and inference of batch norm

BATCHNORM2D

CLASS `torch.nn.BatchNorm2d(num_features, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True, device=None, dtype=None)` [SOURCE]

Applies Batch Normalization over a 4D input (a mini-batch of 2D inputs with additional channel dimension) as described in the paper Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift .

$$y = \frac{x - \mathrm{E}[x]}{\sqrt{\mathrm{Var}[x] + \epsilon}} * \gamma + \beta$$

The mean and standard-deviation are calculated per-dimension over the mini-batches and γ and β are learnable parameter vectors of size C (where C is the input size). By default, the elements of γ are set to 1 and the elements of β are set to 0. The standard-deviation is calculated via the biased estimator, equivalent to *torch.var(input, unbiased=False)*.

Also by default, during training this layer keeps running estimates of its computed mean and variance, which are then used for normalization during evaluation. The running estimates are kept with a default `momentum` of 0.1.

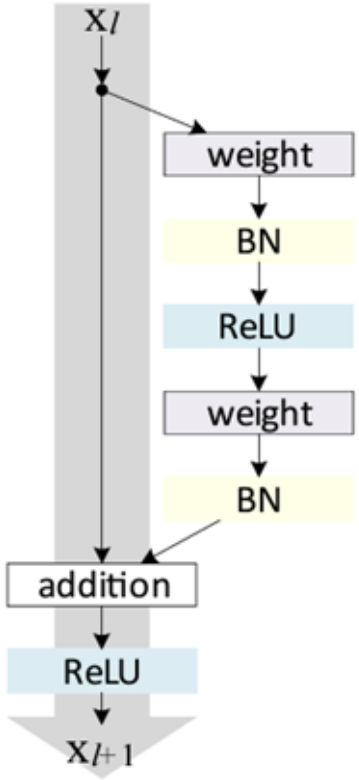
If `track_running_stats` is set to `False`, this layer then does not keep running estimates, and batch statistics are instead used during evaluation time as well.

https://pytorch.org/docs/stable/generated/torch.nn.BatchNorm2d.html

Standardized Architecture

CNN architectures are more standardized after ResNet

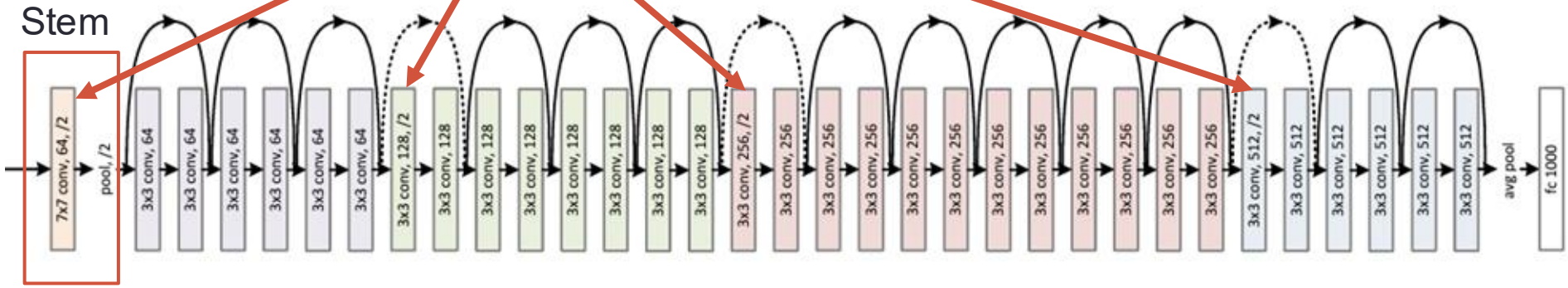
- Big stem to quickly downsample
- small filter (3x3 or 1x1)
- Global average pooling at the end
- Conv-BN-Relu block



(a) original

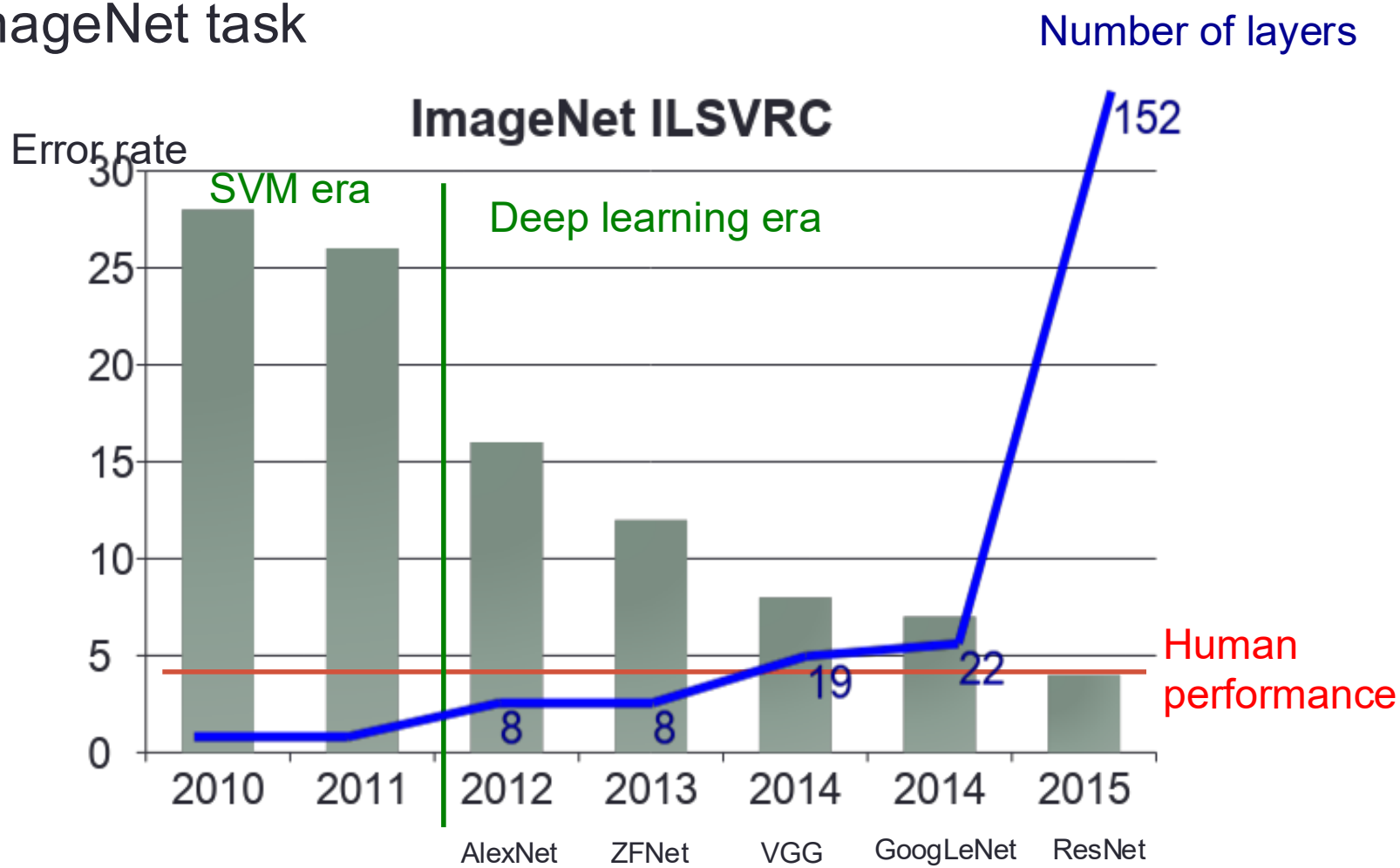
Convolution with strides

Stem



A brief history of imagenet architectures

- ImageNet task

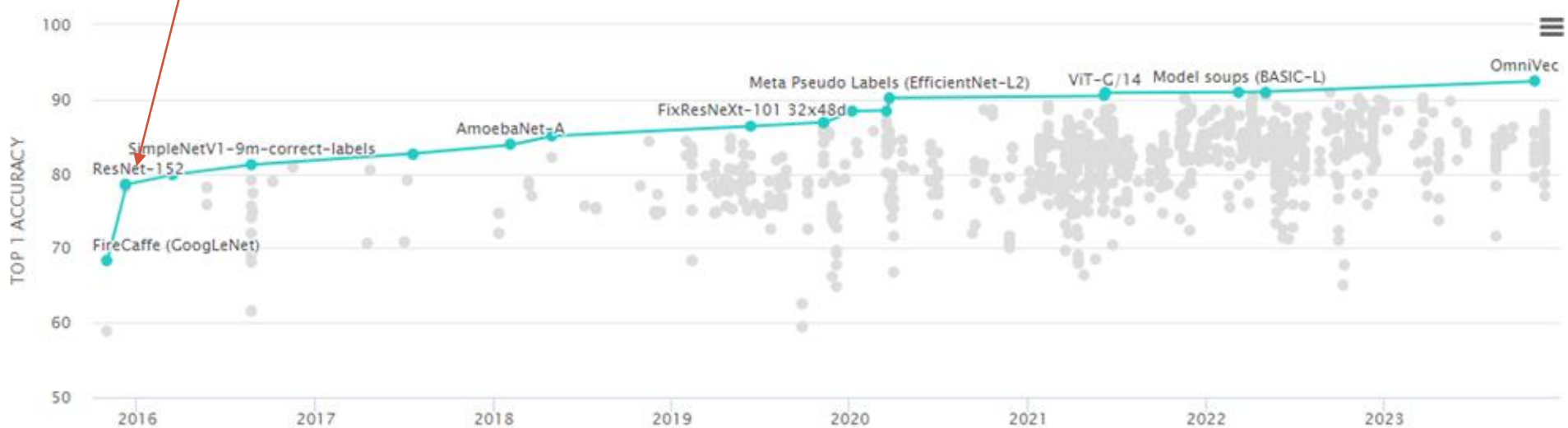


Olga Russakovsky, ImageNet Large Scale Visual Recognition Challenge, 2014 https://arxiv.org/abs/1409.0575

Trends after ResNet

- More efficient convolutional block design
- Wider networks, deeper models, denser connections
- Global Context + Transformer architecture
- Better representation learning (self-supervision + multimodality)

2016



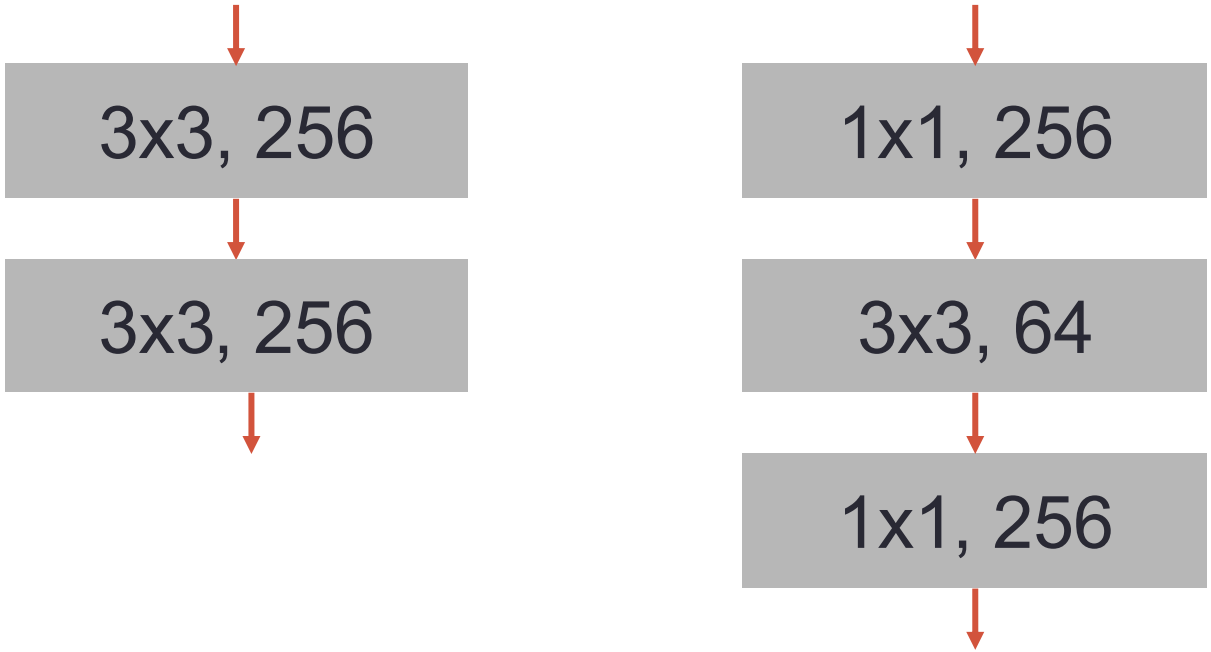
Convolutional block design

- 1x1 convolution
- factorized convolution
- group convolution

More efficient convolution block

Consider Input tensor of size (W x H x 64)

Q: which block has more parameters?



More efficient convolution block

Parameters : 3x3x64x256 +
3x3x256x256
=737,280

3x3, 256

3x3, 256

Parameters : 1x1x64x256 +
3x3x256x64 +
1x1x64x256
= 180,224

1x1, 256

3x3, 64

1x1, 256

The one on the right use 4 times less parameters

Better Tradeoff!

Bottleneck block

Factorized convolution

- k x k convolution vs 1 x k then k x1 convolution
- Not efficient on small filter

7x7, 256

Parameters : 7x7x64x256
=802,816

1x7, 256

7x1, 256

Parameters : 1x7x64x256 +
7x1x256x256
= 573,440

Consider Input tensor of size (W x H x 64)

Inception v2+v3

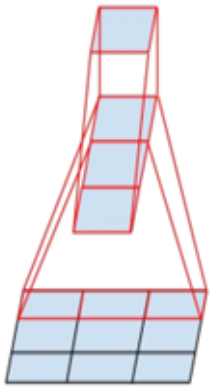
Implement 5x5 with two 3x3s

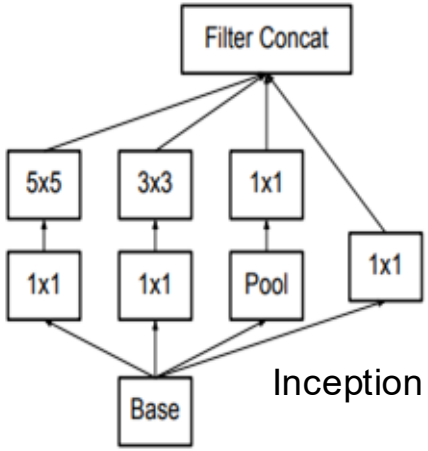
Factorized convolution

3x3 -> 3x1 and 1x3

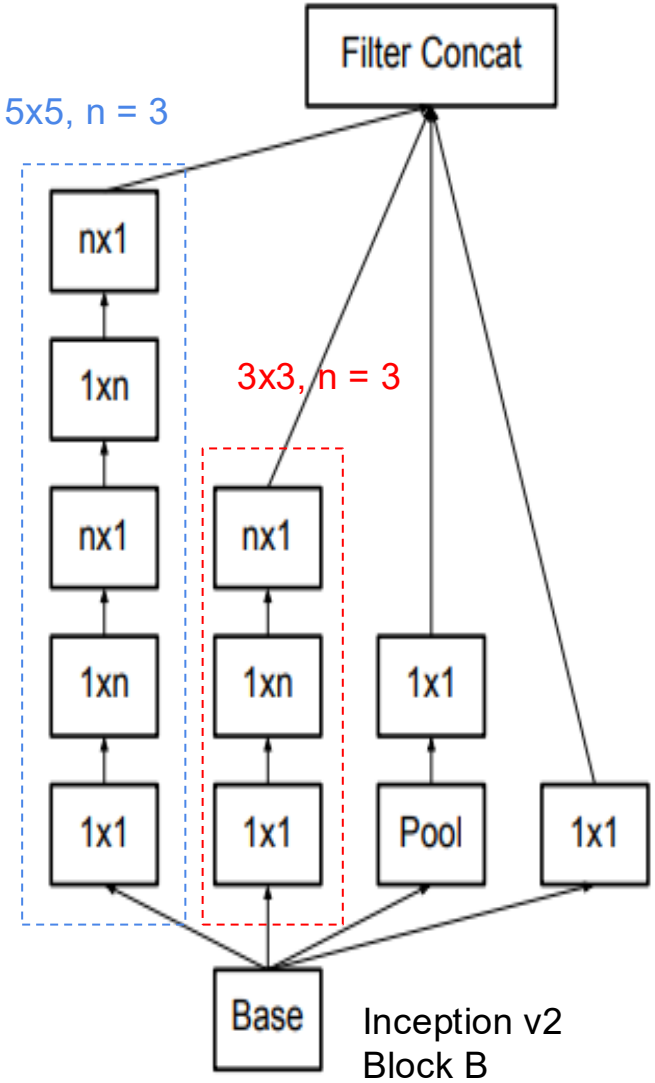
3 types of inception blocks

RMSprop, Batch norm, label smoothing





Inception v1



Inception v2
Block B

Christian Szegedy, “Rethinking the Inception Architecture for Computer Vision,” 2015

Group convolutions

Magic!

	setting	top-1 error (%)
ResNet-50	1 × 64d	23.9
ResNeXt-50	2 × 40d	23.0
ResNeXt-50	4 × 24d	22.6
ResNeXt-50	8 × 14d	22.3
ResNeXt-50	32 × 4d	**22.2**
ResNet-101	1 × 64d	22.0
ResNeXt-101	2 × 40d	21.7
ResNeXt-101	4 × 24d	21.4
ResNeXt-101	8 × 14d	21.3
ResNeXt-101	32 × 4d	**21.2**

- ResNext reintroduce concept of group convolutions
- split-transform-merge strategy
- Original : 3x3 filter with d = 256
- Modified : 64 groups of 3x3 filters with d = 4

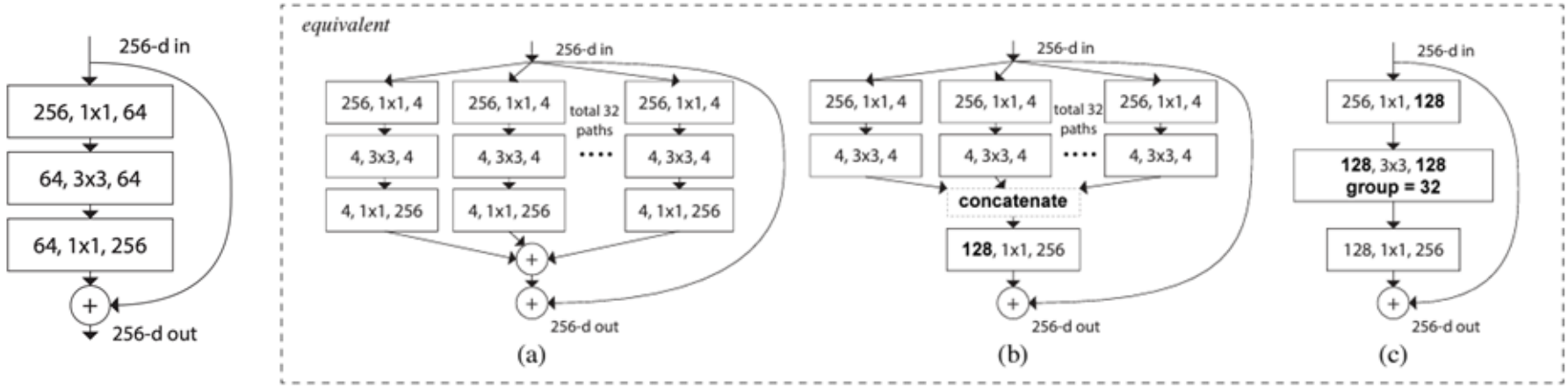


Figure 1. **Left**: A block of ResNet [14]. **Right**: A block of ResNeXt with cardinality = 32, with roughly the same complexity. A layer is shown as (# in channels, filter size, # out channels).

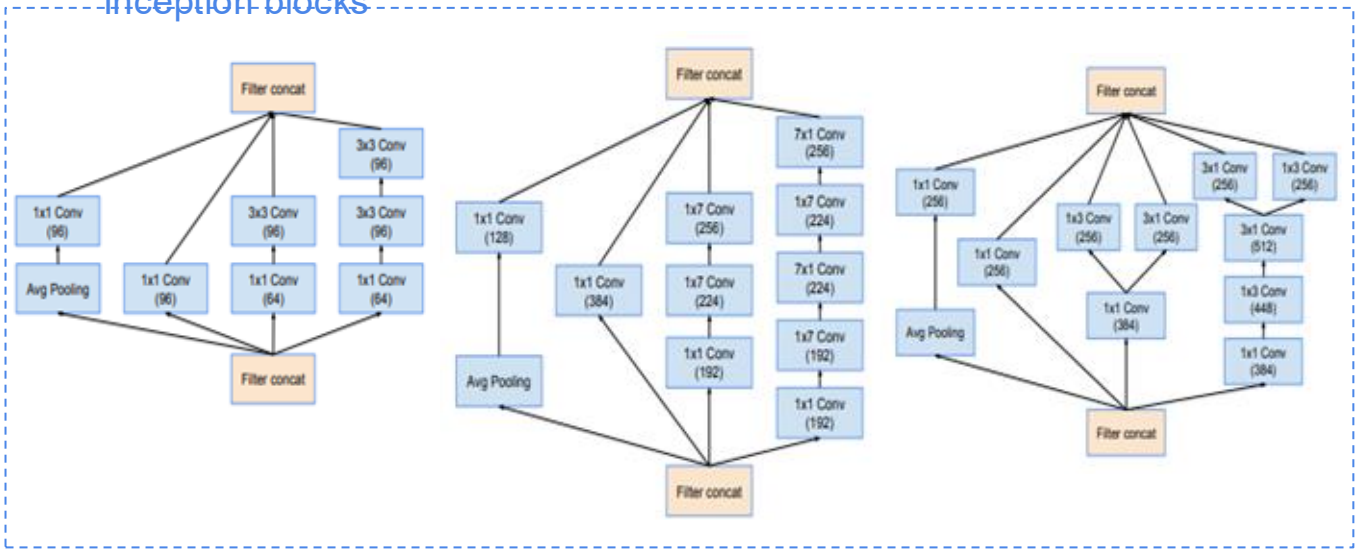
Aggregated Residual Transformations for Deep Neural Networks (2016)

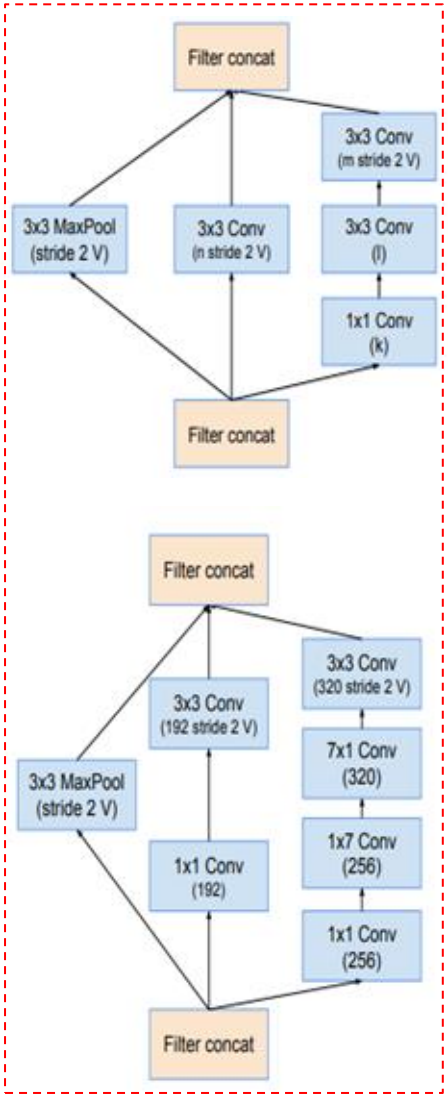
Inception v4

Same three types of inception blocks from v3

Add two types of reduction blocks for reducing the size of the grid (super pooling blocks)

Inception blocks





Reduction blocks

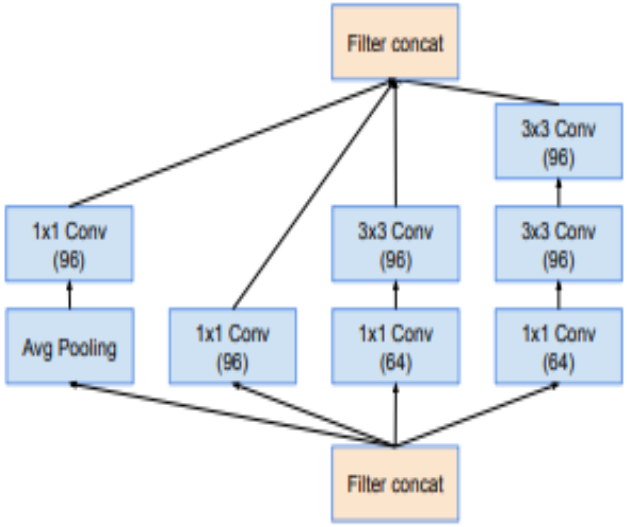
Christian Szegedy, “Inception-v4, Inception-ResNet and the Impact of Residual Connections on Learning” 2016.

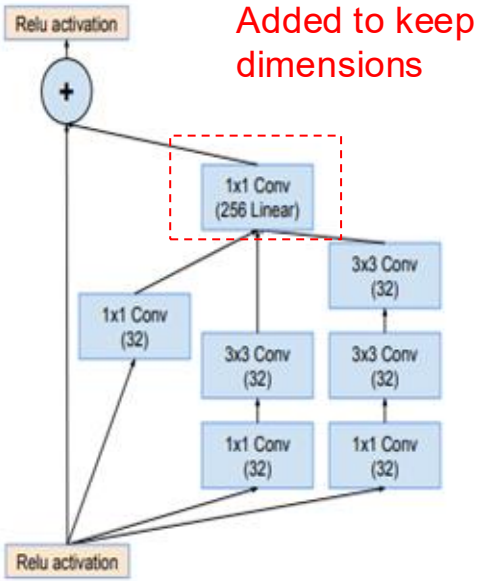
Inception-ResNet v1-v2

Introduce residual connections into inception blocks

Poolings changed to additions, 1x1 added to keep dimensions for the residual

Similar idea to ResNeXt



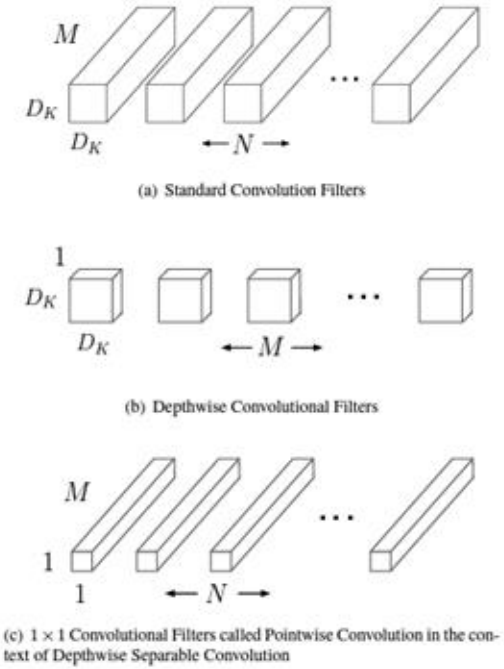


Christian Szegedy, “Inception-v4, Inception-ResNet and the Impact of Residual Connections on Learning” 2016.

Depthwise separable convolution

Depthwise separable convolution: two-step convolution

1. Depthwise convolution
   - Could be seen as an extreme version of group convolution (number of groups = number of channels)
2. 1x1 convolution

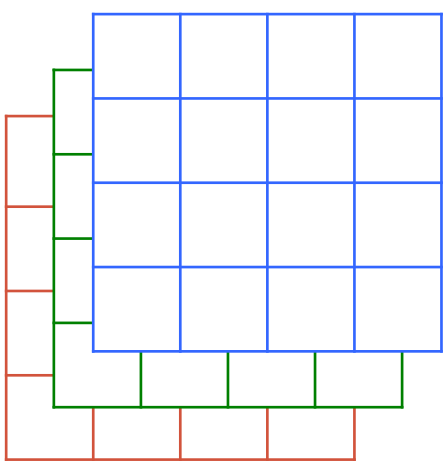


(a) Standard Convolution Filters

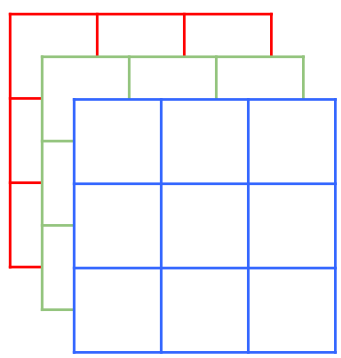
(b) Depthwise Convolutional Filters

(c) 1 × 1 Convolutional Filters called Pointwise Convolution in the context of Depthwise Separable Convolution

input



One regular convolution filter



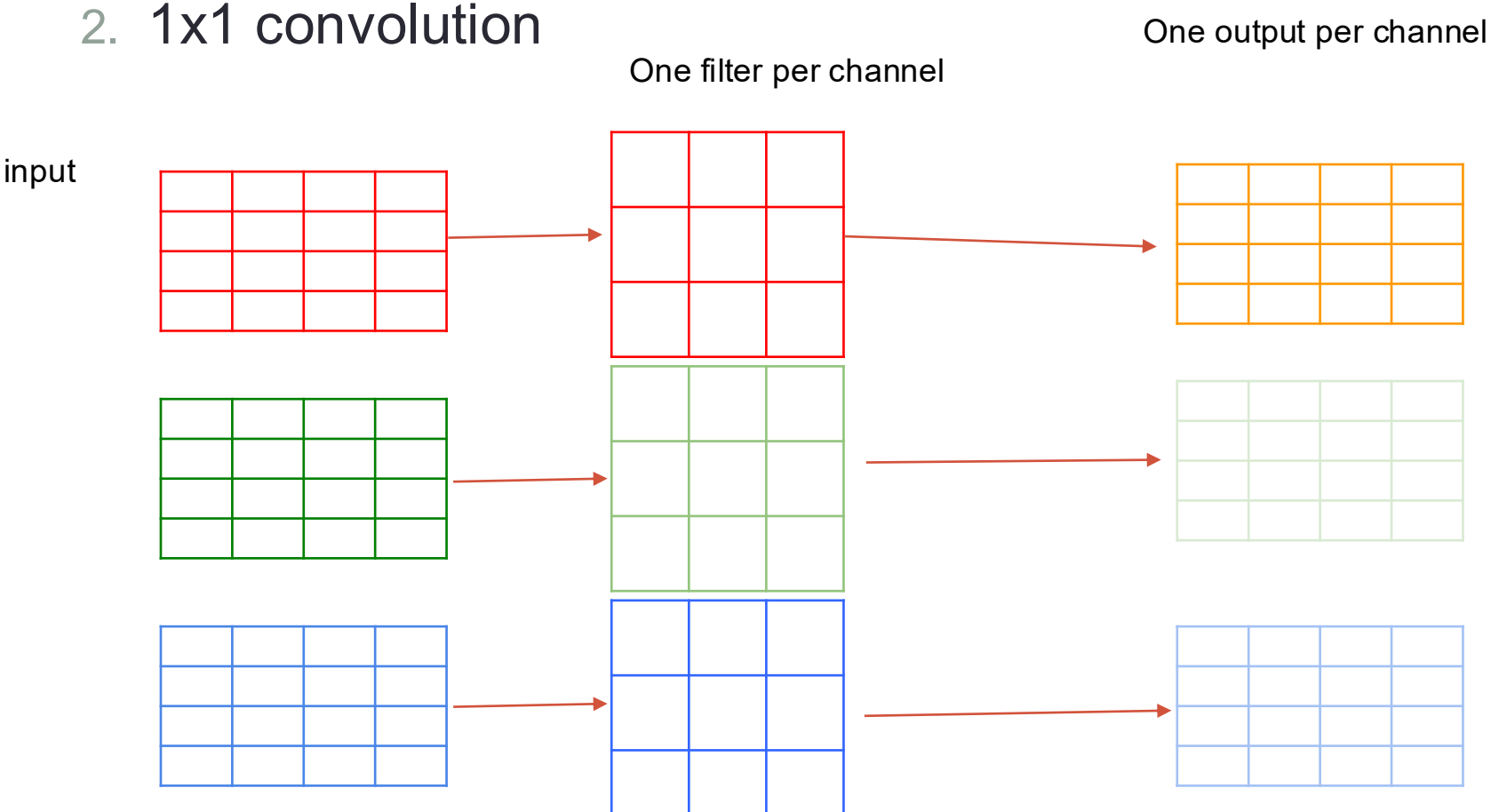
Typical convolution 3x3 filter is 3x3xinput channel

Depthwise convolution 3x3 filter is 3x3x1
1 filter per input channel

Depthwise separable convolution

Depthwise separable convolution: two-step convolution

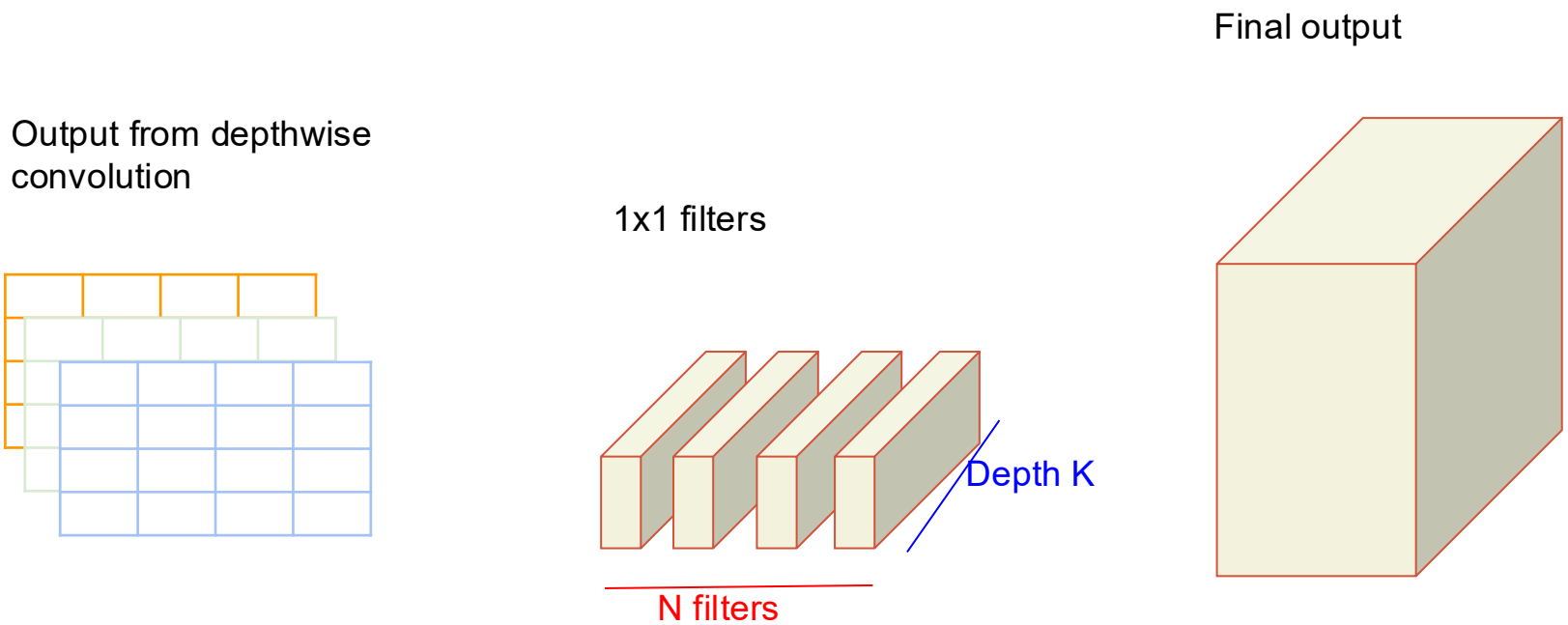
1. Depthwise convolution (learn from each channel, spatial info)
2. 1x1 convolution



Depthwise separable convolution

Depthwise separable convolution: two-step convolution

1. Depthwise convolution
2. 1x1 convolution (combine each channel, feature info)



10 3x3 depthwise separable convoltuion will have 3x3x3 + 3x10 = 57 parameters

More efficient convolution block (2)

Parameters : 3x3x64x256 + 3x3x256x256 =737,280

3x3, 256

3x3, 256

Consider Input tensor of size (W x H x 64)

Parameters : 3x3x64 + 1x1x64x256 + 3x3x256 + 1x1x256x256 = 84,800

depthwise 3x3 (64)

1x1, 256

depthwise 3x3 (256)

1x1, 256

The one on the right use 9 times less parameters!

Even Better Tradeoff!

Xception

Replace convolutions in inception with depthwise separable convolutions

Smaller model

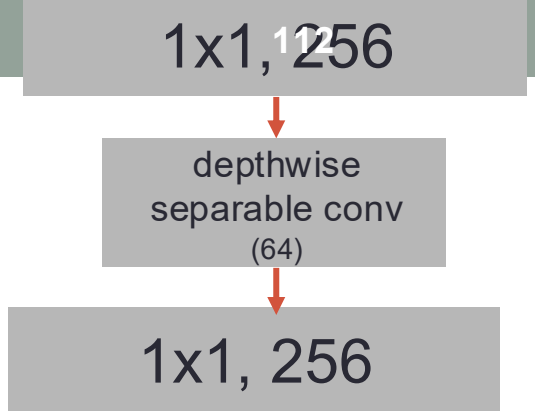
Faster compute

Comparable with Inception v3 while much faster

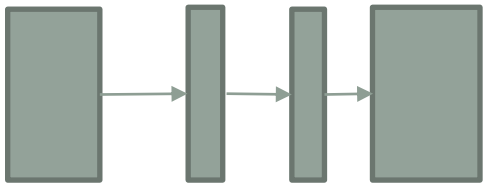
Used in MobileNet and other models

François Chollet, “Xception: Deep Learning with Depthwise Separable Convolutions” 2016.
Andrew G. Howard, et al., “MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications” 2017.

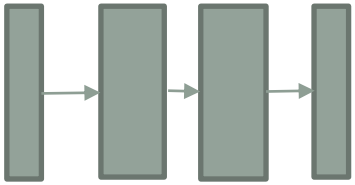
Inverted bottleneck



- A bottleneck block
  - reduce channels using 1x1
  - Regular conv on fewer channels
  - Increase channel using 1x1



- Inverted bottleneck
  - Expand bottleneck instead of shrink)
  - Increase channels using 1x1
  - Regular conv on more channels
  - Reduce channels using 1x1



- Bottleneck combines information between channels
- With residual connections, inverted bottleneck uses less runtime memory during inference (DAG computation graph)

https://arxiv.org/abs/1801.04381

Mobile inverted Bottleneck

- Use depthwise convo instead of conv (MobileNetv2)
- Add Squeeze and Excitation (SE) (EfficientNetv1)
- Can fused 1x1 with depthwise (EfficientNetv2)

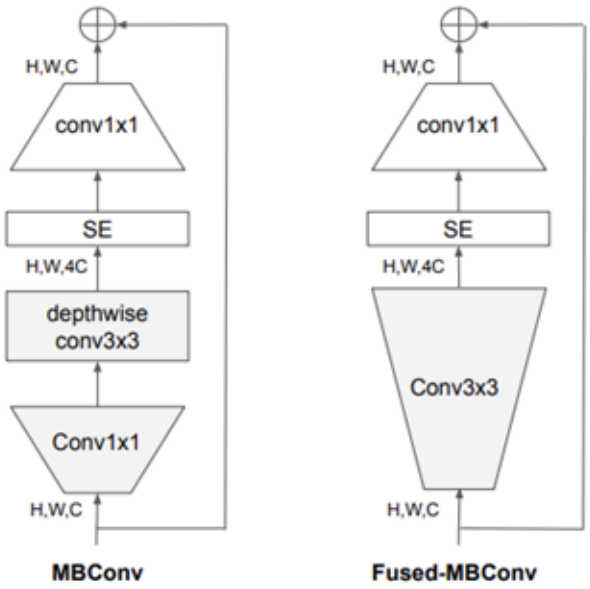
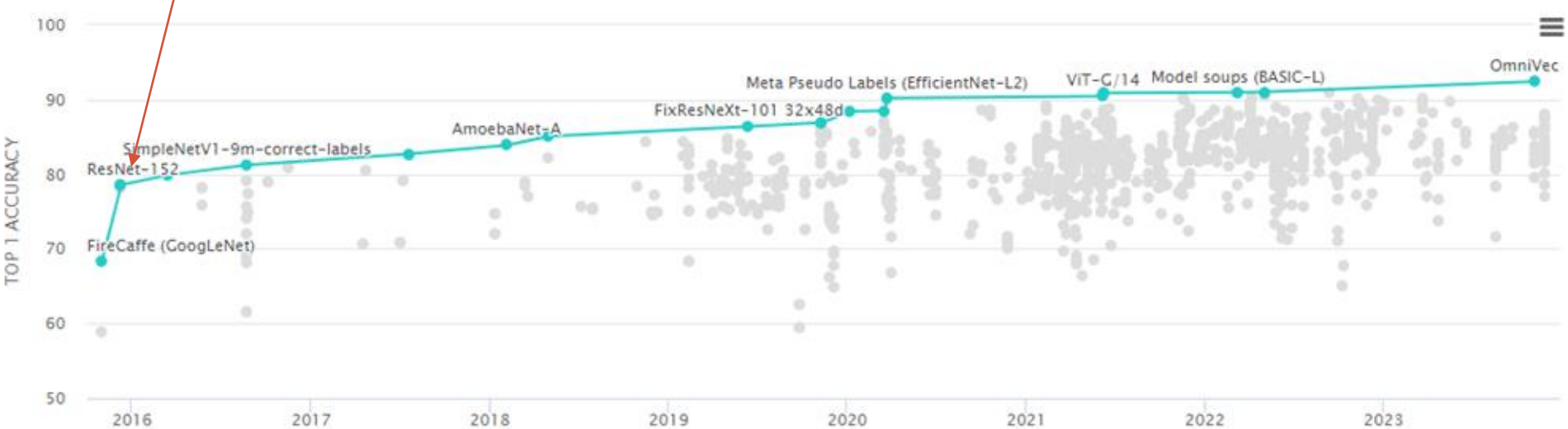


Figure 2. Structure of MBConv and Fused-MBConv.

Trends after ResNet

- More efficient convolutional block design
- **Wider networks, deeper models, denser connections**
- Global Context + Transformer architecture
- Better representation learning (self-supervision + multimodality)

2016



WideResNet

- Find which N, k works the best
- The too-deep but wide network outperforms the deeper networks

group name	output size	block type = $B(3,3)$
conv1	32×32	$[3\times3,\ 16]$
conv2	32×32	$\begin{bmatrix} 3\times3,\ 16\times k \\ 3\times3,\ 16\times k \end{bmatrix} \times N$
conv3	16×16	$\begin{bmatrix} 3\times3,\ 32\times k \\ 3\times3,\ 32\times k \end{bmatrix} \times N$
conv4	8×8	$\begin{bmatrix} 3\times3,\ 64\times k \\ 3\times3,\ 64\times k \end{bmatrix} \times N$
avg-pool	1×1	$[8 \times 8]$

Abstract

Deep residual networks were shown to be able to scale up to thousands of layers and still have improving performance. However, each fraction of a percent of improved accuracy costs nearly doubling the number of layers, and so training very deep residual networks has a problem of diminishing feature reuse, which makes these networks very slow to train. To tackle these problems, in this paper we conduct a detailed experimental study on the architecture of ResNet blocks, based on which we propose a novel architecture where we decrease depth and increase width of residual networks. We call the resulting network structures wide residual networks (WRNs) and show that these are far superior over their commonly used thin and very deep counterparts. For example, we demonstrate that even a simple 16-layer-deep wide residual network outperforms in accuracy and efficiency all previous deep residual networks, including thousand-layer-deep networks, achieving new state-of-the-art results on CIFAR, SVHN, COCO, and significant improvements on ImageNet. Our code and models are available at https://github.com/szagoruyko/wide-residual-networks.

Wide Residual Networks (2016)

ResNetv2

Q : What happens if we goes deeper than 200 layers?

A : The performance gets better with some modifications though the return is diminished.

dataset	network	[redacted]	pre-activation unit
CIFAR-10	ResNet-110 (1layer skip)		8.91
	ResNet-110		6.37
	ResNet-164		5.46
	ResNet-1001		4.92
CIFAR-100	ResNet-164		24.33
	ResNet-1001		22.71

Kaiming He, et al., Identity Mappings in Deep Residual Networks (2016)

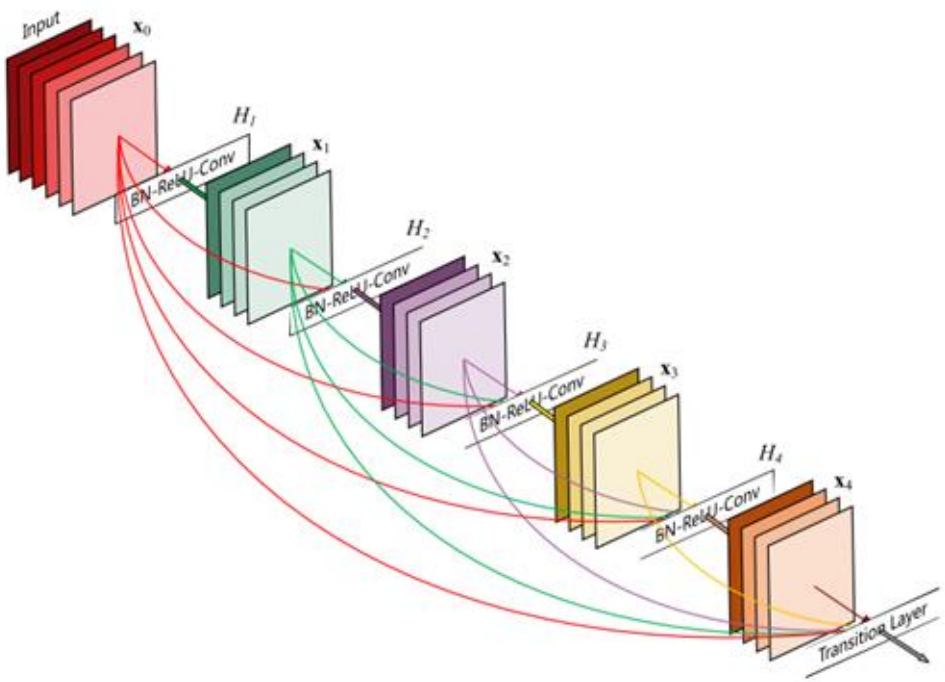
DenseNet

Instead of adding the residuals, concatenates the feature maps from previous layers

Densely connect multiple layers

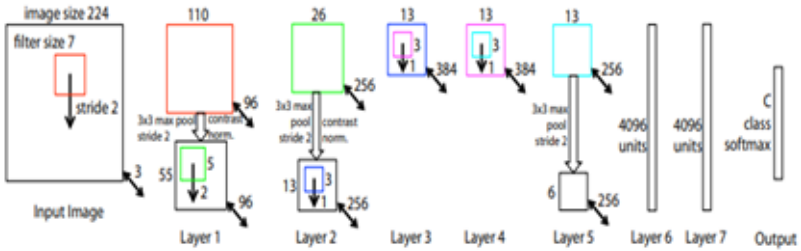
Multi-scale feature maps

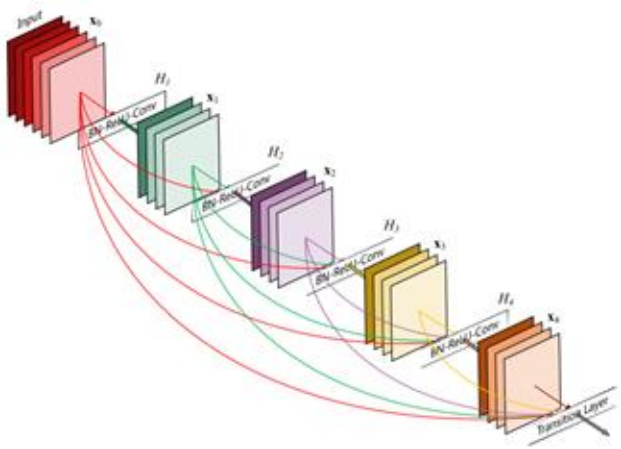
Smaller model due to smaller number of channels

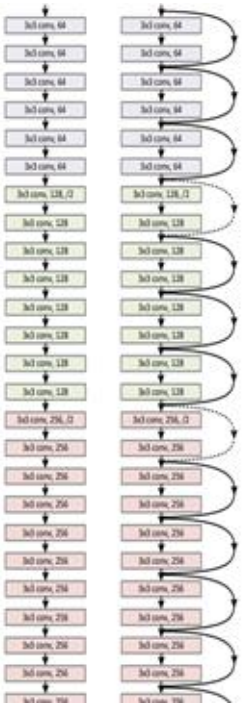


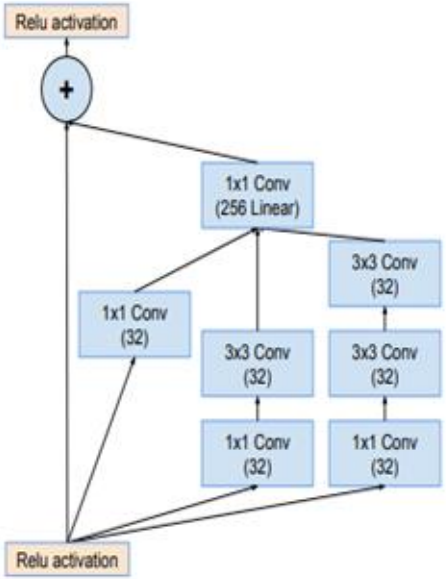
Densely Connected Convolutional Networks 2016

Do people keep tweaking network architectures until the end of time?







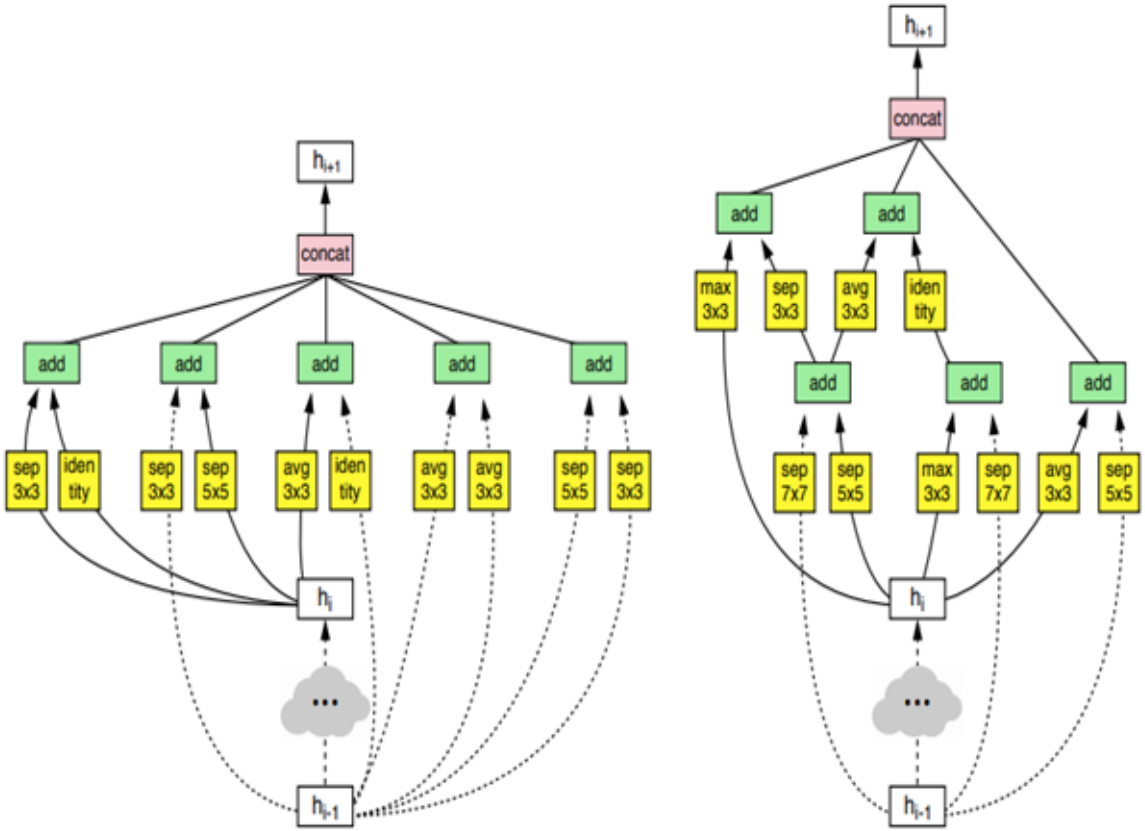


NasNet

Neural Architecture Search Network

Use reinforcement learning to search for the best network configuration

Lowers compute (FLOPs) while maintaining high accuracy



Inception block

Reduction block

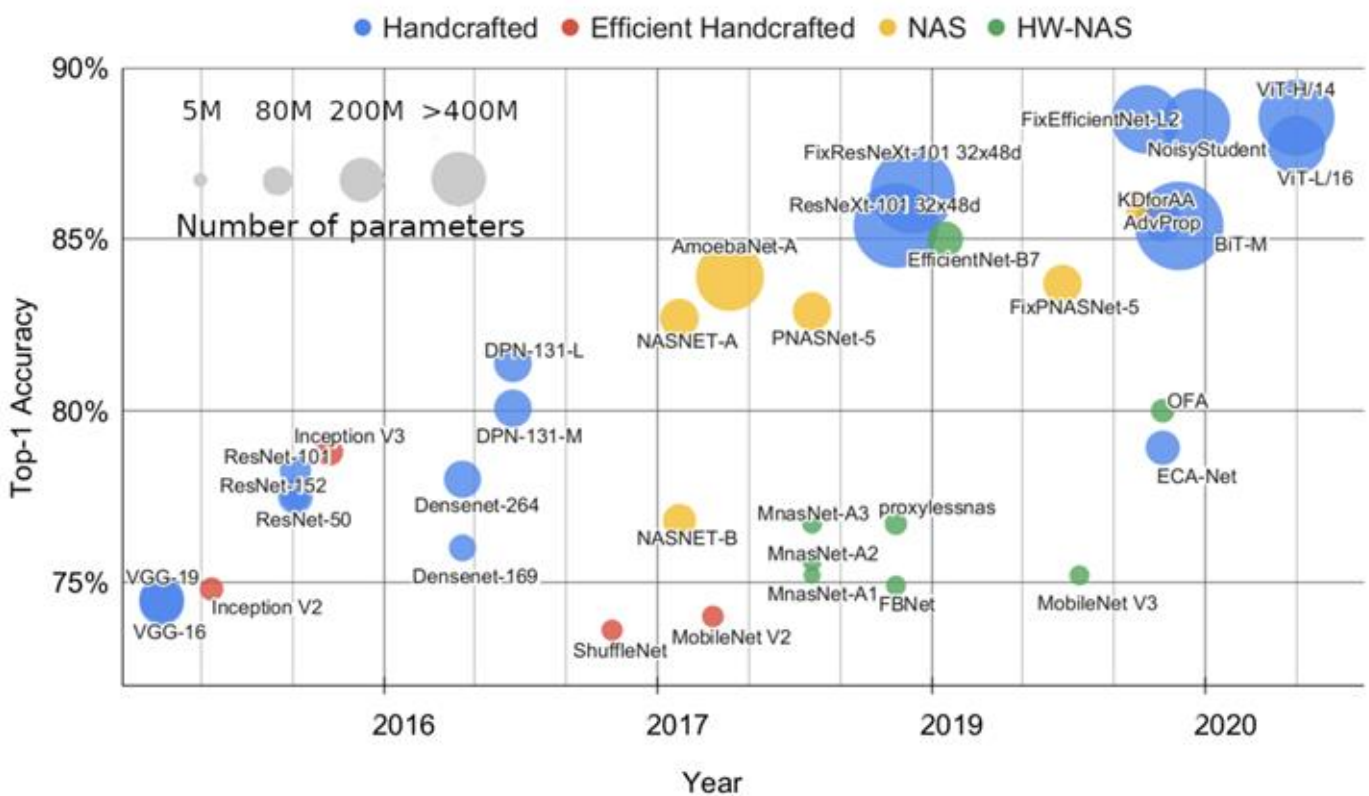
Barret Zoph, et al. “Learning Transferable Architectures for Scalable Image Recognition,” 2017.

NAS doesn't know com sys arch

Performance Measurement

- Response Time — *Elapse Time = CPU Time + I/O Time + Memory Time*
  - Elapsed Time - Wall clock
  - CPU Time - Only CPU time (without the wait time) ... we will revisit this later,
  - From O.S. class, we use: user time, real time, sys time.
- Throughput
  - Tasks per time unit
- Units for commercialization (Not a real performance)
  - MIPS
    - Millions of instructions per second
  - FLOPS
    - Floating Point Operations per second

- FLOPs is not a good measure of performance



https://arxiv.org/abs/2101.09336

EfficientNetv1, v2

- Architecture/hardware aware NAS + handcrafted building blocks (from mobilenetV2)
- Tradeoff between width, depth, resolution

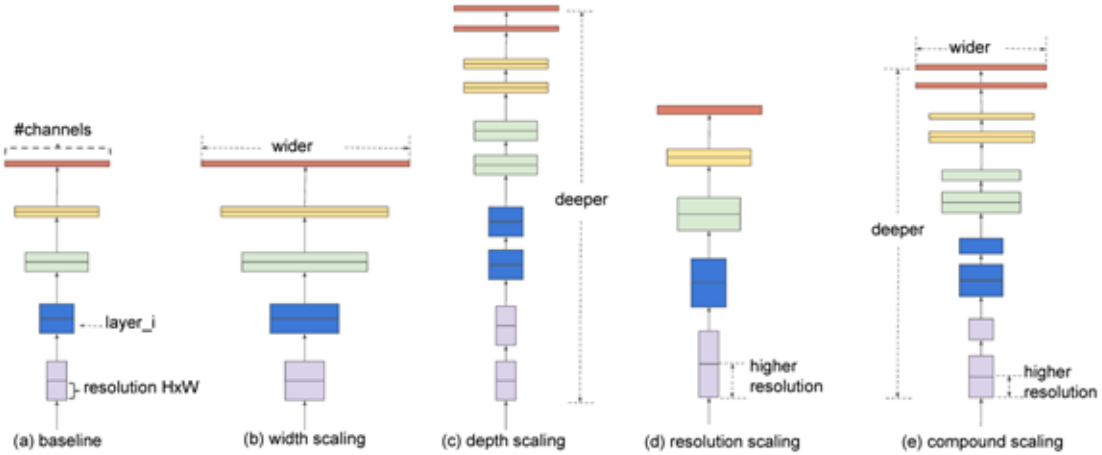


Figure 2. **Model Scaling.** (a) is a baseline network example; (b)-(d) are conventional scaling that only increases one dimension of network width, depth, or resolution. (e) is our proposed compound scaling method that uniformly scales all three dimensions with a fixed ratio.

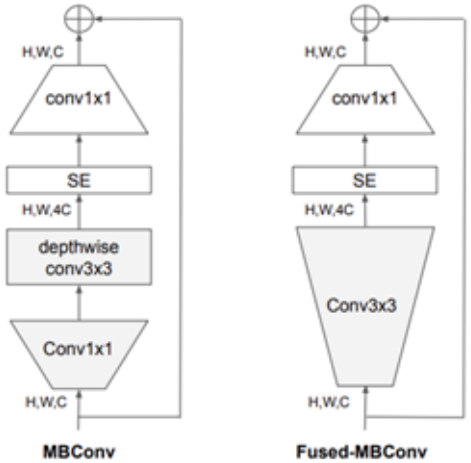
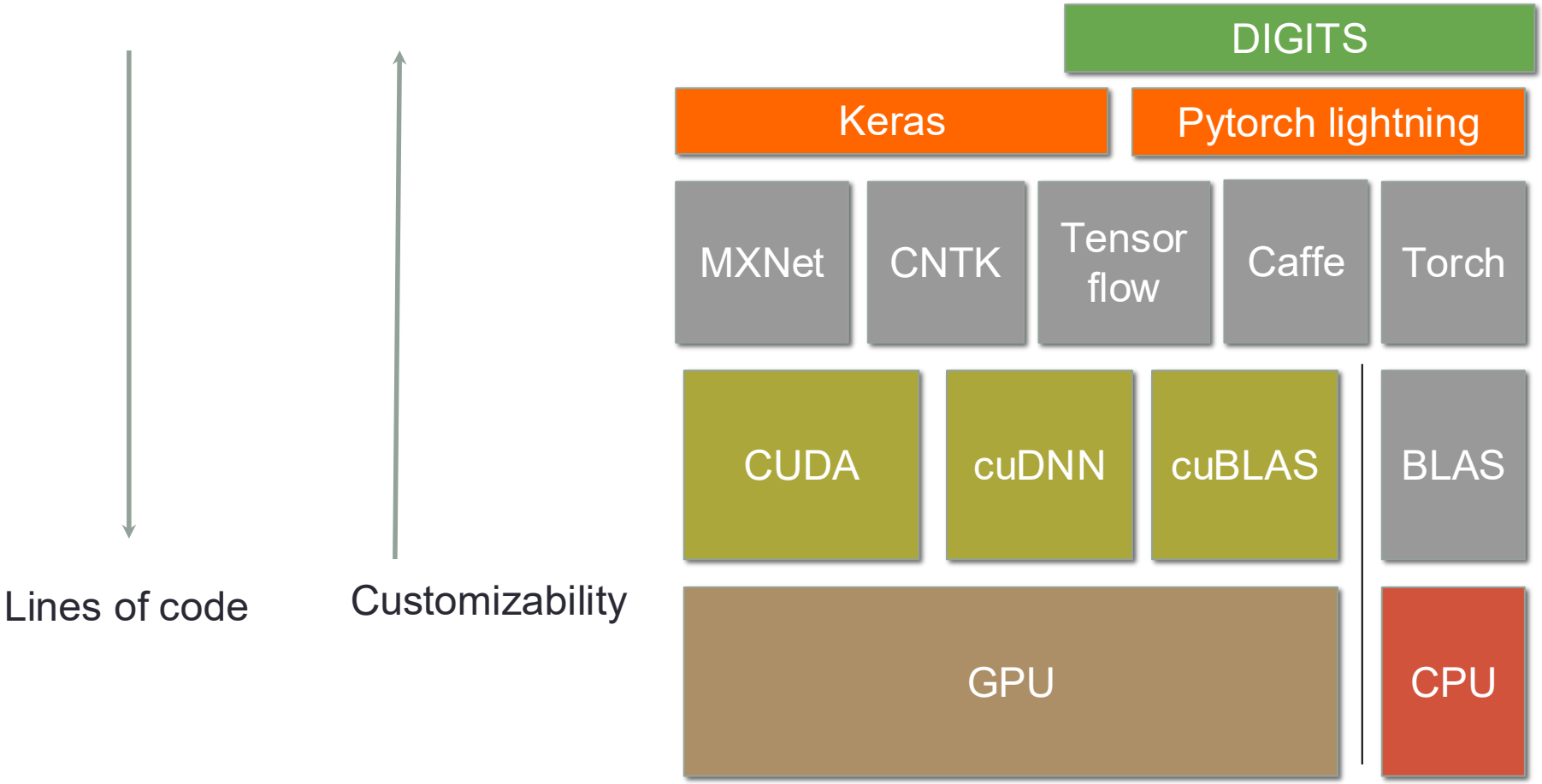


Figure 2. Structure of MBConv and Fused-MBConv.

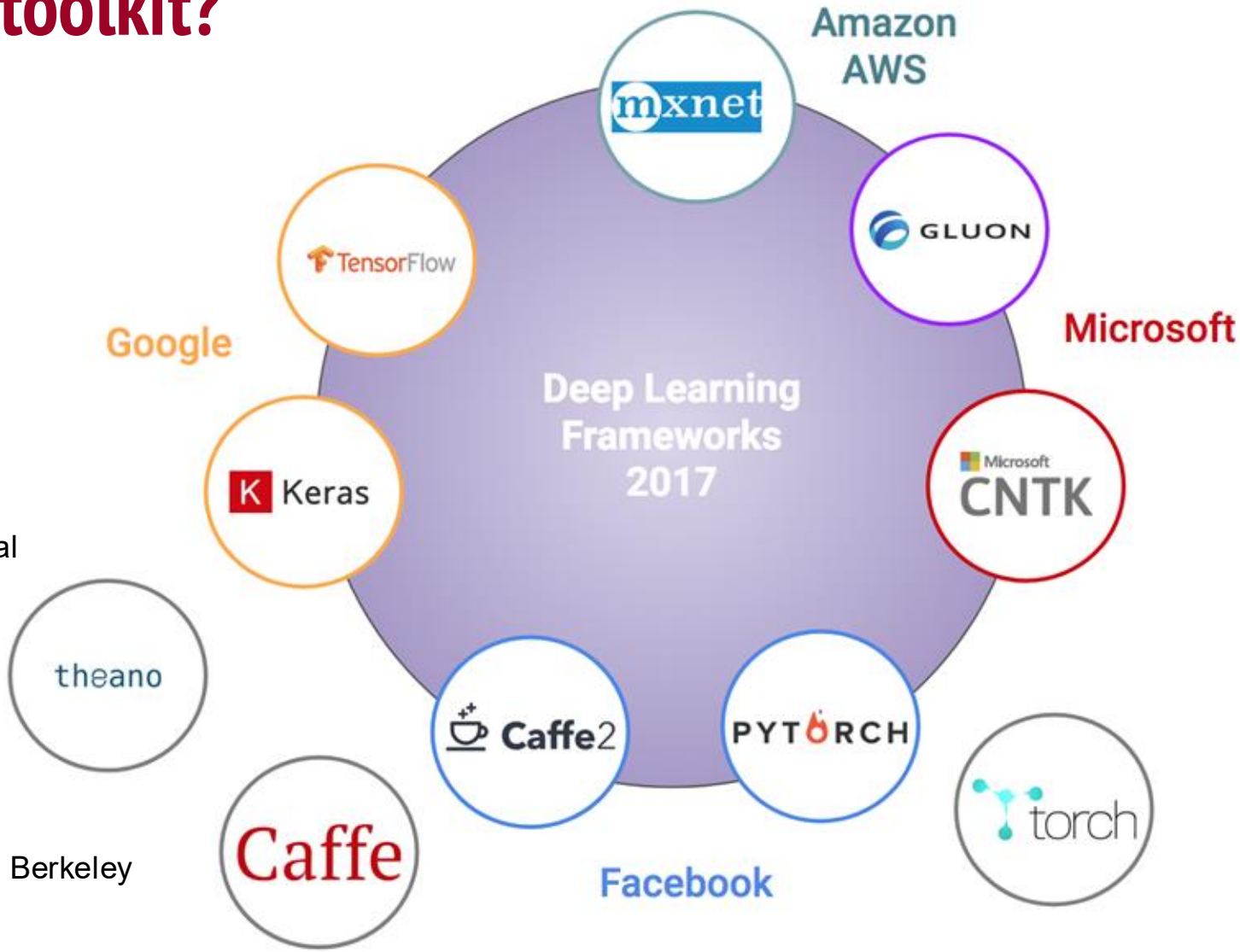
https://arxiv.org/abs/1905.11946
https://arxiv.org/abs/2104.00298

What toolkit

Tradeoff between customizability and ease of use



What toolkit?

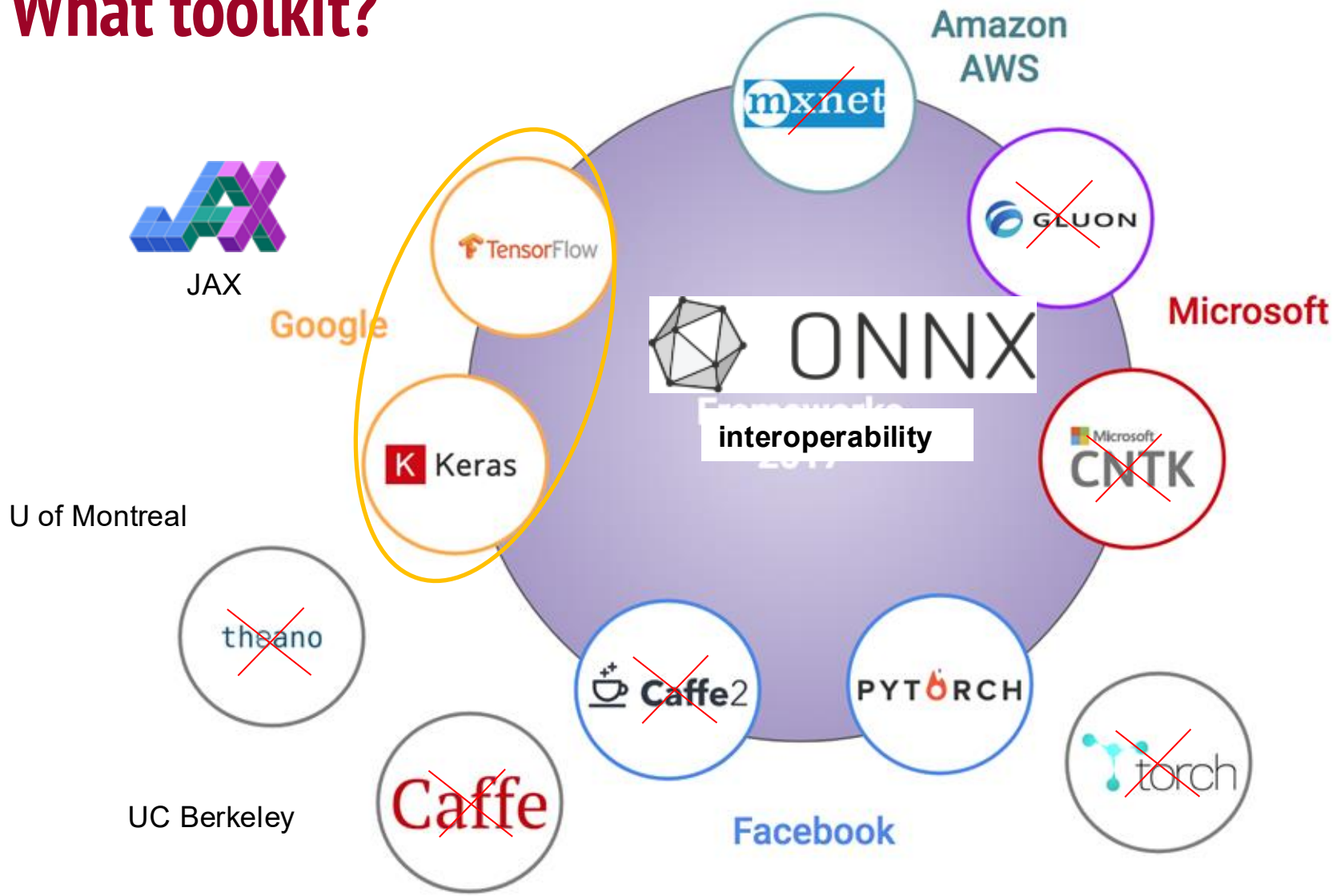


U of Montreal

UC Berkeley

https://towardsdatascience.com/battle-of-the-deep-learning-frameworks-part-i-cff0e3841750

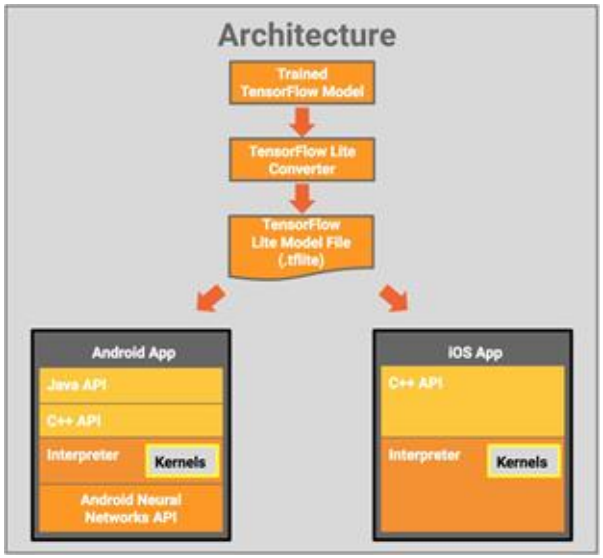
What toolkit?

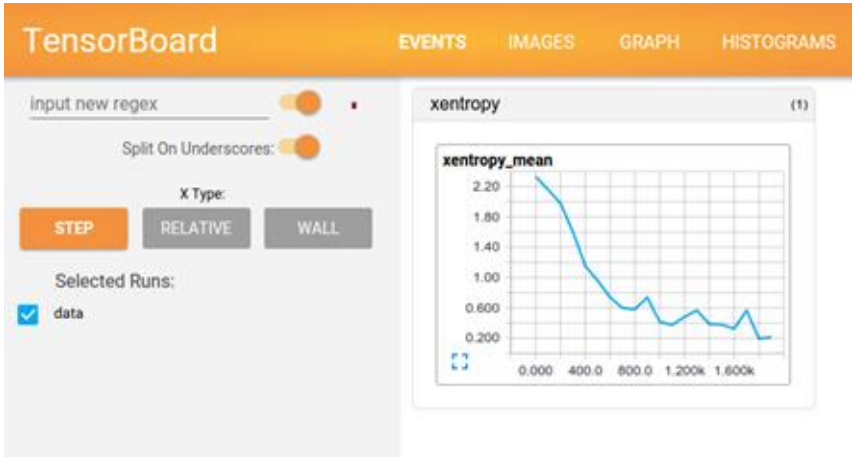


https://towardsdatascience.com/battle-of-the-deep-learning-frameworks-part-i-cff0e3841750

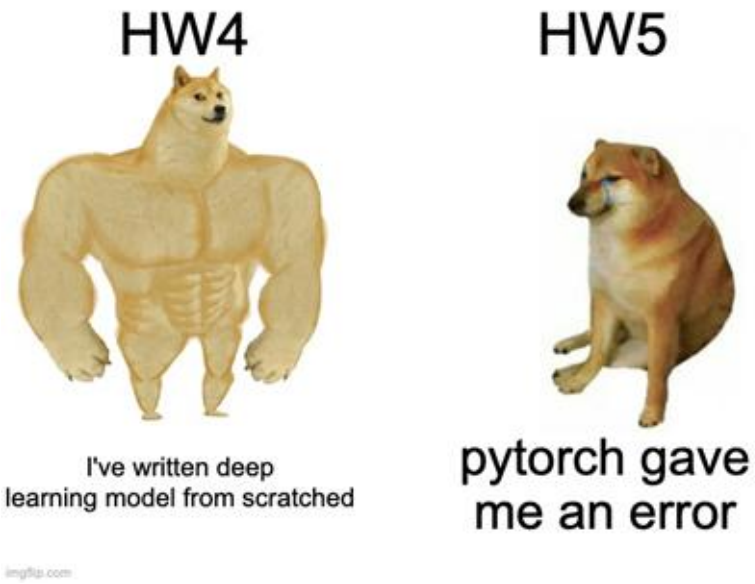
Which?

- Easiest to use and play with deep learning: Keras
- Easiest to use and tweak: pytorch
- Best tools: pytorch
- Community: pytorch





Pytorch steps

- Setting up dataloader
  - Gives minibatch
- Define a network
  - Init weights
  - Define computation graph
- Setup optimization method
  - Pick LR scheduler
  - Pick optimizer
- Training loop
  - Forward (compute Loss)
  - Backward (compute gradient and apply gradient)

- Let’s demo